

Typical Interview Algorithm Pattern

Most interview problems follow one of these patterns:

1. **Filtering** – select records that satisfy a condition (`filter()`, `WHERE`).
2. **Mapping/Transformation** – convert data into another form (`map()`).
3. **Sorting** – order elements (`sorted()`, `ORDER BY`).
4. **Grouping** – organize records into categories (`groupingBy()`, `GROUP BY`).
5. **Aggregation** – compute values such as `COUNT`, `SUM`, `AVG`, `MIN`, or `MAX`.
6. **Deduplication** – remove duplicate values (`distinct()`, `DISTINCT`).
7. **Ranking** – identify top-N or nth highest values (`sorted().skip()`, `RANK()`, `DENSE_RANK()`).
8. **Partitioning** – divide data into two groups based on a predicate (`partitioningBy()`, `CASE`).
9. **Joining** – combine related datasets (`JOIN` or in-memory mapping).
10. **Reduction** – combine all elements into a single result (`reduce()`, aggregate functions).

Mastering these core patterns enables you to solve the vast majority of Java 8 Stream API and SQL interview questions by adapting the same underlying logic to different scenarios.

This is a very broad topic. A "list of all common interview programming questions" covers hundreds of problems. Below is a structured roadmap of the **most frequently asked interview questions** and how they are solved using both **Java 8 Stream API** and **SQL**.

Java 8 Stream API vs SQL

Java 8 Stream API	SQL
Processes collections in memory	Processes data stored in databases
Functional programming	Declarative querying
Uses lambda expressions	Uses SQL statements
Works on Java objects	Works on database tables
Intermediate + Terminal operations	SELECT, WHERE, GROUP BY, HAVING, ORDER BY

Common Interview Questions

These questions are repeatedly asked in interviews at companies such as TCS, Infosys, Capgemini, Wipro, Cognizant, Accenture, IBM, Oracle, Amazon, Microsoft, and Google.

1. Find Maximum

Problem

Find the highest salary.

Logic

Start

Assume first value is maximum

Compare every element

If larger found
 update maximum

End

Time Complexity

$O(n)$

Java 8 Stream

```
employees.stream()  
    .map(Employee::getSalary)  
    .max(Integer::compare);
```

SQL

```
SELECT MAX(salary)  
FROM Employee;
```

2. Find Minimum

Logic

Compare every value.

Store smallest.

Java

```
employees.stream()  
    .map(Employee::getSalary)  
    .min(Integer::compare);
```

SQL

```
SELECT MIN(salary)  
FROM Employee;
```

3. Find Second Highest Salary

Logic

Remove duplicates

Sort descending

Skip first

Take second

Java

```
employees.stream()  
    .map(Employee::getSalary)  
    .distinct()  
    .sorted(Collections.reverseOrder())  
    .skip(1)  
    .findFirst();
```

SQL

```
SELECT MAX(salary)  
FROM Employee  
WHERE salary <  
(  
    SELECT MAX(salary)  
    FROM Employee  
);
```

4. Find Third Highest Salary

Logic

Skip first two highest salaries.

Java

```
.skip(2)  
.findFirst();
```

SQL

```
SELECT DISTINCT salary  
FROM Employee  
ORDER BY salary DESC  
LIMIT 1 OFFSET 2;
```

5. Remove Duplicate Elements

Logic

Store only unique values.

Java

```
list.stream()  
.distinct()  
.collect(Collectors.toList());
```

SQL

```
SELECT DISTINCT name  
FROM Employee;
```

6. Count Duplicate Elements

Logic

Group identical elements.

Count occurrences.

Java

```
list.stream()  
.collect(Collectors.groupingBy(  
Function.identity(),  
Collectors.counting()));
```

SQL

```
SELECT name,  
COUNT(*)  
FROM Employee  
GROUP BY name;
```

7. Find Duplicate Elements

Logic

Count each element.

Return count > 1.

Java

groupingBy()

followed by

filter(e -> e.getValue()>1)

SQL

```
SELECT name  
FROM Employee  
GROUP BY name  
HAVING COUNT(*)>1;
```

8. Count Employees Department-wise

Java

```
Collectors.groupingBy(  
Employee::getDepartment,  
Collectors.counting());
```

SQL

```
SELECT department,  
COUNT(*)  
FROM Employee  
GROUP BY department;
```

9. Highest Salary Department-wise

Java

Group

Find maximum

SQL

```
SELECT department,  
MAX(salary)  
FROM Employee  
GROUP BY department;
```

10. Average Salary Department-wise

Java

```
Collectors.averagingInt(Employee::getSalary)
```

SQL

```
SELECT department,  
AVG(salary)  
FROM Employee  
GROUP BY department;
```

11. Sum of Salaries

Java

```
Collectors.summingInt(Employee::getSalary)
```

SQL

```
SELECT SUM(salary)  
FROM Employee;
```

12. Count Total Employees

Java

```
employees.stream().count();
```

SQL

```
SELECT COUNT(*)  
FROM Employee;
```

13. Find Employees Older Than 30

Logic

Filter records.

Java

```
filter(e->e.getAge()>30)
```

SQL

```
SELECT *  
FROM Employee  
WHERE age>30;
```

14. Sort Employees

Ascending

Java

```
sorted()
```

SQL

```
ORDER BY salary ASC;
```

Descending

Java

```
sorted(reverseOrder())
```

SQL

```
ORDER BY salary DESC;
```

15. First Employee

Java

findFirst()

SQL

LIMIT 1;

16. Last Employee

Java

reduce((a,b)->b)

SQL

ORDER BY id DESC
LIMIT 1;

17. Check Any Employee Exists

Java

anyMatch()

SQL

EXISTS

18. Check All Employees Match Condition

Java

allMatch()

SQL

NOT EXISTS

19. Find Employee with Highest Salary

Java

max(Comparator.comparing(Employee::getSalary))

SQL


```
SELECT *  
FROM Employee  
ORDER BY salary DESC  
LIMIT 1;
```

20. Partition Employees

Logic

Split into two groups.

Java

```
Collectors.partitioningBy(  
e->e.getSalary()>50000);
```

SQL

```
CASE  
WHEN salary>50000  
THEN 'High'  
ELSE 'Low'  
END
```

21. Group Employees by Department

Java

```
Collectors.groupingBy(  
Employee::getDepartment)
```

SQL

```
GROUP BY department
```

22. Find Frequency of Characters

Java

Convert string

Group characters

Count

SQL

Generally not practical in standard SQL; typically handled in application code.

23. Reverse String

Java

```
StringBuilder.reverse()
```

SQL

Depends on the database (some provide a REVERSE() function).

24. Find Palindrome

Logic

Reverse

Compare

Java

```
str.equals(  
    new StringBuilder(str)  
        .reverse()  
        .toString())
```

SQL

Possible with database-specific functions but uncommon.

25. Find Even Numbers

Java

```
filter(n->n%2==0)
```

SQL

```
WHERE number%2=0;
```

26. Find Odd Numbers

Java

```
filter(n->n%2!=0)
```

SQL

```
WHERE number%2!=0;
```

27. Find Unique Elements

Java

```
distinct()
```

SQL

```
DISTINCT
```

28. Join Two Tables

Java

Nested processing or Map-based lookup.

SQL

```
INNER JOIN  
LEFT JOIN  
RIGHT JOIN  
FULL JOIN
```

29. Rank Employees

Java

Requires sorting and assigning ranks manually.

SQL

```
RANK()
```

or

DENSE_RANK()

30. Running Total

Java

Use `reduce()` or a custom accumulator.

SQL

```
SUM(salary)
OVER(
ORDER BY id)
```

Stream Operations Used

Intermediate	Terminal
<code>filter()</code>	<code>collect()</code>
<code>map()</code>	<code>count()</code>
<code>flatMap()</code>	<code>reduce()</code>
<code>distinct()</code>	<code>max()</code>
<code>sorted()</code>	<code>min()</code>
<code>peek()</code>	<code>findFirst()</code>
<code>limit()</code>	<code>findAny()</code>
<code>skip()</code>	<code>forEach()</code>

SQL Clauses Used

Clause	Purpose
SELECT	Retrieve data
WHERE	Filter rows

GROUP BY	Group records
HAVING	Filter groups
ORDER BY	Sort results
DISTINCT	Remove duplicates
JOIN	Combine tables
UNION	Merge results
EXISTS	Check existence
CASE	Conditional logic
OVER	Window functions
RANK() / DENSE_RANK()	Ranking
ROW_NUMBER()	Sequential numbering

Finding the Highest Salary Using Java 1.5, Java 8 Stream API, SQL, and Python

Finding the **highest salary** is one of the most common interview questions. Although it appears simple, interviewers use it to test your understanding of:

- Algorithms
 - Data Structures
 - Iteration
 - Comparison
 - Time Complexity
 - Collections
 - Streams
 - SQL Aggregate Functions
 - Python Built-in Functions
-

Problem Definition

Suppose we have the following Employee data.

Employee	Salary
John	50000
David	70000
Alice	60000
Robert	90000
James	80000

We need to find

Highest Salary = 90000

Mathematical Definition

Given

$S = \{50000, 70000, 60000, 90000, 80000\}$

Find

MAX(S)

Result

90000

Algorithm

The basic algorithm is identical in every programming language.

Step 1

Assume first salary is highest.

Step 2

Compare every remaining salary.

Step 3

If current salary is greater
Update highest salary.

Step 4

Repeat until end.

Step 5

Return highest salary.

Flow Diagram

Start

|

Take first salary as maximum

|

Read next salary

|

Is Current Salary > Maximum?

|

Yes ----- No

|

|

Update Maximum Ignore

|

More Employees?

|

Yes

Repeat

|

No

Print Maximum

|

End

Dry Run

Given

50000

70000

60000

90000

80000

Initial

Maximum = 50000

Compare

$70000 > 50000$

Maximum = 70000

Compare

$60000 > 70000$

No

Maximum

70000

Compare

$90000 > 70000$

Yes

Maximum = 90000

Compare

$80000 > 90000$

No

Final

Maximum = 90000

Time Complexity

$O(n)$

Every salary is visited exactly once.

Space Complexity

$O(1)$

Only one variable stores the maximum.

Java 1.5 Approach

Employee Class

```
public class Employee {  
    private int id;  
    private String name;  
    private double salary;
```

```
public Employee(int id, String name, double salary) {
    this.id = id;
    this.name = name;
    this.salary = salary;
}

public double getSalary() {
    return salary;
}

public String getName() {
    return name;
}
}
```

Logic

Take first employee salary.

Store it.

Loop through list.

If employee salary > highest

Update highest.

Finally print highest.

Java 1.5 Code

```
import java.util.*;

public class HighestSalary {

    public static void main(String[] args) {

        List<Employee> employees = new ArrayList<Employee>();

        employees.add(new Employee(1,"John",50000));
        employees.add(new Employee(2,"David",70000));
        employees.add(new Employee(3,"Alice",60000));
        employees.add(new Employee(4,"Robert",90000));
        employees.add(new Employee(5,"James",80000));

        double highest = employees.get(0).getSalary();
```

```
    for(Employee emp : employees){  
        if(emp.getSalary() > highest){  
            highest = emp.getSalary();  
        }  
    }  
    System.out.println(highest);  
}
```

Output

90000.0

Internal Working

highest = 50000

↓

70000 > 50000

highest = 70000

↓

60000 > 70000

No

↓

90000 > 70000

highest = 90000

↓

80000 > 90000

No

↓

Return 90000

Java 8 Stream API

Java 8 introduced Streams, which allow operations on collections in a declarative style.

Approach 1

Using max()

```
double highestSalary = employees.stream()
    .map(Employee::getSalary)
    .max(Double::compare)
    .get();
```

```
System.out.println(highestSalary);
```

Output

90000.0

Stream Pipeline

employees

↓

stream()

↓

map(Employee::getSalary)

↓

50000

70000

60000

90000

80000

↓

max()

↓

90000

How max() Works Internally

Internally it repeatedly compares

max = first

for each value

if(value > max)

max = value

Exactly the same algorithm.

Streams simply hide the loop.

Approach 2

Using Comparator

```
Employee emp = employees.stream()
    .max(Comparator.comparing(Employee::getSalary))
    .get();
```

```
System.out.println(emp.getName());
System.out.println(emp.getSalary());
```

Output

Robert

90000

Internal Pipeline

Employee

↓

Comparator

↓

Compare Salary

↓

Keep Larger Employee

↓

Return Employee

SQL Solution

Suppose

Employee

Id	Name	Salary
1	John	50000
2	David	70000
3	Alice	60000
4	Robert	90000
5	James	80000

Method 1

Using MAX

```
SELECT MAX(salary)
FROM Employee;
```

Output

90000

Internal SQL Algorithm

Database performs

Maximum = First Salary

Compare

Update

Compare

Update

Return Maximum

This process is optimized internally by the database engine.

Method 2

Return Employee Name

```
SELECT *  
FROM Employee  
WHERE salary =  
(  
SELECT MAX(salary)  
FROM Employee  
);
```

Output

Name	Salary
Robert	90000

Method 3

Using ORDER BY

```
SELECT *  
FROM Employee  
ORDER BY salary DESC  
LIMIT 1;
```

SQL Server

```
SELECT TOP 1 *  
FROM Employee  
ORDER BY salary DESC;
```

Oracle

```
SELECT *  
FROM Employee  
ORDER BY salary DESC  
FETCH FIRST 1 ROW ONLY;
```

Python Solution

Method 1: Manual Loop

```
employees = [  
    ("John", 50000),  
    ("David", 70000),  
    ("Alice", 60000),  
    ("Robert", 90000),  
    ("James", 80000)  
]  
  
highest = employees[0][1]  
  
for employee in employees:  
    if employee[1] > highest:  
        highest = employee[1]  
  
print(highest)
```

Output

90000

Internal Working

highest = 50000

↓

70000

highest = 70000

↓

60000

No

↓

90000

highest = 90000

↓

80000

No

Method 2: Using max()

```
employees = [  
    ("John", 50000),  
    ("David", 70000),  
    ("Alice", 60000),  
    ("Robert", 90000),  
    ("James", 80000)  
]
```

```
highest = max(employee[1] for employee in employees)
```

```
print(highest)
```

Output

90000

Method 3: Return the Employee with Highest Salary

```
employees = [  
    ("John", 50000),  
    ("David", 70000),  
    ("Alice", 60000),  
    ("Robert", 90000),
```

```
("James", 80000)
]

employee = max(employees, key=lambda emp: emp[1])

print(employee)
```

Output

```
('Robert', 90000)
```

Comparison

Language	Method	Returns
Java 1.5	Loop + if	Highest salary
Java 8	stream().max()	Highest salary or employee
SQL	MAX(salary)	Highest salary
SQL	ORDER BY salary DESC LIMIT 1	Highest-paid employee
Python	Manual loop	Highest salary
Python	max()	Highest salary or employee

Interview Questions

1. How do you find the highest salary using Java 1.5?
 2. How does Stream.max() work internally?
 3. What is the time complexity of finding the maximum salary?
 4. What happens if the employee list is empty?
 5. What is the difference between max() and sorted().findFirst() in Java Streams?
 6. How can you retrieve the employee object instead of just the salary?
 7. How does SQL's MAX() aggregate function work conceptually?
 8. When would ORDER BY salary DESC LIMIT 1 be preferable to MAX()?
 9. How does Python's max() function use the key parameter?
 10. How would you handle multiple employees who share the highest salary?
-

Summary

- The core algorithm for finding the highest salary is a **linear scan**: initialize the maximum with the first value, compare each remaining value, update when a larger value is found, and return the final maximum.
- The algorithm runs in **$O(n)$** time and uses **$O(1)$** extra space.
- Java 1.5 implements this explicitly with a loop and an if statement.
- Java 8 Stream API provides `max()` with a Comparator, but internally it still performs a linear scan.
- SQL offers optimized aggregate functions such as `MAX(salary)` and can also return the highest-paid employee using a subquery or `ORDER BY`.
- Python supports both explicit iteration and the built-in `max()` function, including a key function to compare objects by salary.

Finding the Smallest Salary Using Java 1.5, Java 8 Stream API, SQL, and Python

Finding the **smallest salary (minimum salary)** is another fundamental interview problem. It is the opposite of finding the highest salary. The only change in the algorithm is the comparison operator: instead of checking for a larger value, we check for a smaller one.

This problem is commonly asked to test your understanding of:

- Algorithms
- Iteration
- Comparison logic
- Collections
- Java Streams
- SQL Aggregate Functions
- Python Built-in Functions
- Time and Space Complexity

Problem Definition

Suppose we have the following Employee data.

Employee	Salary
John	50000
David	70000
Alice	60000

Robert	90000
James	80000

We need to find

Smallest Salary = 50000

Mathematical Definition

Given

$S = \{50000, 70000, 60000, 90000, 80000\}$

Find

$\text{MIN}(S)$

Result

50000

What is the Minimum Value?

The **minimum** (or **smallest**) value in a collection is the value that is **less than or equal to every other value** in that collection.

Example

10 30 5 80 40

Minimum = 5

Algorithm

The algorithm is identical in every programming language.

Step 1

Assume the first salary is the smallest.

Step 2

Compare every remaining salary.

Step 3

If the current salary is smaller

Update the smallest salary.

Step 4

Repeat until all salaries are processed.

Step 5

Return the smallest salary.

Flow Diagram

Start

|

Take first salary as minimum

|

Read next salary

|

Is Current Salary < Minimum?

|

Yes ----- No

|

|

Update Minimum Ignore

|

More Employees?

|

Yes

Repeat

|

No

Print Minimum

|

End

Dry Run

Input

50000

70000

60000

90000

80000

Initial

Minimum = 50000

Compare

$70000 < 50000$

No

Minimum

50000

Compare

$60000 < 50000$

No

Compare

$90000 < 50000$

No

Compare

$80000 < 50000$

No

Final Result

Minimum = 50000

Example 2

Input

90000

60000

50000

80000

70000

Initial

Minimum = 90000

Compare

$60000 < 90000$

Yes

Minimum = 60000

Compare

$50000 < 60000$

Yes

Minimum = 50000

Compare

$80000 < 50000$

No

Compare

$70000 < 50000$

No

Final

50000

Time Complexity

$O(n)$

Every salary is visited exactly once.

Space Complexity

$O(1)$

Only one variable is used to store the minimum salary.

Java 1.5 Solution

Employee Class

```
public class Employee {  
  
    private int id;  
    private String name;  
    private double salary;  
  
    public Employee(int id, String name, double salary) {  
        this.id = id;  
        this.name = name;  
        this.salary = salary;  
    }  
  
    public double getSalary() {  
        return salary;  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```

Logic

Store first salary.

Loop through every employee.

Compare current salary with minimum salary.

If current salary is smaller

Update minimum salary.

Print minimum salary.

Java 1.5 Code

```
import java.util.*;

public class SmallestSalary {

    public static void main(String[] args) {

        List<Employee> employees = new ArrayList<Employee>();

        employees.add(new Employee(1,"John",50000));
        employees.add(new Employee(2,"David",70000));
        employees.add(new Employee(3,"Alice",60000));
        employees.add(new Employee(4,"Robert",90000));
        employees.add(new Employee(5,"James",80000));

        double minimum = employees.get(0).getSalary();

        for(Employee emp : employees){

            if(emp.getSalary() < minimum){

                minimum = emp.getSalary();

            }

        }

        System.out.println(minimum);

    }

}
```

Output

50000.0

Internal Working

minimum = 50000

↓

70000 < 50000

No

↓

60000 < 50000

No

↓

90000 < 50000

No

↓

80000 < 50000

No

↓

Return 50000

Java 8 Stream API

Java 8 introduced the `min()` terminal operation.

Method 1

Using `min()`

```
double minimumSalary = employees.stream()
```

```
.map(Employee::getSalary)
.min(Double::compare)
.get();
```

```
System.out.println(minimumSalary);
```

Output

50000.0

Stream Pipeline

employees

↓

stream()

↓

map(Employee::getSalary)

↓

50000
70000
60000
90000
80000

↓

min()

↓

50000

Internal Working of min()

Internally, the Stream API performs a linear scan similar to a loop.

Conceptually:

```
minimum = firstValue;
```

```
for(each value){
```

```
    if(value < minimum){  
        minimum = value;  
    }  
}
```

The Stream API hides the iteration but follows the same algorithm.

Method 2

Return Employee Object

```
Employee employee = employees.stream()  
    .min(Comparator.comparing(Employee::getSalary))  
    .get();
```

```
System.out.println(employee.getName());  
System.out.println(employee.getSalary());
```

Output

```
John  
50000.0
```

SQL Solution

Suppose the Employee table contains

Id	Name	Salary
1	John	50000
2	David	70000
3	Alice	60000
4	Robert	90000
5	James	80000

Method 1

Using MIN()

```
SELECT MIN(salary)
FROM Employee;
```

Output

50000

Internal SQL Algorithm

Conceptually, the database engine performs:

minimum = first salary

Compare next salary

Update if smaller

Repeat

Return minimum

Although actual database implementations may use internal optimizations, the logical result is equivalent.

Method 2

Return Complete Employee Record

```
SELECT *
FROM Employee
WHERE salary =
(
    SELECT MIN(salary)
    FROM Employee
);
```

Output

Name Salary

Name Salary

John 50000

Method 3

Using ORDER BY

MySQL / PostgreSQL

```
SELECT *  
FROM Employee  
ORDER BY salary ASC  
LIMIT 1;
```

SQL Server

```
SELECT TOP 1 *  
FROM Employee  
ORDER BY salary ASC;
```

Oracle

```
SELECT *  
FROM Employee  
ORDER BY salary ASC  
FETCH FIRST 1 ROW ONLY;
```

Python Solution

Method 1

Manual Loop

```
employees = [  
    ("John",50000),  
    ("David",70000),  
    ("Alice",60000),  
    ("Robert",90000),  
    ("James",80000)  
]
```

```
minimum = employees[0][1]
```

```
for employee in employees:  
    if employee[1] < minimum:  
        minimum = employee[1]  
print(minimum)
```

Output

50000

Internal Working

minimum = 50000

↓

70000

No

↓

60000

No

↓

90000

No

↓

80000

No

↓

Return 50000

Method 2

Using min()

```
employees = [  
    ("John",50000),  
    ("David",70000),  
    ("Alice",60000),  
    ("Robert",90000),  
    ("James",80000)  
]  
  
minimum = min(employee[1] for employee in employees)  
  
print(minimum)
```

Output

50000

Method 3

Return Employee with Minimum Salary

```
employees = [  
    ("John",50000),  
    ("David",70000),  
    ("Alice",60000),  
    ("Robert",90000),  
    ("James",80000)  
]  
  
employee = min(employees, key=lambda emp: emp[1])  
  
print(employee)
```

Output

('John', 50000)

Edge Cases

1. Empty Employee List

Java 1.5

Always check before accessing the first element.

```
if (employees.isEmpty()) {  
    System.out.println("No employees available.");  
    return;  
}
```

Java 8

`min()` returns an `Optional`.

```
Optional<Employee> employee = employees.stream()  
    .min(Comparator.comparing(Employee::getSalary));
```

```
if (employee.isPresent()) {  
    System.out.println(employee.get().getSalary());  
} else {  
    System.out.println("No employees available.");  
}
```

SQL

`MIN(salary)` returns `NULL` if there are no rows.

Python

```
if employees:  
    print(min(emp[1] for emp in employees))  
else:  
    print("No employees available.")
```

2. Multiple Employees with the Same Minimum Salary

If multiple employees have the minimum salary, `MIN(salary)` returns that salary once, while a query such as:

```
SELECT *  
FROM Employee  
WHERE salary = (  
    SELECT MIN(salary)  
    FROM Employee  
);
```

returns **all employees** who earn the minimum salary.

Example

Name	Salary
John	50000
Alice	50000

Both employees are returned.

Comparison

Language	Method	Returns
Java 1.5	Loop + if	Smallest salary
Java 8	stream().min()	Smallest salary or employee
SQL	MIN(salary)	Smallest salary
SQL	ORDER BY salary ASC LIMIT 1	One employee with the smallest salary
Python	Manual loop	Smallest salary
Python	min()	Smallest salary or employee

Common Interview Questions

1. What is the algorithm for finding the minimum value in a list?
2. Why is the time complexity **O(n)**?
3. How does Stream.min() work internally?
4. What is the difference between min() and sorted().findFirst() in Java Streams?
5. How do you retrieve the employee object with the smallest salary instead of just the salary?
6. How does SQL's MIN() aggregate function differ from ORDER BY ... LIMIT 1?
7. How does Python's min() function use the key parameter?
8. How should you handle an empty employee list safely?
9. What happens when multiple employees share the same minimum salary?
10. Why is sorting the data just to find the minimum generally less efficient than a single linear scan?

Summary

- The **smallest salary** is the minimum salary in the employee collection.
- The algorithm initializes the minimum with the first salary, compares each remaining salary, updates the minimum when a smaller value is found, and returns the final result.
- The algorithm has a **time complexity of $O(n)$** and a **space complexity of $O(1)$** .
- Java 1.5 uses an explicit loop and comparison.
- Java 8 Stream API provides `min()` with a Comparator, while internally still performing a linear scan.
- SQL uses the optimized `MIN()` aggregate function or `ORDER BY salary ASC` to retrieve the minimum salary or the corresponding employee.
- Python supports both manual iteration and the built-in `min()` function, including the key parameter for finding the employee object with the smallest salary.

Finding the Second Highest Salary Using Java 1.5, Java 8 Stream API, SQL, and Python

Finding the **second highest salary** is one of the most frequently asked interview questions. It evaluates your understanding of:

- Algorithms
 - Iteration
 - Comparisons
 - Collections
 - Java Streams
 - SQL queries
 - Python built-in functions
 - Handling duplicate values
 - Time and space complexity
-

Problem Definition

Given the following employee data:

Employee	Salary
John	50000
David	70000
Alice	60000
Robert	90000
James	80000

Find the **second highest salary**.

Result:

80000

Mathematical Definition

Given

$S = \{50000, 70000, 60000, 90000, 80000\}$

Sort in descending order

90000

80000

70000

60000

50000

Second Highest

80000

What is the Second Highest Salary?

The **second highest salary** is the **largest salary that is smaller than the maximum salary**.

Suppose

10

40

25

80

60

Largest

80

Second Largest

60

Important Interview Rule

Many interviewers expect the **second highest distinct salary**.

Example

100
90
90
80

The second highest **distinct** salary is

90

Another example

100
100
90
80

The second highest **distinct** salary is

90

Algorithm (Without Sorting)

Sorting works, but it takes $O(n \log n)$ time.

A better solution scans the data once in $O(n)$ time.

Maintain two variables:

- Highest
 - Second Highest
-

Algorithm

Step 1
Initialize

highest = $-\infty$
secondHighest = $-\infty$

Step 2
Read every salary

Step 3

If salary > highest

secondHighest = highest

highest = salary

Step 4

Else if

salary > secondHighest

AND

salary != highest

Update secondHighest

Step 5

Repeat until end

Return secondHighest

Flow Diagram

Start

↓

highest = MIN_VALUE

secondHighest = MIN_VALUE

↓

Read Salary

↓

salary > highest ?

↓

Yes

↓

secondHighest = highest

highest = salary

↓

No

↓

salary > secondHighest

AND

salary != highest ?

↓

Yes

↓

secondHighest = salary

↓

Repeat

↓

Print secondHighest

↓

End

Dry Run

Input

50000

70000

60000

90000

80000

Initially

highest = $-\infty$

secondHighest = $-\infty$

Read

50000

highest = 50000

secondHighest = $-\infty$

Read

70000

secondHighest = 50000

highest = 70000

Read

60000

60000 > secondHighest

Yes

secondHighest = 60000

Read

90000

secondHighest = 70000

highest = 90000

Read

80000

80000 > secondHighest

Yes

secondHighest = 80000

Final

Highest = 90000

Second Highest = 80000

Time Complexity

$O(n)$

Space Complexity

$O(1)$

Java 1.5 Solution

Employee Class

```
public class Employee {  
  
    private int id;  
    private String name;  
    private double salary;  
  
    public Employee(int id, String name, double salary){  
        this.id = id;  
        this.name = name;  
        this.salary = salary;  
    }  
  
    public double getSalary(){  
        return salary;  
    }  
  
    public String getName(){  
        return name;  
    }  
}
```

Java 1.5 Logic

Take two variables

highest

secondHighest

Loop through employees

Compare salary

Update highest

Update secondHighest

Print secondHighest

Java 1.5 Code

```
import java.util.*;

public class SecondHighest {

    public static void main(String[] args) {

        List<Employee> employees = new ArrayList<Employee>();

        employees.add(new Employee(1,"John",50000));
        employees.add(new Employee(2,"David",70000));
        employees.add(new Employee(3,"Alice",60000));
        employees.add(new Employee(4,"Robert",90000));
        employees.add(new Employee(5,"James",80000));

        double highest = Double.MIN_VALUE;
        double secondHighest = Double.MIN_VALUE;

        for(Employee emp : employees){

            double salary = emp.getSalary();

            if(salary > highest){

                secondHighest = highest;

                highest = salary;

            }
            else if(salary > secondHighest && salary != highest){

                secondHighest = salary;

            }

        }

        System.out.println(secondHighest);

    }
}
```

Output

80000

Note: In production code, prefer initializing with `Double.NEGATIVE_INFINITY` instead of `Double.MIN_VALUE`, because `Double.MIN_VALUE` is the smallest **positive** non-zero value, not the most negative value.

Java 8 Stream API

Method 1

Distinct + Sort + Skip

```
double salary = employees.stream()
    .map(Employee::getSalary)
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst()
    .get();
```

```
System.out.println(salary);
```

Output

80000

Stream Pipeline

Employees

↓

Stream

↓

Map Salary

↓

50000

70000

60000

90000

80000

↓

Distinct

↓

Sorted Descending

↓

90000

80000

70000

60000

↓

Skip First

↓

80000

↓

findFirst()

↓

80000

Internal Working

distinct()

↓

Remove duplicate salaries

↓

sorted()

↓

Sort descending

↓

skip(1)

↓

Ignore highest salary

↓

findFirst()

↓

Return next salary

Method 2

Return Employee Object

```
Employee employee = employees.stream()
    .sorted(Comparator.comparing(Employee::getSalary).reversed())
    .skip(1)
    .findFirst()
    .get();
```

```
System.out.println(employee.getName());
System.out.println(employee.getSalary());
```

Output

```
James
80000
```

If duplicate highest salaries exist, add logic such as `distinct()` on salaries or use grouping before selecting the employee, depending on the requirement.

SQL Solution

Employee Table

Name	Salary
------	--------

John	50000
David	70000
Alice	60000
Robert	90000
James	80000

Method 1

Using MAX

```
SELECT MAX(salary)
FROM Employee
WHERE salary <
(
    SELECT MAX(salary)
    FROM Employee
);
```

Output

80000

Internal Working

Find Highest

↓

90000

↓

Ignore 90000

↓

Find Maximum

↓

80000

Method 2

Using ORDER BY

MySQL / PostgreSQL

```
SELECT DISTINCT salary
FROM Employee
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
```

SQL Server

```
SELECT DISTINCT salary
FROM Employee
ORDER BY salary DESC
OFFSET 1 ROW FETCH NEXT 1 ROW ONLY;
```

Oracle

```
SELECT salary
FROM
(
    SELECT DISTINCT salary
    FROM Employee
    ORDER BY salary DESC
)
FETCH FIRST 1 ROW OFFSET 1;
```

Method 3

Using DENSE_RANK()

```
SELECT name,
       salary
FROM
(
    SELECT name,
           salary,
           DENSE_RANK() OVER(ORDER BY salary DESC) rnk
    FROM Employee
) x
WHERE rnk = 2;
```

Output

Name Salary

James 80000

Python Solution

Method 1

Manual Algorithm

```
employees = [  
    ("John",50000),  
    ("David",70000),  
    ("Alice",60000),  
    ("Robert",90000),  
    ("James",80000)  
]
```

```
highest = float('-inf')
```

```
second = float('-inf')
```

```
for employee in employees:
```

```
    salary = employee[1]
```

```
    if salary > highest:
```

```
        second = highest
```

```
        highest = salary
```

```
    elif salary > second and salary != highest:
```

```
        second = salary
```

```
print(second)
```

Output

80000

Method 2

Sorting


```
employees = [  
    ("John",50000),  
    ("David",70000),  
    ("Alice",60000),  
    ("Robert",90000),  
    ("James",80000)  
]  
  
salaries = sorted(  
    set(emp[1] for emp in employees),  
    reverse=True  
)  
  
print(salaries[1])
```

Output

80000

Method 3

Return Employee

```
employees = [  
    ("John",50000),  
    ("David",70000),  
    ("Alice",60000),  
    ("Robert",90000),  
    ("James",80000)  
]  
  
second_salary = sorted(  
    set(emp[1] for emp in employees),  
    reverse=True  
)[1]  
  
employee = next(  
    emp for emp in employees  
    if emp[1] == second_salary  
)  
  
print(employee)
```

Output

('James', 80000)

Handling Duplicate Salaries

Example

Name	Salary
A	90000
B	90000
C	80000
D	70000

Distinct salaries

90000

80000

70000

Second highest distinct salary

80000

Edge Cases

Empty Collection

- Java 1.5: Check isEmpty().
- Java 8: findFirst() returns an Optional; handle absence safely.
- SQL: Query returns NULL if there is no second highest salary.
- Python: Check the collection length before accessing the second element.

Single Employee

Input

90000

There is **no second highest salary**.

All Salaries Equal

Input

50000
50000
50000

There is **no second highest distinct salary**.

Comparison

Language	Method	Time Complexity	Space Complexity
Java 1.5	One-pass algorithm	$O(n)$	$O(1)$
Java 8	<code>distinct()</code> → <code>sorted()</code> → <code>skip()</code>	$O(n \log n)$	$O(n)$
SQL	<code>MAX()</code> WHERE <code>salary < MAX()</code>	Depends on indexing and optimizer	DB-managed
SQL	<code>DENSE_RANK()</code>	Depends on execution plan	DB-managed
Python	One-pass algorithm	$O(n)$	$O(1)$
Python	<code>sorted(set(...))</code>	$O(n \log n)$	$O(n)$

One-Pass vs Sorting

Feature	One-Pass Algorithm	Sorting
Time Complexity	$O(n)$	$O(n \log n)$
Extra Space	$O(1)$	$O(n)$ (or implementation- dependent)
Best for Interviews	✓ Yes	Usually acceptable but less optimal
Best for Large Datasets	✓ Yes	Less efficient

Common Interview Questions

1. What is the difference between the highest salary and the second highest salary?
2. Why is the one-pass algorithm $O(n)$?
3. Why is sorting $O(n \log n)$?
4. How do you handle duplicate salaries?
5. What happens if all employees have the same salary?
6. How do you find the second highest employee object instead of only the salary?
7. Why should `Double.NEGATIVE_INFINITY` be preferred over `Double.MIN_VALUE` for initialization?
8. What is the difference between `SQL RANK()`, `DENSE_RANK()`, and `ROW_NUMBER()` for this problem?
9. How does `skip(1)` work in a Java Stream pipeline?
10. How would you modify the algorithm to find the third or nth highest distinct salary?

Summary

- The **second highest salary** is the largest **distinct** salary that is less than the maximum salary.
- The most efficient solution maintains **two variables** (`highest` and `secondHighest`) and processes the data in a **single pass**, achieving $O(n)$ time and $O(1)$ extra space.
- Java 1.5 uses explicit iteration and comparisons, while Java 8 offers a concise Stream solution using `distinct()`, `sorted()`, and `skip()`, though that approach has $O(n \log n)$ time complexity because of sorting.
- SQL solutions include a nested `MAX()` query, `ORDER BY` with `DISTINCT`, and window functions such as `DENSE_RANK()`, with `DENSE_RANK()` being especially useful when duplicate salaries are present.
- Python supports both an optimal one-pass algorithm and a simpler `sorted(set(...))` approach, depending on whether performance or code simplicity is the priority.

Finding the Third Highest Salary Using Java 1.5, Java 8 Stream API, SQL, and Python

Finding the **third highest salary** is a classic interview problem that extends the logic of finding the highest and second highest salaries. It tests your understanding of:

- Algorithms
- Iteration
- Comparison logic
- Java Collections
- Java 8 Stream API
- SQL Window Functions
- Python
- Time and Space Complexity
- Handling duplicate salaries

Problem Definition

Suppose we have the following Employee data.

Employee	Salary
John	50000
David	70000
Alice	60000
Robert	90000
James	80000

Find the **third highest distinct salary**.

Expected Result

70000

Mathematical Definition

Given

$S = \{50000, 70000, 60000, 90000, 80000\}$

Sort in descending order.

90000
80000
70000
60000
50000

Third Highest

70000

What is the Third Highest Salary?

The **third highest salary** is the third largest **distinct** salary after sorting the salaries in descending order.

Example

Input

95
80
70
60
40

Output

70

Important Interview Rule

Interviewers usually expect the **third highest distinct salary**, not simply the third row after sorting.

Example

100
100
90
80
70

Distinct salaries

100
90
80
70

Third Highest

80

Algorithm 1 (Single Pass Without Sorting)

Instead of sorting, maintain the three largest distinct salaries while scanning the data once.

Variables

highest

secondHighest

thirdHighest

Initially

highest = $-\infty$

secondHighest = $-\infty$

thirdHighest = $-\infty$

Algorithm

Step 1

Initialize

highest

secondHighest

thirdHighest

Step 2

Read each salary.

Step 3

If salary > highest

thirdHighest = secondHighest

secondHighest = highest

highest = salary

Step 4

Else if

```
salary > secondHighest
AND salary != highest
thirdHighest = secondHighest
secondHighest = salary

Step 5

Else if

salary > thirdHighest
AND salary != secondHighest
AND salary != highest
thirdHighest = salary

Step 6

Repeat until the end.

Return thirdHighest.
```

Flow

Start

↓

Read Salary

↓

salary > highest ?

↓

Yes

third = second

second = highest

highest = salary

↓

No

↓

salary > second ?

↓

Yes

third = second

second = salary

↓

No

↓

salary > third ?

↓

Yes

third = salary

↓

Repeat

↓

End

Dry Run

Input

50000

70000

60000

90000

80000

Initially

highest = $-\infty$

second = $-\infty$

third = $-\infty$

Read

50000

highest = 50000

Read

70000

highest = 70000

second = 50000

Read

60000

highest = 70000

second = 60000

third = 50000

Read

90000

highest = 90000

second = 70000

third = 60000

Read

80000

highest = 90000

second = 80000

third = 70000

Output

70000

Time Complexity

$O(n)$

Every salary is processed once.

Space Complexity

$O(1)$

Only three variables are maintained.

Java 1.5 Solution

Employee Class

```
public class Employee {  
  
    private int id;  
    private String name;  
    private double salary;  
  
    public Employee(int id, String name, double salary) {  
        this.id = id;  
        this.name = name;  
        this.salary = salary;  
    }  
  
    public double getSalary() {  
        return salary;  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```

Java 1.5 Code

```
import java.util.*;

public class ThirdHighestSalary {

    public static void main(String[] args) {

        List<Employee> employees = new ArrayList<Employee>();

        employees.add(new Employee(1,"John",50000));
        employees.add(new Employee(2,"David",70000));
        employees.add(new Employee(3,"Alice",60000));
        employees.add(new Employee(4,"Robert",90000));
        employees.add(new Employee(5,"James",80000));

        double highest = Double.NEGATIVE_INFINITY;
        double second = Double.NEGATIVE_INFINITY;
        double third = Double.NEGATIVE_INFINITY;

        for(Employee emp : employees){

            double salary = emp.getSalary();

            if(salary > highest){

                third = second;
                second = highest;
                highest = salary;

            } else if(salary > second && salary != highest){

                third = second;
                second = salary;

            } else if(salary > third &&
                salary != second &&
                salary != highest){

                third = salary;
            }
        }

        System.out.println("Third Highest Salary = " + third);
    }
}
```

Output

Third Highest Salary = 70000.0

Internal Working

highest

↓

90000

second

↓

80000

third

↓

70000

Java 8 Stream API

The easiest Stream solution is:

- Convert salaries
 - Remove duplicates
 - Sort descending
 - Skip first two
 - Return third
-

Method 1

```
double thirdSalary = employees.stream()
    .map(Employee::getSalary)
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(2)
    .findFirst()
    .orElseThrow(() ->
        new NoSuchElementException("Less than 3 distinct salaries"));
```

```
System.out.println(thirdSalary);
```

Output

70000.0

Stream Pipeline

Employee List

↓

Stream

↓

Map Salary

↓

50000

70000

60000

90000

80000

↓

Distinct

↓

90000

80000

70000

60000

50000

↓

Skip 2

↓

70000

↓

findFirst()

Method 2 (Return Employee Object)

```
List<Double> topSalaries = employees.stream()
    .map(Employee::getSalary)
    .distinct()
    .sorted(Comparator.reverseOrder())
    .toList();
```

```
double thirdSalary = topSalaries.get(2);
```

```
Employee employee = employees.stream()
    .filter(e -> e.getSalary() == thirdSalary)
    .findFirst()
    .orElse(null);
```

```
System.out.println(employee.getName());
System.out.println(employee.getSalary());
```

Output

```
David
70000.0
```

Note: Stream.toList() is available in Java 16+. In Java 8, use:

```
.collect(Collectors.toList())
```

SQL Solution

Employee Table

Name	Salary
John	50000
David	70000
Alice	60000
Robert	90000
James	80000

Method 1

Nested MAX()

```
SELECT MAX(salary)
FROM Employee
WHERE salary <
(
  SELECT MAX(salary)
  FROM Employee
  WHERE salary <
  (
    SELECT MAX(salary)
    FROM Employee
  )
);
```

Output

70000

Method 2

Using ORDER BY

MySQL / PostgreSQL

```
SELECT DISTINCT salary
FROM Employee
ORDER BY salary DESC
LIMIT 1 OFFSET 2;
```

SQL Server

```
SELECT DISTINCT salary
FROM Employee
ORDER BY salary DESC
OFFSET 2 ROWS
FETCH NEXT 1 ROW ONLY;
```

Oracle

```
SELECT salary
FROM
(
```



```
SELECT DISTINCT salary
FROM Employee
ORDER BY salary DESC
)
OFFSET 2 ROWS
FETCH NEXT 1 ROW ONLY;
```

Method 3 (Recommended)

Using DENSE_RANK()

```
SELECT name,
       salary
FROM
(
    SELECT name,
           salary,
           DENSE_RANK() OVER(ORDER BY salary DESC) AS rank_no
    FROM Employee
) x
WHERE rank_no = 3;
```

Output

Name Salary

David 70000

Python Solution

Method 1

Single-Pass Algorithm

```
employees = [
    ("John",50000),
    ("David",70000),
    ("Alice",60000),
    ("Robert",90000),
    ("James",80000)
]
```

```
highest = float('-inf')
second = float('-inf')
third = float('-inf')
```

```
for _, salary in employees:

    if salary > highest:

        third = second
        second = highest
        highest = salary

    elif salary > second and salary != highest:

        third = second
        second = salary

    elif (salary > third and
          salary != second and
          salary != highest):

        third = salary

print(third)
```

Output

70000

Method 2

Sorting

```
employees = [
    ("John",50000),
    ("David",70000),
    ("Alice",60000),
    ("Robert",90000),
    ("James",80000)
]

salaries = sorted(
    set(emp[1] for emp in employees),
    reverse=True
)

print(salaries[2])
```

Output

70000

Method 3

Return Employee

```
employees = [  
    ("John",50000),  
    ("David",70000),  
    ("Alice",60000),  
    ("Robert",90000),  
    ("James",80000)  
]  
  
third_salary = sorted(  
    set(emp[1] for emp in employees),  
    reverse=True  
)[2]  
  
employee = next(  
    emp for emp in employees  
    if emp[1] == third_salary  
)  
  
print(employee)
```

Output

('David', 70000)

Handling Duplicate Salaries

Example

Employee	Salary
A	90000
B	90000
C	80000
D	80000
E	70000

Distinct salaries

90000

80000

70000

Third Highest

70000

Edge Cases

1. Fewer Than Three Distinct Salaries

Example

90000

80000

Result

No third highest salary exists.

2. All Salaries Equal

50000

50000

50000

Result

No third highest distinct salary.

3. Empty List

Return an appropriate message or throw an exception instead of returning an invalid value.

Comparison

Language	Method	Time Complexity	Space Complexity
----------	--------	-----------------	------------------

Java 1.5	Single-pass algorithm	O(n)	O(1)
Java 8	distinct() → sorted() → skip(2)	O(n log n)	O(n)
SQL	Nested MAX()	Optimizer-dependent	DB-managed
SQL	DENSE_RANK()	Optimizer-dependent	DB-managed
Python	Single-pass algorithm	O(n)	O(1)
Python	sorted(set(...))	O(n log n)	O(n)

Single-Pass vs Sorting

Feature	Single-Pass Algorithm	Sorting Approach
Time Complexity	O(n)	O(n log n)
Extra Space	O(1)	O(n)
Best for Interviews	✓ Yes	Acceptable but less optimal
Best for Large Datasets	✓ Yes	Less efficient

Common Interview Questions

1. What is the difference between the highest, second highest, and third highest salary?
2. Why is the single-pass algorithm **O(n)**?
3. Why does the Stream API solution require sorting?
4. How do you handle duplicate salaries?
5. Why is DENSE_RANK() preferred over RANK() for finding the third highest distinct salary?
6. What should happen if fewer than three distinct salaries exist?
7. How would you modify the one-pass algorithm to find the **nth highest** distinct salary?
8. What is the difference between skip(2) and accessing index 2 after collecting a sorted list?
9. Why should Double.NEGATIVE_INFINITY be used instead of Double.MIN_VALUE for initialization?
10. Which approach would you choose for millions of employee records and why?

Summary

- The **third highest salary** is the **third largest distinct salary** in the dataset.
- The most efficient solution uses a **single-pass algorithm** with three variables (highest, secondHighest, and thirdHighest), achieving **$O(n)$** time and **$O(1)$** extra space.
- Java 1.5 implements this algorithm explicitly with loops and conditional updates.
- Java 8 Streams provide a concise solution using `distinct()`, `sorted()`, `skip(2)`, and `findFirst()`, but this incurs **$O(n \log n)$** time due to sorting.
- SQL solutions include nested `MAX()` queries, `ORDER BY` with `DISTINCT`, and the recommended `DENSE_RANK()` window function for handling duplicate salaries correctly.
- Python supports both the optimal single-pass algorithm and a simpler `sorted(set(...))` solution, making it easy to balance performance and readability.

Removing Duplicate Elements Using Java 1.5, Java 8 Stream API, SQL, and Python

Removing duplicate elements is one of the most frequently asked interview questions. It tests your understanding of:

- Algorithms
 - Collections
 - Hashing
 - Sets
 - Java Streams
 - SQL
 - Python Data Structures
 - Time Complexity
 - Space Complexity
-

1. What is a Duplicate?

Definition

A **duplicate** is an element that appears **more than once** in a collection.

Example

Input

10 20 30 20 40 10 50

Duplicates

10

20

Unique elements

10
20
30
40
50

Problem Statement

Given

10 20 30 20 40 10 50

Remove duplicate values.

Output

10 20 30 40 50

Real World Examples

Duplicate

- Employee IDs
- Email IDs
- Phone Numbers
- Aadhaar Numbers
- PAN Numbers
- Product Codes

Removing duplicates prevents

- Data inconsistency
 - Duplicate payments
 - Duplicate users
 - Duplicate records
-

Mathematical Definition

Given

S =

{10,20,30,20,40,10,50}

Unique Set

{10,20,30,40,50}

Basic Algorithm

The most efficient algorithm uses a **Hash Set** (or any set data structure that stores unique values).

Algorithm

Step 1

Create an empty Set.

Step 2

Read each element.

Step 3

Check whether it already exists.

Step 4

If it does not exist

Add it to the Set.

Step 5

Ignore duplicates.

Step 6

Return all unique values.

Flow Diagram

Start

↓

Read Element

↓

Exists in Set?

↓

Yes

↓

Ignore

↓

No

↓

Insert into Set

↓

Repeat

↓

End

Dry Run

Input

10

20

30

20

40

10

50

Initially

Set

{}

Read

10
{10}

Read

20
{10,20}

Read

30
{10,20,30}

Read

20

Already exists

Ignore

{10,20,30}

Read

40
{10,20,30,40}

Read

10

Ignore

Read

50

Final

10 20 30 40 50

Time Complexity

Using HashSet

$O(n)$

Average case.

Space Complexity

$O(n)$

The set stores the unique elements.

Java 1.5 Solution

Method 1 (Recommended)

Using LinkedHashSet

LinkedHashSet preserves insertion order while removing duplicates.

```
import java.util.*;

public class RemoveDuplicates {

    public static void main(String[] args) {

        List<Integer> numbers = Arrays.asList(
            10,20,30,20,40,10,50);

        Set<Integer> unique =
            new LinkedHashSet<Integer>(numbers);

        System.out.println(unique);

    }

}
```

Output

[10, 20, 30, 40, 50]

Internal Working

Input

10
20
30
20
40
10
50

Hash Table

10 ✓

20 ✓

30 ✓

20 Already Exists

40 ✓

10 Already Exists

50 ✓

Result

10 20 30 40 50

Method 2

Without Set

```
import java.util.*;
```

```
public class RemoveDuplicateManual {
```

```
    public static void main(String[] args) {
```

```
        List<Integer> list =  
            Arrays.asList(10,20,30,20,40,10,50);
```

```
List<Integer> result =  
    new ArrayList<Integer>();  
  
for(Integer number : list){  
    if(!result.contains(number)){  
        result.add(number);  
    }  
}  
  
System.out.println(result);  
}
```

Output

[10,20,30,40,50]

Complexity

Since contains() on an ArrayList performs a linear search, this approach has:

- **Time Complexity:** $O(n^2)$
 - **Space Complexity:** $O(n)$
-

Java 8 Stream API

The Stream API introduced the distinct() operation.

Method 1

```
List<Integer> list =  
    Arrays.asList(10,20,30,20,40,10,50);  
  
List<Integer> unique =  
    list.stream()  
        .distinct()  
        .collect(Collectors.toList());
```

```
System.out.println(unique);
```

Output

```
[10,20,30,40,50]
```

Stream Pipeline

List

↓

Stream

↓

distinct()

↓

Unique Elements

↓

Collect

↓

List

How distinct() Works Internally

Conceptually, it behaves like this:

```
Set<Object> seen = new HashSet<>();
```

```
for (Object value : stream) {
```

```
    if (seen.add(value)) {
```

```
        output(value);
```

```
    }
```

```
}
```

Since `HashSet.add()` returns:

- `true` if the value is new
- `false` if the value already exists

the Stream API automatically filters duplicates.

Remove Duplicate Employees

Suppose

Employee

101 John

102 David

101 John

103 Alice

To remove duplicate employee objects, the `Employee` class must correctly override `equals()` and `hashCode()`.

```
employees.stream()
    .distinct()
    .collect(Collectors.toList());
```

Without overriding `equals()` and `hashCode()`, two different `Employee` instances with the same data are treated as different objects.

SQL Solution

Employee Table

ID	Name
101	John
102	David
101	John
103	Alice

--	--

Method 1

Using DISTINCT

```
SELECT DISTINCT *  
FROM Employee;
```

Output

ID	Name
101	John
102	David
103	Alice

Remove Duplicate Salaries

```
SELECT DISTINCT salary  
FROM Employee;
```

Find Duplicate Values

```
SELECT salary,  
       COUNT(*)  
FROM Employee  
GROUP BY salary  
HAVING COUNT(*) > 1;
```

Output

Salary Count

50000 2

Delete Duplicate Records (Keep One)

Example (using a window function)


```
WITH CTE AS
(
    SELECT *,
        ROW_NUMBER() OVER
        (
            PARTITION BY id,name
            ORDER BY id
        ) rn
    FROM Employee
)
```

```
DELETE FROM CTE
WHERE rn > 1;
```

The ROW_NUMBER() assigns sequential numbers within each group of duplicates. Keeping rn = 1 preserves the first record and removes the rest.

Python Solution

Method 1

Using set()

```
numbers = [10,20,30,20,40,10,50]
```

```
unique = list(set(numbers))
```

```
print(unique)
```

Possible Output

```
[40,10,50,20,30]
```

Note: A Python set does **not** preserve insertion order in a way you should rely on across implementations or versions.

Method 2 (Preserve Order)

```
numbers = [10,20,30,20,40,10,50]
```

```
unique = list(dict.fromkeys(numbers))
```

```
print(unique)
```

Output

[10,20,30,40,50]

dict.fromkeys() preserves insertion order in modern Python (Python 3.7+ language guarantee).

Method 3

Manual Algorithm

```
numbers = [10,20,30,20,40,10,50]
```

```
result = []
```

```
for number in numbers:
```

```
    if number not in result:
```

```
        result.append(number)
```

```
print(result)
```

Output

[10,20,30,40,50]

Algorithm Comparison

Method	Time Complexity	Space Complexity
HashSet / Set	$O(n)$	$O(n)$
Nested Loop	$O(n^2)$	$O(1)$ (excluding output storage)
Java Stream distinct()	$O(n)$	$O(n)$
SQL DISTINCT	Depends on the execution plan (often sort- or hash-based)	DB-managed
Python set()	$O(n)$ average	$O(n)$

HashSet vs LinkedHashSet vs TreeSet

Feature	HashSet	LinkedHashSet	TreeSet
Removes Duplicates	✓	✓	✓
Maintains Insertion Order	✗	✓	✗
Keeps Elements Sorted	✗	✗	✓
Average Search Time	O(1)	O(1)	O(log n)
Allows One null	✓	✓	Depends on comparator/natural ordering

Handling Duplicate Objects

Suppose

Employee(101,"John")

Employee(101,"John")

To consider these objects equal in Java:

```
@Override
public boolean equals(Object obj) {
    // compare relevant fields
}
```

```
@Override
public int hashCode() {
    // compute hash based on the same fields
}
```

Without these methods, collections such as HashSet and Stream distinct() compare object identity rather than logical equality.

Edge Cases

Empty List

Input

[]

Output

[]

All Duplicate Values

Input

10 10 10 10

Output

10

No Duplicate Values

Input

10 20 30 40

Output

10 20 30 40

Duplicate null Values

Java

HashSet and LinkedHashSet allow one null value.

Example

null

10

null

20

Output

null

10

20

Common Interview Questions

1. What is a duplicate element?
2. Why is a HashSet commonly used to remove duplicates?
3. What is the difference between HashSet, LinkedHashSet, and TreeSet?
4. How does Stream distinct() work internally?
5. Why does distinct() require proper equals() and hashCode() implementations for custom objects?
6. Why is the manual ArrayList.contains() approach $O(n^2)$?
7. What is the difference between SQL DISTINCT and GROUP BY?
8. How do you delete duplicate rows while keeping one copy in SQL?
9. Why doesn't set() preserve insertion order in a guaranteed way across all Python implementations?
10. How would you remove duplicates from a list of employee objects based only on employee ID?

Summary

- A **duplicate** is an element that appears more than once.
- The most efficient approach is to use a **hash-based set**, which removes duplicates in **$O(n)$** average time.
- Java 1.5 commonly uses LinkedHashSet when insertion order should be preserved.
- Java 8 Stream API provides the distinct() operation, which internally tracks previously seen elements and relies on equals() and hashCode() for object comparison.
- SQL removes duplicate rows using DISTINCT and identifies duplicates with GROUP BY ... HAVING COUNT(*) > 1.
- Python offers multiple approaches, including set() for simplicity and dict.fromkeys() when preserving insertion order is important.
- For interview questions involving custom objects, understanding how equality is defined (especially in Java) is just as important as knowing the duplicate-removal algorithm itself.

Counting Duplicate Elements Using Java 1.5, Java 8 Stream API, SQL, and Python

Counting duplicate elements is one of the most common coding interview questions. It is frequently asked in interviews at companies such as TCS, Infosys, Wipro, Accenture, Capgemini, Cognizant, IBM, Deloitte, Amazon, Microsoft, Google, and many others.

This problem tests your understanding of:

- Algorithms
- Hashing

- Collections
 - Maps
 - Java 8 Stream API
 - SQL Aggregation
 - Python Dictionaries
 - Time Complexity
 - Space Complexity
-

1. What is a Duplicate Element?

Definition

A **duplicate element** is an element that appears **more than once** in a collection.

Example

Input

10
20
30
20
40
10
50
20

Occurrences

Element	Count
10	2
20	3
30	1
40	1
50	1

Duplicate Elements

10

Problem Statement

Given

10 20 30 20 40 10 50 20

Find the frequency of every element.

Expected Output

Element	Count
10	2
20	3
30	1
40	1
50	1

Duplicate elements are those whose count is greater than 1.

Real-World Examples

Counting duplicates is useful in:

- Employee IDs
 - Email IDs
 - Phone Numbers
 - Aadhaar Numbers
 - Product IDs
 - Customer Orders
 - Log Analysis
 - Word Frequency
 - Character Frequency
 - Transaction Analysis
-

Mathematical Definition

Given

S =

{10,20,30,20,40,10,50,20}

Frequency Function

$f(x)$

where

$f(10)=2$

$f(20)=3$

$f(30)=1$

$f(40)=1$

$f(50)=1$

Basic Algorithm

The most efficient algorithm uses a **HashMap** (or Dictionary).

Algorithm

Step 1

Create an empty HashMap.

Step 2

Read one element.

Step 3

Is the element already present?

Yes

Increase count.

No

Insert count = 1.

Step 4

Repeat.

Step 5

Print every element with its count.

Step 6

Elements having count > 1 are duplicates.

Flow Diagram

Start

↓

Read Element

↓

Exists in Map?

↓

No

↓

Insert Count = 1

↓

Yes

↓

Count++

↓

Repeat

↓

Print Map

↓

End

Dry Run

Input

10

20

30

20

40

10

50

20

Initially

{}

Read 10

{10=1}

Read 20

{10=1,20=1}

Read 30

{10=1,20=1,30=1}

Read 20

{10=1,20=2,30=1}

Read 40

{10=1,20=2,30=1,40=1}

Read 10

{10=2,20=2,30=1,40=1}

Read 50

{10=2,20=2,30=1,40=1,50=1}

Read 20

{10=2,20=3,30=1,40=1,50=1}

Final Result

10 = 2

20 = 3

30 = 1

40 = 1

50 = 1

Time Complexity

Using HashMap

$O(n)$

Average case.

Space Complexity

$O(n)$

Java 1.5 Solution

Method 1 (Recommended)

Using HashMap

```
import java.util.*;

public class CountDuplicates {

    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(
            10,20,30,20,40,10,50,20);

        Map<Integer,Integer> map =
            new HashMap<Integer,Integer>();

        for(Integer number : list){

            if(map.containsKey(number)){

                map.put(number,
                    map.get(number)+1);

            }else{

                map.put(number,1);

            }

        }

        System.out.println(map);

    }

}
```

Output

```
{10=2,20=3,30=1,40=1,50=1}
```

Internal Working

Input

10

20

30

20

40

10

50

20

HashMap

Key

↓

10

Value

↓

2

20

↓

3

30

↓

1

40

↓

1

50

↓

1

Print Only Duplicate Elements

```
for(Map.Entry<Integer,Integer> entry  
    : map.entrySet()){
```

```
if(entry.getValue()>1){  
    System.out.println(  
        entry.getKey()+" -> "+  
        entry.getValue());  
}  
}
```

Output

10 -> 2

20 -> 3

Java 8 Stream API

Java 8 introduced `Collectors.groupingBy()` and `Collectors.counting()`.

Count Frequency

```
import java.util.*;  
import java.util.stream.Collectors;  
  
List<Integer> list =  
    Arrays.asList(10,20,30,20,40,10,50,20);  
  
Map<Integer,Long> frequency =  
    list.stream()  
  
        .collect(Collectors.groupingBy(  
            number -> number,  
            Collectors.counting()));  
  
System.out.println(frequency);
```

Output

{10=2,20=3,30=1,40=1,50=1}

Stream Pipeline

List

↓

Stream

↓

groupingBy()

↓

Grouping

↓

counting()

↓

Frequency Map

Internal Working

Conceptually

HashMap map

```
for(each value){
```

```
    map.put(value,  
    map.getOrDefault(value,0)+1);
```

```
}
```

The Stream API performs the grouping and counting internally.

Print Duplicate Elements

```
frequency.entrySet()
```

```
    .stream()
```

```
    .filter(entry ->  
        entry.getValue()>1)
```

```
    .forEach(System.out::println);
```

Output

10=2

20=3

Count Duplicate Characters in a String

```
String text = "programming";
```

```
Map<Character,Long> map =  
    text.chars()
```

```
        .mapToObj(ch->(char)ch)
```

```
        .collect(Collectors.groupingBy(  
            ch->ch,  
            Collectors.counting()));
```

```
System.out.println(map);
```

Output

```
{p=1,r=2,o=1,g=2,a=1,m=2,i=1,n=1}
```

SQL Solution

Suppose Employee table

Name	Salary
John	50000
David	70000
Alice	60000
Robert	90000
James	70000

Count Salaries

```
SELECT salary,
```



```
COUNT(*)
FROM Employee
GROUP BY salary;
```

Output

Salary	Count
50000	1
60000	1
70000	2
90000	1

Show Only Duplicate Values

```
SELECT salary,
       COUNT(*)
FROM Employee
GROUP BY salary
HAVING COUNT(*) > 1;
```

Output

Salary	Count
70000	2

Count Duplicate Employee Names

```
SELECT name,
       COUNT(*)
FROM Employee
GROUP BY name
HAVING COUNT(*) > 1;
```

Python Solution

Method 1 (Recommended)

Using collections.Counter

```
from collections import Counter

numbers = [10,20,30,20,40,10,50,20]

frequency = Counter(numbers)

print(frequency)
```

Output

```
Counter({20:3,10:2,30:1,40:1,50:1})
```

Print Only Duplicate Elements

```
for key,value in frequency.items():

    if value>1:

        print(key,value)
```

Output

```
10 2
```

```
20 3
```

Method 2

Using Dictionary

```
numbers = [10,20,30,20,40,10,50,20]

frequency = {}

for number in numbers:

    if number in frequency:

        frequency[number]+=1

    else:

        frequency[number]=1

print(frequency)
```

Output

{10:2,20:3,30:1,40:1,50:1}

Count Characters

```
text = "programming"
```

```
frequency = {}
```

```
for ch in text:
```

```
    frequency[ch] = frequency.get(ch,0)+1
```

```
print(frequency)
```

Output

```
{'p':1,'r':2,'o':1,'g':2,'a':1,'m':2,'i':1,'n':1}
```

Algorithm Comparison

Method	Time Complexity	Space Complexity
Java HashMap	$O(n)$	$O(n)$
Java Stream groupingBy()	$O(n)$	$O(n)$
SQL GROUP BY	Depends on execution plan (hash or sort aggregation)	Database-managed
Python Counter	$O(n)$	$O(n)$
Python Dictionary	$O(n)$	$O(n)$

HashMap vs HashSet

Feature	HashMap	HashSet
Stores Key	✓	✓
Stores Value	✓	✗
Counts Frequency	✓	✗

Removes Duplicates	✗	✓
Allows Retrieval of Count	✓	✗

Handling Custom Objects

Suppose

Employee

101 John

102 David

101 John

To count duplicate employees correctly in Java, the Employee class should override both `equals()` and `hashCode()`.

Example

```
Map<Employee,Long> map =
employees.stream()
```

```
.collect(Collectors.groupingBy(
    employee -> employee,
    Collectors.counting()));
```

Without proper implementations of `equals()` and `hashCode()`, logically identical Employee objects are treated as different keys.

Edge Cases

Empty List

Input

```
[]
```

Output

```
{}
```

All Unique

Input

10 20 30

Output

10=1

20=1

30=1

All Duplicates

Input

5 5 5 5

Output

5=4

Null Values

Java HashMap supports one null key.

Example

null

10

null

Output

null=2

10=1

When using Streams, be aware that `groupingBy()` can group null values if the classifier function returns null, but downstream operations should be designed to handle them appropriately.

Common Interview Questions

1. What is the difference between removing duplicates and counting duplicates?
2. Why is HashMap preferred over HashSet for counting frequencies?
3. How does Collectors.groupingBy() work internally?
4. What is the purpose of Collectors.counting()?
5. What is the difference between SQL GROUP BY and DISTINCT?
6. Why is Counter preferred over a manual dictionary in Python?
7. How do you count duplicate characters in a string?
8. How do you count duplicate words in a sentence?
9. How would you count duplicate employee objects based on employee ID only?
10. How would you find the most frequently occurring element after counting all frequencies?

Summary

- A **duplicate element** is an element that appears more than once in a collection, while **counting duplicates** means determining the frequency of every distinct element.
- The most efficient general-purpose solution uses a **hash-based map** (HashMap in Java or dict/Counter in Python), achieving **$O(n)$** average time and **$O(n)$** extra space.
- Java 1.5 uses a HashMap with explicit iteration, whereas Java 8 simplifies the task with Collectors.groupingBy() and Collectors.counting().
- In SQL, duplicate counting is performed using GROUP BY together with the COUNT() aggregate function, and duplicate values can be filtered using HAVING COUNT(*) > 1.
- Python provides the highly optimized collections.Counter class, though the same result can be achieved manually with a dictionary.
- For custom Java objects, correct implementations of equals() and hashCode() are essential when using objects as keys in a HashMap or grouping them in Java Streams.

Finding Duplicate Elements Using Java 1.5, Java 8 Stream API, SQL, and Python

Finding duplicate elements is one of the most frequently asked interview questions. It is commonly asked in interviews for Java developers, Python developers, SQL developers, and Full Stack developers.

This problem evaluates your understanding of:

- Algorithms
- Hashing
- Collections
- Sets
- Maps
- Java 8 Stream API
- SQL Aggregation
- Python Data Structures
- Time Complexity

- Space Complexity

1. What is a Duplicate Element?

Definition

A **duplicate element** is an element that appears **more than one time** in a collection.

Example

Input

10
20
30
20
40
10
50
20

Occurrences

Element	Occurrences
10	2
20	3
30	1
40	1
50	1

Duplicate Elements

10
20

Difference Between Duplicate and Unique Elements

Element	Count	Type
10	2	Duplicate
20	3	Duplicate
30	1	Unique
40	1	Unique
50	1	Unique

Problem Statement

Given

10 20 30 20 40 10 50 20

Find only duplicate values.

Expected Output

10

20

Real-World Examples

Finding duplicate values is useful for:

- Duplicate Employee IDs
 - Duplicate Email Addresses
 - Duplicate Phone Numbers
 - Duplicate PAN Numbers
 - Duplicate Aadhaar Numbers
 - Duplicate Orders
 - Duplicate Transactions
 - Duplicate User Accounts
-

Mathematical Definition

Given

S =

{10,20,30,20,40,10,50,20}

Frequency Function

$f(x)$

Duplicate

$f(x) > 1$

Basic Algorithm

The most efficient algorithm uses:

- HashSet
 - HashMap
-

Algorithm Using HashSet

Logic

Step 1

Create an empty HashSet.

Step 2

Read each element.

Step 3

Insert into HashSet.

Step 4

If insertion fails

Element already exists.

Step 5

Store duplicate.

Step 6

Continue.

Step 7

Print duplicate elements.

Flow Diagram

Start

↓

Read Element

↓

Add to HashSet

↓

Already Exists?

↓

Yes

↓

Duplicate

↓

No

↓

Continue

↓

Repeat

↓

End

Dry Run

Input

10

20

30

20

40

10

50

20

Initially

Seen = {}

Duplicates = {}

Read

10

Seen

{10}

Read

20

Seen

{10,20}

Read

30

Seen

{10,20,30}

Read

20

Already Exists

Duplicates

{20}

Read

40

Seen

{10,20,30,40}

Read

10

Already Exists

Duplicates

{20,10}

Read

50

Read

20

Already Exists

Still

Duplicates

{20,10}

Output

10

20

Time Complexity

Using HashSet

$O(n)$

Average case.

Space Complexity

$O(n)$

Java 1.5 Solution

Method 1 (Recommended)

Using two HashSets

```
import java.util.*;

public class FindDuplicates {

    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(
            10,20,30,20,40,10,50,20);

        Set<Integer> seen = new HashSet<Integer>();

        Set<Integer> duplicates =
            new LinkedHashSet<Integer>();

        for(Integer number : list){

            if(!seen.add(number)){

                duplicates.add(number);

            }

        }

        System.out.println(duplicates);
    }
}
```

```
}  
}
```

Output

[20,10]

Note: HashSet does not preserve insertion order. If you want duplicates in the order they are first detected, use LinkedHashSet for the duplicates set as shown.

Internal Working

Seen

10 ✓

20 ✓

30 ✓

20 Already Exists

Duplicate

40 ✓

10 Already Exists

Duplicate

50 ✓

Method 2

Using HashMap

```
import java.util.*;
```

```
public class FindDuplicate {
```

```
    public static void main(String[] args){
```

```
List<Integer> list =
    Arrays.asList(10,20,30,20,40,10,50,20);

Map<Integer,Integer> map =
    new HashMap<Integer,Integer>();

for(Integer number : list){

    if(map.containsKey(number)){

        map.put(number,
            map.get(number)+1);

    }else{

        map.put(number,1);

    }

}

for(Map.Entry<Integer,Integer> entry
    : map.entrySet()){

    if(entry.getValue()>1){

        System.out.println(entry.getKey());

    }

}

}
```

Output

10

20

Java 8 Stream API

Method 1

Using groupingBy()

```
import java.util.*;
import java.util.stream.Collectors;

List<Integer> list =
    Arrays.asList(10,20,30,20,40,10,50,20);

Set<Integer> duplicates =

list.stream()

.collect(Collectors.groupingBy(
    number -> number,
    Collectors.counting()))

.entrySet()

.stream()

.filter(entry -> entry.getValue() > 1)

.map(Map.Entry::getKey)

.collect(Collectors.toSet());

System.out.println(duplicates);
```

Output

[20,10]

Stream Pipeline

List

↓

Stream

↓

Grouping

↓

Count

↓

Filter

↓

Count > 1

↓

Duplicate Elements

Method 2

Using HashSet Inside Filter

```
Set<Integer> seen = new HashSet<>();
```

```
Set<Integer> duplicates =
```

```
list.stream()
```

```
.filter(number -> !seen.add(number))
```

```
.collect(Collectors.toSet());
```

```
System.out.println(duplicates);
```

Output

[20,10]

This approach relies on a side effect (`seen.add(...)`) inside the stream pipeline. It is acceptable for simple sequential streams in interviews, but it is **not recommended** for parallel streams because stateful side effects can produce incorrect or unpredictable results.

Find Duplicate Employee Objects

Suppose

Employee

101 John

102 David

101 John

Java 8

```
Map<Employee,Long> map =  
employees.stream()  
  
.collect(Collectors.groupingBy(  
    employee -> employee,  
    Collectors.counting()));  
  
map.entrySet()  
  
.stream()  
  
.filter(entry -> entry.getValue() > 1)  
  
.forEach(System.out::println);
```

For this to work correctly, the Employee class must override equals() and hashCode().

SQL Solution

Employee Table

Name	Salary
John	50000
David	70000
Alice	60000
Robert	90000
James	70000

Find Duplicate Salaries

```
SELECT salary,  
    COUNT(*)  
FROM Employee  
GROUP BY salary  
HAVING COUNT(*) > 1;
```

Output

Salary	Count
70000	2

Find Duplicate Names

```
SELECT name,  
       COUNT(*)  
FROM Employee  
GROUP BY name  
HAVING COUNT(*) > 1;
```

Find Complete Duplicate Rows

```
SELECT id,  
       name,  
       salary,  
       COUNT(*)  
FROM Employee  
GROUP BY id,  
         name,  
         salary  
HAVING COUNT(*) > 1;
```

Output

Returns only rows that appear more than once.

Python Solution

Method 1 (Recommended)

Using Counter

```
from collections import Counter
```

```
numbers = [10,20,30,20,40,10,50,20]
```

```
frequency = Counter(numbers)
```

```
duplicates = []
```

```
for key,value in frequency.items():
```

```
    if value>1:
```

```
        duplicates.append(key)
```

```
print(duplicates)
```

Output

```
[10,20]
```

Method 2

Using Set

```
numbers = [10,20,30,20,40,10,50,20]
```

```
seen = set()
```

```
duplicates = set()
```

```
for number in numbers:
```

```
    if number in seen:
```

```
        duplicates.add(number)
```

```
    else:
```

```
        seen.add(number)
```

```
print(duplicates)
```

Output

```
{10,20}
```

Method 3

Using List Comprehension

```
numbers = [10,20,30,20,40,10,50,20]
```

```
duplicates = {
```

```

x
for x in numbers
    if numbers.count(x)>1
}

print(duplicates)

```

Output

{10,20}

Note: numbers.count(x) scans the entire list each time, making this approach **$O(n^2)$** . It is concise but not suitable for large datasets.

Algorithm Comparison

Method	Time Complexity	Space Complexity
Java HashSet	$O(n)$	$O(n)$
Java HashMap	$O(n)$	$O(n)$
Java Stream groupingBy()	$O(n)$	$O(n)$
SQL GROUP BY	Depends on execution plan (hash or sort aggregation)	Database-managed
Python Counter	$O(n)$	$O(n)$
Python set()	$O(n)$	$O(n)$
Python list.count()	$O(n^2)$	$O(n)$

HashSet vs HashMap

Feature	HashSet	HashMap
Stores Unique Values	✓	Keys only
Stores Frequency	✗	✓

Detects Duplicates	✓	✓
Removes Duplicates	✓	✗
Counts Duplicates	✗	✓

Edge Cases

Empty List

Input

[]

Output

[]

All Unique

Input

10 20 30

Output

No duplicates

All Duplicate

Input

5 5 5 5

Output

5

Null Values

Example

null

10

null

Output

null

In Java, HashSet and HashMap support one null key/value reference as appropriate, so duplicate null values can also be detected.

Common Interview Questions

1. What is a duplicate element?
2. What is the difference between finding duplicates and removing duplicates?
3. Why is HashSet an efficient choice for duplicate detection?
4. What is the difference between HashSet and HashMap?
5. How does Collectors.groupingBy() work internally?
6. Why is using mutable state inside a Stream filter() discouraged for parallel streams?
7. What is the difference between SQL DISTINCT and GROUP BY ... HAVING COUNT(*) > 1?
8. Why is Python's Counter generally preferred over repeatedly calling list.count()?
9. How do you find duplicate characters in a string?
10. How would you detect duplicate employee objects based only on employee ID?

Interview Tip

There are three related but different interview questions:

Problem	Example Output
Remove duplicates	10,20,30,40,50
Count duplicates	10=2,20=3,30=1...
Find duplicates	10,20

Be sure to clarify which one the interviewer is asking before writing code.

Summary

- A **duplicate element** is an element whose frequency is greater than one.
- The most efficient algorithms use **hash-based data structures** (HashSet for detection and HashMap for frequency counting), achieving **$O(n)$** average time complexity.
- Java 1.5 uses explicit loops with HashSet or HashMap, while Java 8 provides concise solutions using `Collectors.groupingBy()` and stream operations.
- SQL identifies duplicate values using GROUP BY together with `HAVING COUNT(*) > 1`.
- Python offers efficient approaches using `collections.Counter` or `set`, while repeated calls to `list.count()` should generally be avoided because they result in **$O(n^2)$** time complexity.
- For custom Java objects, correctly implementing `equals()` and `hashCode()` is essential for accurate duplicate detection with hash-based collections and Stream operations.

Counting Employees Department-Wise Using Java 1.5, Java 8 Stream API, SQL, and Python

Counting employees department-wise is one of the most common interview questions asked in Java, SQL, and Python interviews. It tests your understanding of:

- HashMap (Dictionary)
- Grouping
- Aggregation
- Java Collections
- Java 8 Stream API
- SQL GROUP BY
- Python Dictionaries
- Time Complexity
- Space Complexity

1. Problem Definition

Suppose a company has the following employee data.

Employee ID	Name	Department	Salary
101	John	IT	50000
102	David	HR	60000
103	Alice	IT	70000
104	Robert	Finance	90000

105	James	HR	65000
106	Peter	IT	55000
107	Maria	Finance	80000
108	Sophia	HR	75000

Find the **number of employees in each department**.

Expected Output

Department	Employee Count
IT	3
HR	3
Finance	2

2. What is Department-wise Counting?

Definition

Department-wise counting means:

Group all employees by their department and count how many employees belong to each department.

This is an example of a **Grouping and Aggregation** problem.

3. Real-World Applications

Department-wise counting is used in:

- HR Management Systems
- Payroll Systems
- Employee Analytics
- Company Dashboards
- Workforce Planning
- Organizational Reports

- Business Intelligence (BI)

Example Report

Department	Employees
IT	350
HR	42
Finance	61
Sales	290
Marketing	80

4. Mathematical Definition

Given employees

$$E = \{e_1, e_2, e_3, \dots e_n\}$$

Each employee belongs to exactly one department.

Define a function

Department(Employee)

Group employees by department.

Then calculate

Count(Department)

Example

IT

John

Alice

Peter

Count = 3

5. Algorithm

The most efficient solution uses a **HashMap**.

Algorithm

Step 1

Create an empty HashMap.

Key = Department

Value = Count

Step 2

Read one employee.

Step 3

Check department.

Step 4

If department already exists

Increase count.

Else

Insert count = 1.

Step 5

Repeat for every employee.

Step 6

Print the HashMap.

6. Flow Diagram

Start

↓

Read Employee

↓

Get Department

↓

Exists in Map?

↓

Yes

↓

Count++

↓

No

↓

Insert Count = 1

↓

Repeat

↓

Print Department Counts

↓

End

7. Dry Run

Input

John IT
David HR
Alice IT
Robert Finance
James HR
Peter IT

Maria Finance
Sophia HR

Initially

{}

Read John

IT = 1

Read David

IT = 1

HR = 1

Read Alice

IT = 2

HR = 1

Read Robert

IT = 2

HR = 1

Finance = 1

Read James

IT = 2

HR = 2

Finance = 1

Read Peter

IT = 3

HR = 2

Finance = 1

Read Maria

IT = 3

HR = 2

Finance = 2

Read Sophia

IT = 3

HR = 3

Finance = 2

Output

IT = 3

HR = 3

Finance = 2

8. Time Complexity

$O(n)$

Each employee is processed once.

9. Space Complexity

$O(k)$

where **k** is the number of distinct departments.

If there are 5 departments, only 5 map entries are stored.

10. Java 1.5 Solution

Employee Class

```
public class Employee {  
    private int id;
```

```
private String name;
private String department;
private double salary;

public Employee(int id, String name,
                String department,
                double salary){

    this.id=id;
    this.name=name;
    this.department=department;
    this.salary=salary;
}

public String getDepartment(){

    return department;

}
}
```

Java 1.5 Logic

Create HashMap

Department

↓

Count

Loop through employees

↓

Check department

↓

Increase count

↓

Print map

Java 1.5 Code

```

import java.util.*;

public class DepartmentCount {

    public static void main(String[] args) {

        List<Employee> employees =
            new ArrayList<Employee>();

        employees.add(new Employee(101,"John","IT",50000));
        employees.add(new Employee(102,"David","HR",60000));
        employees.add(new Employee(103,"Alice","IT",70000));
        employees.add(new Employee(104,"Robert","Finance",90000));
        employees.add(new Employee(105,"James","HR",65000));
        employees.add(new Employee(106,"Peter","IT",55000));
        employees.add(new Employee(107,"Maria","Finance",80000));
        employees.add(new Employee(108,"Sophia","HR",75000));

        Map<String,Integer> map =
            new HashMap<String,Integer>();

        for(Employee emp : employees){

            String dept = emp.getDepartment();

            if(map.containsKey(dept)){

                map.put(dept,map.get(dept)+1);

            }else{

                map.put(dept,1);

            }

        }

        System.out.println(map);

    }

}

```

Output

```
{IT=3, HR=3, Finance=2}
```

11. Internal Working

IT

↓

1

↓

2

↓

3

HR

↓

1

↓

2

↓

3

Finance

↓

1

↓

2

12. Java 8 Stream API

Java 8 introduced `Collectors.groupingBy()` together with `Collectors.counting()`.

Java 8 Code

```
import java.util.*;  
import java.util.stream.Collectors;
```

```
Map<String,Long> result =
```

```
employees.stream()
.collect(Collectors.groupingBy(
    Employee::getDepartment,
    Collectors.counting()
));
System.out.println(result);
```

Output

```
{IT=3, HR=3, Finance=2}
```

13. Stream Pipeline

Employee List

↓

Stream

↓

groupingBy()

↓

Department Groups

↓

counting()

↓

Result Map

14. Internal Working

Conceptually, the Stream operation performs logic equivalent to:

```
HashMap<String,Integer> map = new HashMap<>();
```

```

for(Employee emp : employees){

    String dept = emp.getDepartment();

    map.put(
        dept,
        map.getOrDefault(dept,0)+1
    );

}

```

The Stream API hides the loop and grouping logic.

15. SQL Solution

Employee Table

ID	Name	Department	Salary
101	John	IT	50000
102	David	HR	60000
103	Alice	IT	70000
104	Robert	Finance	90000
105	James	HR	65000
106	Peter	IT	55000
107	Maria	Finance	80000
108	Sophia	HR	75000

SQL Query

```

SELECT department,
    COUNT(*) AS employee_count
FROM Employee
GROUP BY department;

```

Output

Department	Employee Count
IT	3
HR	3
Finance	2

Sort by Highest Count

```
SELECT department,
       COUNT(*) AS employee_count
FROM Employee
GROUP BY department
ORDER BY employee_count DESC;
```

Departments Having More Than Two Employees

```
SELECT department,
       COUNT(*) AS employee_count
FROM Employee
GROUP BY department
HAVING COUNT(*) > 2;
```

Output

Department	Employee Count
IT	3
HR	3

16. Python Solution

Method 1 (Recommended)

Using collections.Counter

```
from collections import Counter
```

```
employees = [
    ("John", "IT"),
    ("David", "HR"),
```

```

("Alice","IT"),
("Robert","Finance"),
("James","HR"),
("Peter","IT"),
("Maria","Finance"),
("Sophia","HR")
]

department_counts = Counter(
    dept for _, dept in employees
)

print(department_counts)

```

Output

```

Counter({
  'IT':3,
  'HR':3,
  'Finance':2
})

```

Method 2

Using Dictionary

```

employees = [
    ("John","IT"),
    ("David","HR"),
    ("Alice","IT"),
    ("Robert","Finance"),
    ("James","HR"),
    ("Peter","IT"),
    ("Maria","Finance"),
    ("Sophia","HR")
]

counts = {}

for _, department in employees:

    counts[department] = counts.get(department,0)+1

print(counts)

```

Output

```

{

```

```
'IT':3,  
'HR':3,  
'Finance':2  
}
```

17. Algorithm Comparison

Method	Time Complexity	Space Complexity
Java 1.5 HashMap	$O(n)$	$O(k)$
Java 8 groupingBy()	$O(n)$	$O(k)$
SQL GROUP BY	Depends on execution plan (hash or sort aggregation)	Database-managed
Python Counter	$O(n)$	$O(k)$
Python Dictionary	$O(n)$	$O(k)$

Where:

- **n** = Number of employees
 - **k** = Number of distinct departments
-

18. groupingBy() vs counting()

Collector	Purpose	Result
groupingBy(Employee::getDepartment)	Groups employees by department	Map<String, List<Employee>>
counting()	Counts items in each group	Long
Combined	Groups and counts	Map<String, Long>

19. Edge Cases

Empty Employee List

Output

}

Single Department

Input

IT

IT

IT

Output

IT = 3

One Employee Per Department

Input

IT

HR

Finance

Output

IT = 1

HR = 1

Finance = 1

Null Department

Java example:

```
String dept = emp.getDepartment();
```

```
if(dept == null){  
    dept = "Unknown";  
}
```

This prevents null departments from appearing as keys if your business logic requires a default category.

20. Common Interview Questions

1. What is department-wise counting?
 2. Why is a HashMap suitable for counting employees by department?
 3. How does `Collectors.groupingBy()` work internally?
 4. What is the purpose of `Collectors.counting()`?
 5. What is the difference between `Collectors.groupingBy()` and SQL GROUP BY?
 6. Why does `Collectors.counting()` return a Long instead of an Integer?
 7. How would you count employees department-wise and also calculate the average salary of each department?
 8. How would you sort departments by employee count in descending order?
 9. How would you find the department with the maximum number of employees?
 10. How would you count employees by both department and designation?
-

21. Summary

- **Department-wise counting** is a grouping and aggregation problem where employees are grouped by department and the number of employees in each group is calculated.
- The most efficient approach in Java and Python uses a **hash-based map**, achieving $O(n)$ time complexity and $O(k)$ space complexity, where k is the number of distinct departments.
- Java 1.5 solves the problem using a HashMap and explicit iteration.
- Java 8 simplifies the implementation with `Collectors.groupingBy()` and `Collectors.counting()`, producing a `Map<String, Long>`.
- SQL uses the GROUP BY clause together with the COUNT() aggregate function to perform the grouping directly in the database.
- Python provides concise and efficient solutions using either `collections.Counter` or a standard dictionary.
- This problem forms the basis for many advanced interview questions, such as calculating **average salary by department**, **maximum salary by department**, **employee lists by department**, and **multi-level grouping (department and designation)**.

Finding the Highest Salary Department-Wise Using Java 1.5, Java 8 Stream API, SQL, and Python

Finding the **highest salary department-wise** is one of the most frequently asked interview questions for Java, SQL, and Python developers. It combines two important concepts:

1. **Grouping** (Group employees by department)
2. **Aggregation** (Find the maximum salary within each group)

This problem tests your understanding of:

- Algorithms
 - HashMap
 - Java Collections
 - Java 8 Stream API
 - SQL GROUP BY
 - SQL Window Functions
 - Python Dictionaries
 - Time Complexity
 - Space Complexity
-

1. Problem Definition

Suppose we have the following Employee table.

ID	Name	Department	Salary
101	John	IT	50000
102	David	HR	60000
103	Alice	IT	70000
104	Robert	Finance	90000
105	James	HR	65000
106	Peter	IT	55000
107	Maria	Finance	80000
108	Sophia	HR	75000

Find the **highest salary in each department**.

Expected Output

Department	Highest Salary
IT	70000
HR	75000

Finance	90000

2. Definition

Department-wise highest salary means:

Divide employees into groups based on their department and find the maximum salary within each group.

This is called a **Grouped Maximum Aggregation** problem.

3. Real-World Applications

This problem is used in:

- HR Analytics
- Payroll Systems
- Employee Dashboards
- Company Reports
- Department Performance Analysis
- Salary Benchmarking
- Business Intelligence (BI)

Example

Department	Highest Salary
IT	₹28,00,000
HR	₹16,00,000
Finance	₹24,00,000

4. Mathematical Definition

Let

Employees = E

Each employee belongs to a department.

For every department

Maximum Salary

MAX(Salary)

Example

IT

50000

70000

55000

Maximum

70000

5. Algorithm

The most efficient solution uses a **HashMap**.

The key is

Department

The value is

Highest Salary

Algorithm

Step 1

Create an empty HashMap.

Key = Department

Value = Highest Salary

Step 2

Read one employee.

Step 3

Get department.

Step 4

Department already exists?

No

Insert salary.

Yes

Compare

Current Salary

vs

Stored Salary

Update if current salary is larger.

Step 5

Repeat.

Step 6

Print map.

6. Flow Diagram

Start

↓

Read Employee

↓

Get Department

↓

Department Exists?

↓

No

↓

Insert Salary

↓

Yes

↓

Current Salary > Stored Salary?

↓

Yes

↓

Update Maximum

↓

Repeat

↓

Print Department Maximum

↓

End

7. Dry Run

Input

John IT 50000

David HR 60000

Alice IT 70000

Robert Finance 90000

James HR 65000

Peter IT 55000

Maria Finance 80000

Sophia HR 75000

Initially

{

Read John

IT = 50000

Read David

IT = 50000

HR = 60000

Read Alice

70000 > 50000

IT = 70000

Read Robert

Finance = 90000

Read James

65000 > 60000

HR = 65000

Read Peter

55000 > 70000

No

Read Maria

80000 > 90000

No

Read Sophia

$75000 > 65000$

Yes

HR = 75000

Final

IT = 70000

HR = 75000

Finance = 90000

8. Time Complexity

$O(n)$

Each employee is processed exactly once.

9. Space Complexity

$O(k)$

where

k

=

number of departments

10. Java 1.5 Solution

Employee Class

```
public class Employee {
```

```
    private int id;  
    private String name;
```

```

private String department;
private double salary;

public Employee(int id,
                String name,
                String department,
                double salary){

    this.id=id;
    this.name=name;
    this.department=department;
    this.salary=salary;

}

public String getDepartment(){

    return department;

}

public double getSalary(){

    return salary;

}

}

```

Java 1.5 Code

```

import java.util.*;

public class HighestSalaryDepartmentWise {

    public static void main(String[] args) {

        List<Employee> employees =
            new ArrayList<Employee>();

        employees.add(new Employee(101,"John","IT",50000));
        employees.add(new Employee(102,"David","HR",60000));
        employees.add(new Employee(103,"Alice","IT",70000));
        employees.add(new Employee(104,"Robert","Finance",90000));
        employees.add(new Employee(105,"James","HR",65000));
        employees.add(new Employee(106,"Peter","IT",55000));
        employees.add(new Employee(107,"Maria","Finance",80000));
        employees.add(new Employee(108,"Sophia","HR",75000));
    }
}

```



```

Map<String,Double> map =
    new HashMap<String,Double>();

for(Employee emp : employees){

    String dept = emp.getDepartment();

    double salary = emp.getSalary();

    if(!map.containsKey(dept)){

        map.put(dept,salary);

    }else{

        if(salary > map.get(dept)){

            map.put(dept,salary);

        }

    }

}

System.out.println(map);

}

}

```

Output

```
{IT=70000.0, HR=75000.0, Finance=90000.0}
```

11. Internal Working

IT

50000

↓

70000

↓

70000

HR

60000

↓

65000

↓

75000

Finance

90000

↓

80000

↓

90000

12. Java 8 Stream API

Java 8 introduced `Collectors.groupingBy()` together with `Collectors.maxBy()`.

Method 1 (Recommended)

```
import java.util.*;
import java.util.stream.Collectors;

Map<String, Optional<Employee>> result =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDepartment,
            Collectors.maxBy(
                Comparator.comparing(Employee::getSalary)
            )
        ));
```

Output

IT → Alice (70000)

HR → Sophia (75000)

Finance → Robert (90000)

Collectors.maxBy() returns an Optional<Employee> because a collector may conceptually operate on an empty group.

Method 2 (Department → Highest Salary)

```
Map<String, Double> result =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDepartment,
            Collectors.collectingAndThen(
                Collectors.maxBy(
                    Comparator.comparing(Employee::getSalary)
                ),
                opt -> opt.get().getSalary()
            )
        ));
```

Output

```
{IT=70000.0, HR=75000.0, Finance=90000.0}
```

13. Stream Pipeline

Employee List

↓

Stream

↓

Grouping

↓

IT

HR

Finance

↓

maxBy()

↓

Highest Employee

↓

Result Map

14. Internal Working

Conceptually, the Stream collector performs logic equivalent to:

```
Map<String, Double> map = new HashMap<>();
```

```
for (Employee emp : employees) {
```

```
    String dept = emp.getDepartment();
```

```
    double salary = emp.getSalary();
```

```
    map.merge(  
        dept,  
        salary,  
        Double::max  
    );
```

```
}
```

15. SQL Solution

Employee Table

Name	Department	Salary
John	IT	50000
David	HR	60000
Alice	IT	70000
Robert	Finance	90000
James	HR	65000
Peter	IT	55000
Maria	Finance	80000

Sophia	HR	75000
---------------	----	-------

Method 1

Highest salary by department

```
SELECT department,
       MAX(salary) AS highest_salary
FROM Employee
GROUP BY department;
```

Output

Department	Highest Salary
IT	70000
HR	75000
Finance	90000

Method 2 (Return Employee Details)

Using a window function:

```
SELECT id,
       name,
       department,
       salary
FROM (
  SELECT id,
         name,
         department,
         salary,
         ROW_NUMBER() OVER (
           PARTITION BY department
           ORDER BY salary DESC
         ) AS rn
  FROM Employee
) x
WHERE rn = 1;
```

Output

Name	Department	Salary
Alice	IT	70000
Sophia	HR	75000
Robert	Finance	90000

If multiple employees share the same highest salary and you want **all** of them, replace ROW_NUMBER() with DENSE_RANK() and filter on rank = 1.

16. Python Solution

Method 1 (Dictionary)

```
employees = [
    ("John","IT",50000),
    ("David","HR",60000),
    ("Alice","IT",70000),
    ("Robert","Finance",90000),
    ("James","HR",65000),
    ("Peter","IT",55000),
    ("Maria","Finance",80000),
    ("Sophia","HR",75000)
]
```

```
highest = {}
```

```
for _, department, salary in employees:
```

```
    if department not in highest:
```

```
        highest[department] = salary
```

```
    elif salary > highest[department]:
```

```
        highest[department] = salary
```

```
print(highest)
```

Output

```
{
'IT':70000,
'HR':75000,
'Finance':90000
}
```

```
}
```

Method 2 (Return Employee Details)

```
employees = [  
    ("John","IT",50000),  
    ("David","HR",60000),  
    ("Alice","IT",70000),  
    ("Robert","Finance",90000),  
    ("James","HR",65000),  
    ("Peter","IT",55000),  
    ("Maria","Finance",80000),  
    ("Sophia","HR",75000)  
]  
  
highest = {}  
  
for employee in employees:  
  
    name, department, salary = employee  
  
    if (department not in highest or  
        salary > highest[department][2]):  
  
        highest[department] = employee  
  
print(highest)
```

Output

```
{  
'IT':('Alice','IT',70000),  
'HR':('Sophia','HR',75000),  
'Finance':('Robert','Finance',90000)  
}
```

17. Algorithm Comparison

Method	Time Complexity	Space Complexity
Java 1.5 HashMap	O(n)	O(k)
Java 8 groupingBy() + maxBy()	O(n)	O(k)
SQL GROUP BY MAX()	Depends on execution plan (typically hash or sort)	Database-managed

	aggregation)	
SQL Window Function	Depends on execution plan	Database-managed
Python Dictionary	$O(n)$	$O(k)$

Where:

- **n** = Number of employees
- **k** = Number of distinct departments

18. Edge Cases

Empty Employee List

Result

{}

One Employee in a Department

Input

IT

John

50000

Output

IT

50000

Multiple Employees with the Same Highest Salary

Example

Department	Employee	Salary
IT	John	70000

IT	Alice	70000
IT	Peter	55000

Highest Salary

70000

- If you need **only one employee**, ROW_NUMBER() (SQL) or maxBy() (Java) is sufficient.
- If you need **all employees** with the highest salary, use DENSE_RANK() in SQL or first determine the maximum salary per department and then filter matching employees in Java or Python.

19. Common Interview Questions

1. What is department-wise maximum aggregation?
2. Why is a HashMap efficient for tracking the highest salary per department?
3. How does Collectors.groupingBy() work internally?
4. What is the difference between Collectors.maxBy() and Stream.max()?
5. Why does Collectors.maxBy() return an Optional?
6. What is the difference between SQL MAX() with GROUP BY and window functions?
7. When should ROW_NUMBER() be replaced with DENSE_RANK()?
8. How would you find both the highest-paid employee and the salary for each department?
9. How would you calculate the top three salaries in each department?
10. How would you handle employees with null departments or null salaries?

20. Summary

- **Highest salary department-wise** is a grouped aggregation problem: group employees by department and compute the maximum salary within each group.
- The most efficient Java and Python solutions process the employee list once using a **hash-based map**, achieving $O(n)$ time and $O(k)$ space complexity, where k is the number of departments.
- Java 1.5 uses a HashMap with explicit comparison logic.
- Java 8 simplifies the implementation using Collectors.groupingBy() and Collectors.maxBy(), producing a map of departments to their highest-paid employee (or highest salary).
- SQL solves the problem using GROUP BY with MAX(), while window functions such as ROW_NUMBER() or DENSE_RANK() are used when complete employee details are required.
- This pattern forms the foundation for many advanced interview problems, including **minimum salary by department**, **average salary by department**, **second-highest salary by department**, **top N salaries by department**, and **department-wise employee ranking**.

Finding the Average Salary Department-Wise Using Java 1.5, Java 8 Stream API, SQL, and Python

Finding the **average salary department-wise** is one of the most common interview questions for Java, SQL, and Python developers. It combines two important concepts:

1. **Grouping** (Group employees by department)
2. **Aggregation** (Calculate the average salary for each department)

This problem tests your understanding of:

- Algorithms
 - HashMap
 - Java Collections
 - Java 8 Stream API
 - SQL GROUP BY
 - Python Dictionaries
 - Average (Arithmetic Mean)
 - Time Complexity
 - Space Complexity
-

1. Problem Definition

Suppose we have the following Employee table.

ID	Name	Department	Salary
101	John	IT	50000
102	David	HR	60000
103	Alice	IT	70000
104	Robert	Finance	90000
105	James	HR	65000
106	Peter	IT	55000
107	Maria	Finance	80000
108	Sophia	HR	75000

Find the **average salary of each department**.

Expected Output

Department	Average Salary
IT	58,333.33
HR	66,666.67
Finance	85,000.00

2. What is Average Salary?

Definition

The **average salary** of a department is the sum of all salaries in that department divided by the number of employees in that department.

The arithmetic mean is the key concept here:

genui{"descriptive_statistics_sampling":{"type_id":"ARITHMETIC_MEAN"}}genui

Mathematically,

Average Salary = Total Salary of Department

Number of Employees

3. Real-World Applications

Department-wise average salary is used in:

- HR Analytics
- Payroll Systems
- Budget Planning
- Salary Benchmarking
- Performance Reports
- Business Intelligence Dashboards
- Workforce Cost Analysis

Example

Department	Average Salary
IT	₹18,50,000
HR	₹9,20,000
Finance	₹15,60,000

4. Mathematical Example

IT Department

Employee	Salary
John	50000
Alice	70000
Peter	55000

Total Salary

$50000 + 70000 + 55000$

$= 175000$

Number of Employees

3

Average

$175000 / 3$

$= 58333.33$

5. Algorithm

The most efficient solution uses a **HashMap**.

Instead of storing only the average, maintain two values for each department:

- Total Salary
- Employee Count

Average is calculated after processing all employees.

Algorithm

Step 1

Create two HashMaps.

Map 1

Department → Total Salary

Map 2

Department → Employee Count

Step 2

Read one employee.

Step 3

Update total salary.

Step 4

Increase employee count.

Step 5

Repeat for all employees.

Step 6

Average

=

Total Salary

/

Count

Step 7

Print averages.

6. Flow Diagram

Start

↓

Read Employee

↓

Get Department

↓

Total Salary += Salary

↓

Count++

↓

Repeat

↓

Average = Total / Count

↓

Print

↓

End

7. Dry Run

Input

John	IT	50000
------	----	-------

David	HR	60000
-------	----	-------

Alice	IT	70000
-------	----	-------

Robert Finance 90000

James HR 65000

Peter IT 55000

Maria Finance 80000

Sophia HR 75000

Totals

IT

175000

HR

200000

Finance

170000

Counts

IT

3

HR

3

Finance

2

Average

IT

$175000 / 3$

=

58333.33

HR

$200000 / 3$

=

66666.67

Finance

170000 / 2

=

85000

8. Time Complexity

$O(n)$

Each employee is processed once.

9. Space Complexity

$O(k)$

Where

- **n** = Number of employees
 - **k** = Number of distinct departments
-

10. Java 1.5 Solution

Employee Class

```
public class Employee {  
  
    private int id;  
    private String name;  
    private String department;  
    private double salary;  
  
    public Employee(int id,  
                    String name,  
                    String department,  
                    double salary){  
  
        this.id=id;  
        this.name=name;  
        this.department=department;
```



```

        this.salary=salary;
    }

    public String getDepartment(){
        return department;
    }

    public double getSalary(){
        return salary;
    }
}

```

Java 1.5 Code

```

import java.util.*;

public class AverageSalaryDepartmentWise {

    public static void main(String[] args) {

        List<Employee> employees = new ArrayList<Employee>();

        employees.add(new Employee(101,"John","IT",50000));
        employees.add(new Employee(102,"David","HR",60000));
        employees.add(new Employee(103,"Alice","IT",70000));
        employees.add(new Employee(104,"Robert","Finance",90000));
        employees.add(new Employee(105,"James","HR",65000));
        employees.add(new Employee(106,"Peter","IT",55000));
        employees.add(new Employee(107,"Maria","Finance",80000));
        employees.add(new Employee(108,"Sophia","HR",75000));

        Map<String, Double> totalSalary = new HashMap<String, Double>();
        Map<String, Integer> employeeCount = new HashMap<String, Integer>();

        for(Employee emp : employees){

            String dept = emp.getDepartment();

            if(totalSalary.containsKey(dept)){

                totalSalary.put(dept,
                    totalSalary.get(dept)+emp.getSalary());

                employeeCount.put(dept,
                    employeeCount.get(dept)+1);

            }else{

```

```

        totalSalary.put(dept, emp.getSalary());

        employeeCount.put(dept,1);

    }

}

for(String dept : totalSalary.keySet()){

    double average =
        totalSalary.get(dept) /
        employeeCount.get(dept);

    System.out.println(
        dept + " = " + average);

}

}

}

```

Output

IT = 58333.33

HR = 66666.67

Finance = 85000.00

11. Internal Working

IT

50000

↓

120000

↓

175000

↓

175000 / 3

↓

58333.33

12. Java 8 Stream API

Java 8 introduced `Collectors.averagingDouble()`.

Method 1 (Recommended)

```
import java.util.*;
import java.util.stream.Collectors;

Map<String, Double> averageSalary =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDepartment,
            Collectors.averagingDouble(
                Employee::getSalary)
        ));

System.out.println(averageSalary);
```

Output

```
{
  IT=58333.33,
  HR=66666.67,
  Finance=85000.0
}
```

13. Stream Pipeline

Employee List

↓

Stream

↓

Grouping

↓

Department

↓

averagingDouble()

↓

Average Salary

↓

Map

14. Internal Working

Conceptually, `Collectors.averagingDouble()` performs the following:

```
Map<String, Double> total = new HashMap<>();  
Map<String, Integer> count = new HashMap<>();
```

```
for(Employee emp : employees){  
    String dept = emp.getDepartment();  
  
    total.put(  
        dept,  
        total.getDefault(dept,0.0)  
        + emp.getSalary());  
  
    count.put(  
        dept,  
        count.getDefault(dept,0)  
        + 1);  
}
```

```
for(String dept : total.keySet()){  
    double average =  
        total.get(dept) /  
        count.get(dept);
```

}

The Stream API hides these implementation details.

15. SQL Solution

Employee Table

Name	Department	Salary
John	IT	50000
David	HR	60000
Alice	IT	70000
Robert	Finance	90000
James	HR	65000
Peter	IT	55000
Maria	Finance	80000
Sophia	HR	75000

Average Salary by Department

```
SELECT department,  
       AVG(salary) AS average_salary  
FROM Employee  
GROUP BY department;
```

Output

Department	Average Salary
IT	58333.33
HR	66666.67
Finance	85000.00

Departments Having Average Salary Greater Than 60000

```
SELECT department,  
       AVG(salary) AS average_salary  
FROM Employee  
GROUP BY department  
HAVING AVG(salary) > 60000;
```

Output

Department	Average Salary
------------	----------------

HR	66666.67
Finance	85000.00

Sort by Highest Average Salary

```
SELECT department,
       AVG(salary) AS average_salary
FROM Employee
GROUP BY department
ORDER BY average_salary DESC;
```

16. Python Solution

Method 1 (Recommended)

```
employees = [
    ("John", "IT", 50000),
    ("David", "HR", 60000),
    ("Alice", "IT", 70000),
    ("Robert", "Finance", 90000),
    ("James", "HR", 65000),
    ("Peter", "IT", 55000),
    ("Maria", "Finance", 80000),
    ("Sophia", "HR", 75000)
]

totals = {}
counts = {}

for _, dept, salary in employees:
    totals[dept] = totals.get(dept, 0) + salary
    counts[dept] = counts.get(dept, 0) + 1

averages = {}

for dept in totals:
    averages[dept] = totals[dept] / counts[dept]

print(averages)

Output
{
'IT': 58333.33,
'HR': 66666.67,
```

```
'Finance':85000.0  
}
```

Method 2 (Using defaultdict)

```
from collections import defaultdict
```

```
employees = [  
    ("John","IT",50000),  
    ("David","HR",60000),  
    ("Alice","IT",70000),  
    ("Robert","Finance",90000),  
    ("James","HR",65000),  
    ("Peter","IT",55000),  
    ("Maria","Finance",80000),  
    ("Sophia","HR",75000)  
]
```

```
totals = defaultdict(int)  
counts = defaultdict(int)
```

```
for _, dept, salary in employees:  
    totals[dept] += salary  
    counts[dept] += 1
```

```
averages = {  
    dept: totals[dept] / counts[dept]  
    for dept in totals  
}
```

```
print(averages)
```

17. Algorithm Comparison

Method	Time Complexity	Space Complexity
Java 1.5 HashMap	$O(n)$	$O(k)$
Java 8 groupingBy() + averagingDouble()	$O(n)$	$O(k)$
SQL GROUP BY AVG()	Depends on execution plan (typically hash or sort aggregation)	Database-managed
Python Dictionary	$O(n)$	$O(k)$
Python defaultdict	$O(n)$	$O(k)$

Where:

- **n** = Number of employees

- **k** = Number of distinct departments

18. averagingDouble() vs summarizingDouble()

Collector	Returns	Use Case
averagingDouble()	Double	Only average salary
summarizingDouble()	DoubleSummaryStatistics	Count, Sum, Min, Max, Average together

Example:

```
Map<String, DoubleSummaryStatistics> stats =  
    employees.stream()  
        .collect(Collectors.groupingBy(  
            Employee::getDepartment,  
            Collectors.summarizingDouble(Employee::getSalary)  
        ));
```

Each DoubleSummaryStatistics object provides:

- getCount()
- getSum()
- getMin()
- getMax()
- getAverage()

19. Edge Cases

Empty Employee List

Output

```
{}
```

One Employee Per Department

Department	Average
IT	50000

The average equals the employee's salary.

All Employees in One Department

Department	Average
IT	62500

Null Salary

If salary values may be null, filter them before calculating averages.

Example (Java 8):

```
employees.stream()
    .filter(e -> e.getSalary() != null)
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.averagingDouble(Employee::getSalary)
    ));
```

This assumes `getSalary()` returns a boxed `Double`; if it returns a primitive double, it cannot be null.

20. Common Interview Questions

1. What is department-wise average salary?
 2. Why do we maintain both total salary and employee count?
 3. How does `Collectors.averagingDouble()` work internally?
 4. What is the difference between `averagingDouble()` and `summarizingDouble()`?
 5. Why does SQL use `AVG()` with `GROUP BY`?
 6. How would you calculate both the average and maximum salary department-wise?
 7. How would you round the average salary to two decimal places?
 8. How would you ignore employees with missing salaries?
 9. How would you calculate the weighted average if each employee had a different weight?
 10. How would you find the department with the highest average salary?
-

21. Summary

- **Average salary department-wise** is a grouped aggregation problem where employees are grouped by department and the arithmetic mean of salaries is calculated for each group.
- The most efficient approach processes each employee exactly once, maintaining the **total salary** and **employee count** for each department, resulting in **$O(n)$** time complexity and **$O(k)$** space complexity.
- Java 1.5 implements this using two HashMaps (or an equivalent structure to maintain totals and counts).

- Java 8 simplifies the implementation with `Collectors.groupingBy()` and `Collectors.averagingDouble()`.
- SQL provides the `AVG()` aggregate function together with `GROUP BY` to calculate department-wise averages directly in the database.
- Python solutions typically use dictionaries or `defaultdict` to accumulate totals and counts before computing averages.
- This pattern is the foundation for more advanced interview questions, such as **department-wise salary statistics, highest average salary department, minimum and maximum salary by department, and salary distribution analysis.**

Finding the Sum of Salaries Using Java 1.5, Java 8 Stream API, SQL, and Python

Finding the **sum of salaries** is one of the most common interview questions in Java, SQL, and Python. It is an **aggregation** problem where multiple salary values are combined into a single total.

This question evaluates your understanding of:

- Algorithms
- Iteration
- Aggregation
- Java Collections
- Java 8 Stream API
- SQL Aggregate Functions
- Python
- Time Complexity
- Space Complexity

1. Problem Definition

Suppose we have the following employee data.

Employee	Department	Salary
John	IT	50000
David	HR	60000
Alice	IT	70000
Robert	Finance	90000
James	HR	65000

Find the **sum of all employee salaries**.

Expected Output

$50000 + 60000 + 70000 + 90000 + 65000$

$= 335000$

Total Salary

335000

2. What is Sum?

Definition

The **sum** is the total obtained by adding all values together.

Example

Numbers

10

20

30

40

Sum

$10 + 20 + 30 + 40$

$=$

100

For salaries

50000

60000

70000

Total

180000

3. Real-World Applications

Calculating the total salary is used in:

- Payroll Systems
- Company Budget Planning
- HR Analytics
- Finance Reports
- Cost Analysis
- Business Intelligence
- Salary Forecasting

Example

Department	Total Salary
IT	₹2,50,00,000
HR	₹80,00,000

4. Mathematical Definition

Given salaries

$S =$

$\{50000, 60000, 70000, 90000, 65000\}$

Total Salary

$SUM(S)$

Calculation

50000

+

60000

+

70000

+

90000

+

65000

=

335000

5. Algorithm

The algorithm is straightforward.

Maintain one variable:

totalSalary

Algorithm

Step 1

Initialize

totalSalary = 0

Step 2

Read one salary.

Step 3

Add salary to total.

Step 4

Repeat for every employee.

Step 5

Print totalSalary.

6. Flow Diagram

Start

↓

total = 0

↓

Read Salary

↓

total = total + salary

↓

More Employees?

↓

Yes

Repeat

↓

No

↓

Print Total

↓

End

7. Dry Run

Input

50000

60000

70000

90000

65000

Initially

total = 0

Read

50000
total = 50000

Read

60000
total = 110000

Read

70000
total = 180000

Read

90000
total = 270000

Read

65000
total = 335000

Output

335000

8. Time Complexity

$O(n)$

Every employee is processed exactly once.

9. Space Complexity

$O(1)$

Only one extra variable (totalSalary) is used.

10. Java 1.5 Solution

Employee Class

```
public class Employee {  
  
    private int id;  
    private String name;  
    private String department;  
    private double salary;  
  
    public Employee(int id,  
                    String name,  
                    String department,  
                    double salary){  
  
        this.id=id;  
        this.name=name;  
        this.department=department;  
        this.salary=salary;  
    }  
  
    public double getSalary(){  
  
        return salary;  
    }  
}
```

Java 1.5 Code

```
import java.util.*;  
  
public class TotalSalary {  
  
    public static void main(String[] args) {  
  
        List<Employee> employees =  
            new ArrayList<Employee>();  
  
        employees.add(new Employee(101,"John","IT",50000));  
        employees.add(new Employee(102,"David","HR",60000));  
        employees.add(new Employee(103,"Alice","IT",70000));  
        employees.add(new Employee(104,"Robert","Finance",90000));  
        employees.add(new Employee(105,"James","HR",65000));  
  
        double totalSalary = 0;  
  
        for(Employee emp : employees){
```



```
        totalSalary += emp.getSalary();
    }
    System.out.println(totalSalary);
}
}
```

Output

335000.0

11. Internal Working

total

↓

0

↓

50000

↓

110000

↓

180000

↓

270000

↓

335000

12. Java 8 Stream API

Java 8 provides multiple ways to calculate the sum.

Method 1 (Recommended)

Using mapToDouble() and sum()

```
double totalSalary =  
employees.stream()  
.mapToDouble(Employee::getSalary)  
.sum();  
System.out.println(totalSalary);
```

Output

335000.0

Method 2

Using reduce()

```
double totalSalary =  
employees.stream()  
.map(Employee::getSalary)  
.reduce(0.0, Double::sum);  
System.out.println(totalSalary);
```

Output

335000.0

How reduce() Works Internally

Conceptually:

```
double result = 0;  
for(Employee emp : employees){  
    result = result + emp.getSalary();  
}
```

reduce() repeatedly combines values into a single result.

13. Stream Pipeline

Employee List

↓

Stream

↓

Map Salary

↓

50000

60000

70000

90000

65000

↓

sum()

↓

335000

14. Internal Working

Conceptually, mapToDouble().sum() performs:

```
double total = 0;
```

```
for(Employee emp : employees){
```

```
    total += emp.getSalary();
```

```
}
```

The Stream API hides the iteration.

15. SQL Solution

Employee Table

Name	Department	Salary
John	IT	50000
David	HR	60000
Alice	IT	70000
Robert	Finance	90000
James	HR	65000

Total Salary

```
SELECT SUM(salary) AS total_salary  
FROM Employee;
```

Output

Total Salary

335000

Sum Department-Wise

```
SELECT department,  
       SUM(salary) AS total_salary  
FROM Employee  
GROUP BY department;
```

Output

Department	Total Salary
IT	120000
HR	125000
Finance	90000

Departments with Total Salary Greater Than 100000

```
SELECT department,
```

```
SUM(salary) AS total_salary
FROM Employee
GROUP BY department
HAVING SUM(salary) > 100000;
```

Output

Department	Total Salary
IT	120000
HR	125000

16. Python Solution

Method 1 (Recommended)

Using sum()

```
employees = [
    ("John","IT",50000),
    ("David","HR",60000),
    ("Alice","IT",70000),
    ("Robert","Finance",90000),
    ("James","HR",65000)
]

total_salary = sum(
    salary for _,_, salary in employees
)

print(total_salary)
```

Output

335000

Method 2

Using a Loop

```
employees = [
    ("John","IT",50000),
    ("David","HR",60000),
    ("Alice","IT",70000),
    ("Robert","Finance",90000),
    ("James","HR",65000)
]
```

```

total = 0

for _, salary in employees:

    total += salary

print(total)

```

Output

335000

Method 3

Using `functools.reduce()`

```

from functools import reduce

```

```

employees = [
    ("John", "IT", 50000),
    ("David", "HR", 60000),
    ("Alice", "IT", 70000),
    ("Robert", "Finance", 90000),
    ("James", "HR", 65000)
]

total = reduce(
    lambda acc, emp: acc + emp[2],
    employees,
    0
)

```

```

print(total)

```

Output

335000

17. Algorithm Comparison

Method	Time Complexity	Space Complexity
Java 1.5 Loop	$O(n)$	$O(1)$
Java 8 <code>mapToDouble().sum()</code>	$O(n)$	$O(1)$
Java 8 <code>reduce()</code>	$O(n)$	$O(1)$

SQL SUM()	Depends on the database execution plan	Database-managed
Python sum()	$O(n)$	$O(1)$
Python reduce()	$O(n)$	$O(1)$

Where:

- **n** = Number of employees

18. sum() vs reduce()

Feature	sum()	reduce()
Purpose	Add numeric values	Combine values into one result
Simplicity	Very simple	More flexible
Readability	Excellent	Moderate
Performance	Optimized for summation	Similar for this use case
Best Use	Numeric totals	General aggregation logic

19. Edge Cases

Empty Employee List

Java 1.5

0.0

Java 8

```
employees.stream()
    .mapToDouble(Employee::getSalary)
    .sum();
```

Returns:

0.0

SQL

```
SELECT SUM(salary)
FROM Employee;
```

If the table contains **no rows**, most SQL databases return **NULL**, not 0.

To return 0 instead:

```
SELECT COALESCE(SUM(salary), 0) AS total_salary
FROM Employee;
```

Python

```
sum([])
```

Returns:

0

One Employee

Employee	Salary
John	50000

Output

50000

Salary Value is Zero

Salary

0

50000

70000

Output

120000

Negative Salary

Although uncommon in payroll systems, the algorithm still works mathematically.

Example

50000

-5000

70000

Total

115000

20. Common Interview Questions

1. What is aggregation?
 2. What is the difference between `sum()` and `reduce()` in Java Streams?
 3. Why is the time complexity $O(n)$?
 4. How does SQL `SUM()` work internally?
 5. What happens when the employee table is empty?
 6. Why does SQL return NULL for `SUM()` on an empty table?
 7. What is the difference between `map()` and `mapToDouble()`?
 8. How would you calculate the total salary department-wise?
 9. How would you ignore employees with null salaries?
 10. How would you calculate both total and average salary in a single pass?
-

21. Summary

- **Sum of salaries** is an aggregation operation that adds all salary values into a single total.
- The most efficient algorithm uses a running total, processing each employee exactly once, resulting in **$O(n)$** time complexity and **$O(1)$** extra space.
- Java 1.5 uses a simple loop with an accumulator variable.
- Java 8 offers `mapToDouble().sum()` as the most direct solution, while `reduce()` provides a more general aggregation mechanism.
- SQL calculates totals using the `SUM()` aggregate function, with `GROUP BY` available for department-wise totals.
- Python provides the built-in `sum()` function for concise and efficient summation, while loops and `functools.reduce()` are alternative approaches.
- Understanding this pattern helps solve many related interview questions, including **sum by department**, **average salary**, **maximum salary**, **minimum salary**, and **combined statistical aggregations**.

Counting Total Employees Using Java 1.5, Java 8 Stream API, SQL, and Python

Counting the **total number of employees** is one of the simplest and most frequently asked interview questions. Although it appears straightforward, it helps interviewers evaluate your understanding of:

- Iteration

- Counting algorithms
 - Java Collections
 - Java 8 Stream API
 - SQL Aggregate Functions
 - Python Built-in Functions
 - Time Complexity
 - Space Complexity
-

1. Problem Definition

Suppose we have the following employee data.

Employee ID	Name	Department	Salary
101	John	IT	50000
102	David	HR	60000
103	Alice	IT	70000
104	Robert	Finance	90000
105	James	HR	65000

Find the **total number of employees**.

Expected Output

Total Employees = 5

2. What is Counting?

Definition

Counting is the process of determining how many elements exist in a collection.

Unlike finding the maximum or average, counting does **not** depend on the values of the elements. Every valid employee contributes exactly **1** to the final count.

Example

Employees

John
David
Alice
Robert

James

Count

5

3. Real-World Applications

Counting employees is used in:

- HR Management Systems
- Payroll Systems
- Attendance Systems
- Employee Dashboards
- Company Reports
- Business Intelligence
- Workforce Planning

Example Dashboard

Department	Employees
IT	350
HR	42
Finance	61
Sales	290

4. Mathematical Definition

Let

$$E = \{e_1, e_2, e_3, \dots, e_n\}$$

The total number of employees is

$$\text{Count}(E) = n$$

Example

Employees

John

David

Alice

Robert

James

Count

5

5. Algorithm

The algorithm is very simple.

Maintain one variable:

count

Algorithm

Step 1

Initialize

count = 0

Step 2

Read one employee.

Step 3

Increase count by 1.

Step 4

Repeat until all employees are processed.

Step 5

Print count.

6. Flow Diagram

Start

↓

count = 0

↓

Read Employee

↓

count++

↓

More Employees?

↓

Yes

Repeat

↓

No

↓

Print Count

↓

End

7. Dry Run

Input

John

David

Alice

Robert

James

Initially

count = 0

Read John

count = 1

Read David

count = 2

Read Alice

count = 3

Read Robert

count = 4

Read James

count = 5

Output

Total Employees = 5

8. Time Complexity

$O(n)$

Every employee is visited exactly once.

Note: If the collection already stores its size (such as `ArrayList.size()` in Java or `len(list)` in Python), retrieving that stored value is **$O(1)$** . The $O(n)$ algorithm above applies when you explicitly count by iterating through the collection.

9. Space Complexity

$O(1)$

Only one counter variable is used.

10. Java 1.5 Solution

Employee Class

```
public class Employee {  
  
    private int id;  
    private String name;  
    private String department;  
    private double salary;  
  
    public Employee(int id,  
                    String name,  
                    String department,  
                    double salary){  
  
        this.id=id;  
        this.name=name;  
        this.department=department;  
        this.salary=salary;  
    }  
}
```

Method 1 (Manual Counting)

```
import java.util.*;  
  
public class EmployeeCount {  
  
    public static void main(String[] args) {  
  
        List<Employee> employees =  
            new ArrayList<Employee>();  
  
        employees.add(new Employee(101,"John","IT",50000));  
        employees.add(new Employee(102,"David","HR",60000));  
        employees.add(new Employee(103,"Alice","IT",70000));  
        employees.add(new Employee(104,"Robert","Finance",90000));  
        employees.add(new Employee(105,"James","HR",65000));  
  
        int count = 0;  
  
        for(Employee emp : employees){  
  
            count++;  
  
        }  
  
        System.out.println(count);  
  
    }  
}
```

```
}
```

Output

5

Method 2 (Using size())

```
int count = employees.size();
```

```
System.out.println(count);
```

Internal Working

ArrayList maintains an internal field named size that tracks how many elements are stored. Calling size() simply returns this field, making it an **O(1)** operation.

11. Internal Working

count

↓

0

↓

1

↓

2

↓

3

↓

4

↓

5

12. Java 8 Stream API

Java 8 introduced the `count()` terminal operation.

Method 1 (Recommended)

```
long totalEmployees =  
employees.stream()  
.count();  
System.out.println(totalEmployees);
```

Output

5

Method 2 (Filtering Example)

Count only IT employees.

```
long totalITEmployees =  
employees.stream()  
.filter(emp ->  
    "IT".equals(emp.getDepartment()))  
.count();  
System.out.println(totalITEmployees);
```

Output

2

13. Stream Pipeline

Employee List

↓

Stream

↓

```
count()
```

↓

```
5
```

14. Internal Working

Conceptually,

```
long count = 0;
```

```
for(Employee employee : employees){
```

```
    count++;
```

```
}
```

The Stream API hides the loop and counting logic.

15. SQL Solution

Employee Table

Name	Department	Salary
John	IT	50000
David	HR	60000
Alice	IT	70000
Robert	Finance	90000
James	HR	65000

Total Employees

```
SELECT COUNT(*) AS total_employees  
FROM Employee;
```

Output

Total Employees
5

Count Department-Wise

```
SELECT department,  
       COUNT(*) AS total_employees  
FROM Employee  
GROUP BY department;
```

Output

Department	Employees
IT	2
HR	2
Finance	1

Departments Having More Than One Employee

```
SELECT department,  
       COUNT(*) AS total_employees  
FROM Employee  
GROUP BY department  
HAVING COUNT(*) > 1;
```

Output

Department	Employees
IT	2
HR	2

16. Python Solution

Method 1 (Recommended)

Using len()

```
employees = [  
    ("John","IT",50000),  
    ("David","HR",60000),  
    ("Alice","IT",70000),  
    ("Robert","Finance",90000),  
    ("James","HR",65000)  
]
```

```
print(len(employees))
```

Output

5

Internal Working

Python lists maintain their current length internally, so `len(employees)` returns the stored size in **O(1)** time.

Method 2 (Manual Counting)

```
employees = [  
    ("John","IT",50000),  
    ("David","HR",60000),  
    ("Alice","IT",70000),  
    ("Robert","Finance",90000),  
    ("James","HR",65000)  
]
```

```
count = 0
```

```
for employee in employees:
```

```
    count += 1
```

```
print(count)
```

Output

5

Method 3

Using `sum()`

```
employees = [  
    ("John","IT",50000),  
    ("David","HR",60000),  
    ("Alice","IT",70000),  
    ("Robert","Finance",90000),  
    ("James","HR",65000)  
]
```

```
count = sum(1 for _ in employees)
```

```
print(count)
```

Output

17. Algorithm Comparison

Method	Time Complexity	Space Complexity
Java Loop	$O(n)$	$O(1)$
Java size()	$O(1)$	$O(1)$
Java Stream count()	$O(n)^*$	$O(1)$
SQL COUNT(*)	Depends on the database engine and execution plan	Database-managed
Python len()	$O(1)$	$O(1)$
Python Loop	$O(n)$	$O(1)$

*For a general stream pipeline, count() traverses the stream. For some sized sources, the JDK may optimize counting internally.

18. size() vs count()

Feature	size()	count()
Available On	Collections	Streams
Time Complexity	$O(1)$ for ArrayList	Usually $O(n)$ over a stream
Returns	int	long
Requires Stream	No	Yes
Supports Filtering	No	Yes

19. Edge Cases

Empty Employee List

Java

```
employees.size()
```

Returns

0

Java Stream

```
employees.stream().count()
```

Returns

0

SQL

```
SELECT COUNT(*)  
FROM Employee;
```

Returns

0

Python

```
len([])
```

Returns

0

One Employee

Employee

John

Output

1

Null Employee References

If a collection contains null elements, a simple loop or `count()` still counts them as elements.

Example

John

null

David

Result

3

If you want to count only valid employee objects:

Java

```
long count = employees.stream()
    .filter(Objects::nonNull)
    .count();
```

Python

```
count = sum(1 for e in employees if e is not None)
```

20. Common Interview Questions

1. What is the difference between `size()` and `count()`?
 2. Why does `Stream.count()` return `long` instead of `int`?
 3. Why is `ArrayList.size()` $O(1)$?
 4. How does SQL `COUNT(*)` differ from `COUNT(column)`?
 5. How would you count only employees in a specific department?
 6. How would you count employees whose salary is greater than ₹50,000?
 7. How would you count distinct departments?
 8. How would you count non-null employee records?
 9. How would you count employees department-wise and designation-wise?
 10. How would you count active versus inactive employees?
-

21. Summary

- **Counting total employees** determines the number of employee records in a collection or table.
- The manual counting algorithm initializes a counter to zero and increments it once for each employee, resulting in $O(n)$ time and $O(1)$ extra space.
- Java 1.5 supports both manual iteration and the more efficient `Collection.size()` method for collections that maintain their size internally.
- Java 8 Stream API provides the `count()` terminal operation, which is especially useful after filtering or other stream operations.
- SQL uses the `COUNT(*)` aggregate function to count rows and `GROUP BY` for grouped counts.
- Python offers the efficient built-in `len()` function for lists, while loops and `sum()` with a generator provide alternative counting approaches.
- Understanding this basic counting pattern is essential for solving more advanced interview questions such as **department-wise employee count**, **filtered counts**, **distinct counts**, and **aggregate reporting**.

Finding Employees Older Than 30 Using Java 1.5, Java 8 Stream API, SQL, and Python

Finding employees **older than 30 years** is one of the most common interview questions. It is a **filtering** problem, where only the records satisfying a given condition are returned.

This question evaluates your understanding of:

- Filtering Algorithms
 - Conditional Statements
 - Iteration
 - Java Collections
 - Java 8 Stream API
 - SQL WHERE Clause
 - Python List Filtering
 - Time Complexity
 - Space Complexity
-

1. Problem Definition

Suppose we have the following employee data.

ID	Name	Age	Department	Salary
101	John	25	IT	50000
102	David	35	HR	60000
103	Alice	32	IT	70000
104	Robert	45	Finance	90000
105	James	28	HR	65000
106	Peter	38	IT	55000

Find all employees whose **age is greater than 30**.

Expected Output

Name	Age
David	35
Alice	32
Robert	45
Peter	38

2. What is Filtering?

Definition

Filtering is the process of selecting only those records that satisfy a specific condition while ignoring all other records.

Example

Numbers

10
25
35
18
50

Condition

Number > 30

Output

35
50

The same idea applies to employees.

3. Real-World Applications

Filtering is used in:

- HR Systems
- Employee Search
- Banking
- Insurance
- Payroll
- Reporting
- Data Analytics
- Business Intelligence

Examples

- Employees older than 30
 - Salary greater than ₹50,000
 - Employees from IT
 - Employees with experience greater than 5 years
-

4. Mathematical Definition

Let

Employees = E

Each employee has an age.

Define

Age(Employee)

Condition

Age(Employee) > 30

Return

All employees satisfying the condition.

5. Algorithm

The algorithm is simple.

Loop through every employee.

Check age.

If age is greater than 30,

Print or store the employee.

Algorithm

Step 1

Read employee.

Step 2

Get age.

Step 3

Age > 30 ?

Yes

Return employee.

No

Ignore employee.

Step 4

Repeat.

Step 5

End.

6. Flow Diagram

Start

↓

Read Employee

↓

Age > 30 ?

↓

Yes

↓

Print Employee

↓

No

↓

Ignore

↓

Repeat

↓

End

7. Dry Run

Input

John 25

David 35

Alice 32

Robert 45

James 28

Peter 38

Read John

$25 > 30$

No

Read David

$35 > 30$

Yes

Output

David

Read Alice

$32 > 30$

Yes

Read Robert

$45 > 30$

Yes

Read James

$28 > 30$

No

Read Peter

$38 > 30$

Yes

Final Output

David

Alice

Robert

Peter

8. Time Complexity

$O(n)$

Every employee is checked once.

9. Space Complexity

If employees are printed immediately

$O(1)$

If matching employees are stored in another collection

$O(m)$

where **m** is the number of employees matching the condition.

10. Java 1.5 Solution

Employee Class

```
public class Employee {  
  
    private int id;  
    private String name;  
    private int age;  
    private String department;  
    private double salary;  
  
    public Employee(int id,  
                    String name,  
                    int age,  
                    String department,  
                    double salary){  
  
        this.id=id;  
        this.name=name;  
        this.age=age;  
        this.department=department;  
        this.salary=salary;  
    }  
  
    public int getAge(){  
        return age;  
    }  
  
    public String getName(){  
        return name;  
    }  
}
```

Java 1.5 Code

```
import java.util.*;  
  
public class EmployeesOlderThan30 {  
  
    public static void main(String[] args){  
  
        List<Employee> employees =  
            new ArrayList<Employee>();  
  
        employees.add(new Employee(101,"John",25,"IT",50000));  
        employees.add(new Employee(102,"David",35,"HR",60000));  
        employees.add(new Employee(103,"Alice",32,"IT",70000));  
        employees.add(new Employee(104,"Robert",45,"Finance",90000));  
        employees.add(new Employee(105,"James",28,"HR",65000));  
    }  
}
```

```
employees.add(new Employee(106,"Peter",38,"IT",55000));  
for(Employee emp : employees){  
    if(emp.getAge() > 30){  
        System.out.println(emp.getName());  
    }  
}  
}
```

Output

David

Alice

Robert

Peter

11. Internal Working

John

25

↓

No

David

35

↓

Yes

Alice

32

↓

Yes

Robert

45

↓

Yes

James

28

↓

No

Peter

38

↓

Yes

12. Java 8 Stream API

Java 8 introduced the `filter()` intermediate operation.

Method 1 (Recommended)

```
employees.stream()
```

```
.filter(emp -> emp.getAge() > 30)
```



```
.forEach(emp ->
    System.out.println(emp.getName()));
```

Output

David

Alice

Robert

Peter

Method 2

Collect the filtered employees.

```
List<Employee> result =
employees.stream()
.filter(emp -> emp.getAge() > 30)
.collect(Collectors.toList());
```

13. Stream Pipeline

Employee List

↓

Stream

↓

filter()

↓

Age > 30

↓

Matching Employees

↓

14. Internal Working

Conceptually, filter() performs:

```
List<Employee> result = new ArrayList<>();
```

```
for(Employee emp : employees){
```

```
    if(emp.getAge() > 30){
```

```
        result.add(emp);
```

```
    }
```

```
}
```

The Stream API hides the loop but applies the predicate to every employee.

15. SQL Solution

Employee Table

Name	Age	Salary
John	25	50000
David	35	60000
Alice	32	70000
Robert	45	90000
James	28	65000
Peter	38	55000

Employees Older Than 30

```
SELECT *  
FROM Employee  
WHERE age > 30;
```

Output

Name	Age
David	35
Alice	32
Robert	45

Peter	38
--------------	----

Employees Older Than 30 Ordered by Age

```
SELECT *  
FROM Employee  
WHERE age > 30  
ORDER BY age DESC;
```

Count Employees Older Than 30

```
SELECT COUNT(*)  
FROM Employee  
WHERE age > 30;
```

Output

4

16. Python Solution

Method 1 (Recommended)

Using a loop

```
employees = [  
    ("John",25),  
    ("David",35),  
    ("Alice",32),  
    ("Robert",45),  
    ("James",28),  
    ("Peter",38)  
]
```

```
for name, age in employees:
```

```
    if age > 30:
```

```
        print(name)
```

Output

David
Alice
Robert

Peter

Method 2

Using List Comprehension

```
employees = [  
    ("John",25),  
    ("David",35),  
    ("Alice",32),  
    ("Robert",45),  
    ("James",28),  
    ("Peter",38)  
]  
  
result = [  
    employee  
    for employee in employees  
    if employee[1] > 30  
]  
  
print(result)
```

Output

```
[  
( 'David',35),  
( 'Alice',32),  
( 'Robert',45),  
( 'Peter',38)  
]
```

Method 3

Using filter()

```
employees = [  
    ("John",25),  
    ("David",35),  
    ("Alice",32),  
    ("Robert",45),  
    ("James",28),  
    ("Peter",38)
```

```

]

result = filter(
    lambda emp: emp[1] > 30,
    employees
)

print(list(result))

```

17. Algorithm Comparison

Method	Time Complexity	Space Complexity
Java Loop	O(n)	O(1) or O(m)
Java Stream filter()	O(n)	O(1) or O(m)
SQL WHERE	Depends on indexing and query plan	Database-managed
Python Loop	O(n)	O(1) or O(m)
Python List Comprehension	O(n)	O(m)

Where:

- **n** = Total employees
 - **m** = Employees older than 30
-

18. filter() vs map()

Feature	filter()	map()
Purpose	Select matching elements	Transform elements
Number of Output Elements	Less than or equal to input	Same as input
Returns	Matching records	Modified records
Java Stream Type	Intermediate Operation	Intermediate Operation

Example

```

employees.stream()

.filter(emp -> emp.getAge() > 30)

.map(Employee::getName)

.forEach(System.out::println);

```

This filters employees first, then transforms each employee into their name.

19. Edge Cases

Empty Employee List

Output

No employees found.

(or an empty list if returning results)

All Employees Younger Than or Equal to 30

Input

Name	Age
John	25
James	28
Anna	30

Output

No matching employees.

All Employees Older Than 30

Input

Name	Age
David	35
Robert	45

Output

David
Robert

Age Exactly 30

Condition

Age > 30

Employee

Name	Age
Alice	30

Result

Not selected

If the requirement is **30 or older**, use:

Age >= 30

20. Common Interview Questions

1. What is filtering?
 2. What is the difference between filtering and searching?
 3. How does Java Stream filter() work internally?
 4. What is the difference between filter() and map()?
 5. Why is the time complexity O(n)?
 6. How does SQL WHERE differ from HAVING?
 7. How would you find employees between 30 and 40 years old?
 8. How would you find employees older than 30 in the IT department?
 9. How would you count employees older than 30?
 10. How would you sort employees older than 30 by salary?
-

21. Summary

- Finding employees **older than 30** is a **filtering problem** where only employees satisfying age > 30 are selected.
- The core algorithm iterates through each employee, evaluates the condition, and returns or prints matching records.
- The algorithm runs in **O(n)** time because every employee is examined once.
- Java 1.5 uses a simple for loop with an if statement.
- Java 8 Stream API uses filter() to express the same logic declaratively and can be combined with operations such as map(), sorted(), count(), and collect().
- SQL uses the WHERE clause to filter rows before any grouping or aggregation.
- Python supports filtering through loops, list comprehensions, and the built-in filter() function.
- This filtering pattern is the basis for many interview problems, such as **employees with salary greater than ₹50,000, employees in a specific department, employees with more than five years of experience, and employees matching multiple conditions.**

Sorting Employees Using Java 1.5, Java 8 Stream API, SQL, and Python

Sorting employees is one of the most frequently asked interview questions in Java, SQL, and Python interviews. It tests your understanding of:

- Sorting Algorithms
 - Comparable and Comparator
 - Java Collections Framework
 - Java 8 Stream API
 - SQL ORDER BY
 - Python Sorting
 - Time Complexity
 - Space Complexity
-

1. What is Sorting?

Definition

Sorting is the process of arranging data in a specific order.

The order can be:

- Ascending (Smallest to Largest)
- Descending (Largest to Smallest)
- Alphabetical (A to Z)
- Reverse Alphabetical (Z to A)

Example

Before Sorting

70
20
90
40
10

Ascending

10
20
40
70
90

Descending

90
70
40
20
10

2. Employee Sorting

Suppose we have the following employees.

ID	Name	Age	Department	Salary
101	John	25	IT	50000
102	David	35	HR	60000
103	Alice	32	IT	70000
104	Robert	45	Finance	90000
105	James	28	HR	65000

Sort by Salary

Ascending

Name	Salary
John	50000
David	60000
James	65000
Alice	70000
Robert	90000

Sort by Name

Ascending

Name
Alice
David
James
John
Robert

3. Types of Sorting

Employees can be sorted by:

- ID
- Name
- Age
- Salary
- Department
- Joining Date
- Experience

Interviewers often ask:

- Sort employees by salary
- Sort employees by age
- Sort employees by name
- Sort by salary descending
- Sort by department then salary

4. Sorting Algorithms

The programming languages and databases provide built-in sorting implementations, but interviewers expect you to understand the underlying algorithms.

Common algorithms include:

Algorithm	Average Time	Worst Time	Stable
Bubble Sort	$O(n^2)$	$O(n^2)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	No
Insertion Sort	$O(n^2)$	$O(n^2)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	Yes
Quick Sort	$O(n \log n)$	$O(n^2)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	No
TimSort	$O(n \log n)$	$O(n \log n)$	Yes

Which algorithms are actually used?

- **Java Collections.sort()** and **List.sort()** use **TimSort** for object collections.
- **Python list.sort()** and **sorted()** also use **TimSort**.
- **SQL** does not mandate a specific sorting algorithm; the database engine chooses the most efficient implementation (such as quicksort, merge sort, external sort, or index-based ordering) depending on the execution plan.

5. Algorithm

Conceptually, sorting works as follows.

Step 1

Read all employees.

Step 2

Compare two employees.

Step 3

If order is incorrect

Swap them.

Step 4

Repeat until all employees are ordered.

Step 5

Return sorted list.

6. Bubble Sort Example

Input

70

20

90

40

Pass 1

20

70

40

90

Pass 2

20

40

70

90

Result

20

40

70

90

Bubble Sort complexity:

- Time: **$O(n^2)$**
- Space: **$O(1)$**

Bubble Sort is mainly useful for teaching. In production code, built-in sorting methods are preferred.

7. Time Complexity

Modern built-in sorting implementations:

Average

$O(n \log n)$

Worst case (TimSort):

$O(n \log n)$

8. Space Complexity

Depends on the implementation.

Typical object sorting:

$O(n)$

for TimSort.

9. Java 1.5 Solution

Employee Class

```
public class Employee {  
  
    private int id;  
    private String name;  
    private int age;  
    private String department;  
    private double salary;  
  
    public Employee(int id,  
                    String name,  
                    int age,  
                    String department,  
                    double salary){  
  
        this.id=id;  
        this.name=name;  
        this.age=age;  
        this.department=department;  
        this.salary=salary;  
    }  
  
    public String getName(){  
        return name;  
    }  
  
    public double getSalary(){  
        return salary;  
    }  
  
    public int getAge(){  
        return age;  
    }  
}
```

Sort by Salary

```
import java.util.*;  
  
Collections.sort(  
    employees,
```

```
new Comparator<Employee>(){  
  
    public int compare(  
        Employee e1,  
        Employee e2){  
  
        return Double.compare(  
            e1.getSalary(),  
            e2.getSalary());  
  
    }  
  
});
```

Sort by Salary Descending

```
Collections.sort(  
    employees,  
    new Comparator<Employee>(){  
  
        public int compare(  
            Employee e1,  
            Employee e2){  
  
            return Double.compare(  
                e2.getSalary(),  
                e1.getSalary());  
  
        }  
  
    });
```

Sort by Name

```
Collections.sort(  
    employees,  
    new Comparator<Employee>(){  
  
        public int compare(  
            Employee e1,  
            Employee e2){  
  
            return e1.getName()  
                .compareTo(  
                    e2.getName());  
  
        }  
  
    });
```

```
});
```

Sort by Age

```
Collections.sort(  
    employees,  
    new Comparator<Employee>(){  
  
        public int compare(  
            Employee e1,  
            Employee e2){  
  
            return Integer.compare(  
                e1.getAge(),  
                e2.getAge());  
  
        }  
  
    });
```

10. Comparable vs Comparator

Comparable

Used when there is **one natural ordering**.

Example

```
public class Employee  
implements Comparable<Employee>{  
  
    @Override  
    public int compareTo(Employee other){  
  
        return this.getId()-other.getId();  
  
    }  
  
}
```

Then

```
Collections.sort(employees);
```

Comparator

Used when multiple sorting orders are required.

Example

- Salary
 - Name
 - Age
 - Department
-

11. Java 8 Stream API

Sort by Salary

```
employees.stream()  
  
    .sorted(  
        Comparator.comparing(  
            Employee::getSalary))  
  
    .forEach(System.out::println);
```

Sort by Salary Descending

```
employees.stream()  
  
    .sorted(  
        Comparator.comparing(  
            Employee::getSalary)  
  
    .reversed())  
  
    .forEach(System.out::println);
```

Sort by Name

```
employees.stream()  
  
    .sorted(  
        Comparator.comparing(  
            Employee::getName))  
  
    .forEach(System.out::println);
```

Sort by Age

```
employees.stream()

.sorted(
    Comparator.comparingInt(
        Employee::getAge))

.forEach(System.out::println);
```

Multiple Field Sorting

Sort by Department and then Salary.

```
employees.stream()

.sorted(

Comparator.comparing(
Employee::getDepartment)

.thenComparing(
Employee::getSalary)

)

.forEach(System.out::println);
```

12. Stream Pipeline

Employee List

↓

Stream

↓

sorted()

↓

Comparator

↓

Sorted Employees

↓

Print

13. Internal Working

Conceptually,

`Collections.sort(list, comparator);`

Internally, Java delegates to **TimSort** for object lists.

The comparator repeatedly compares employee objects until the list is ordered.

14. SQL Solution

Employee Table

Name	Age	Salary
John	25	50000
David	35	60000
Alice	32	70000
Robert	45	90000
James	28	65000

Sort by Salary

```
SELECT *  
FROM Employee  
ORDER BY salary ASC;
```

Salary Descending

```
SELECT *  
FROM Employee  
ORDER BY salary DESC;
```

Sort by Name

```
SELECT *  
FROM Employee  
ORDER BY name;
```

Multiple Columns

Department then Salary Descending

```
SELECT *  
FROM Employee  
ORDER BY department,  
        salary DESC;
```

15. Python Solution

Sort by Salary

```
employees = [  
    ("John",50000),  
    ("David",60000),  
    ("Alice",70000),  
    ("Robert",90000),  
    ("James",65000)  
]  
  
employees.sort(  
    key=lambda employee: employee[1]  
)  
  
print(employees)
```

Descending

```
employees.sort(  
    key=lambda employee: employee[1],  
    reverse=True  
)
```

Using sorted()

```
sorted_employees = sorted(  
    employees,  
    key=lambda employee: employee[1]  
)
```

sort() vs sorted()

Feature	list.sort()	sorted()
Modifies Original List	Yes	No
Returns	None	New sorted list
Memory Usage	Lower	Higher

Sort by Name

```
employees.sort(  
    key=lambda employee: employee[0]  
)
```

Multiple Fields

```
employees.sort(  
    key=lambda employee:  
        (employee[2], employee[1])  
)
```

Example assumes tuples of the form:

(name, salary, department)

16. Algorithm Comparison

Method	Average Time	Extra Space
Java Collections.sort()	$O(n \log n)$	$O(n)$
Java Stream sorted()	$O(n \log n)$	$O(n)$
SQL ORDER BY	Depends on execution plan and indexing	Database-managed
Python sort()	$O(n \log n)$	$O(n)$
Python sorted()	$O(n \log n)$	$O(n)$

17. Stable vs Unstable Sorting

A **stable sort** preserves the relative order of elements that compare equal.

Example

Name	Salary
John	50000

David	60000
Alice	60000

After stable sorting by salary:

Name	Salary
John	50000
David	60000
Alice	60000

David remains before Alice because they had the same salary originally.

TimSort (used by Java and Python for object sorting) is **stable**.

18. Edge Cases

Empty List

Result

[]

Single Employee

Input

John

Output

John

Duplicate Salaries

Name	Salary
John	60000
David	60000
Alice	70000

Sorting by salary keeps both employees together.

Null Values

Java example

```
Comparator<Employee> comparator =  
    Comparator.comparing(  
        Employee::getName,  
        Comparator.nullsLast(  
            String::compareTo  
        )  
    );
```

This places employees with null names at the end.

19. Common Interview Questions

1. What is sorting?
 2. What is the difference between ascending and descending sorting?
 3. What is the difference between Comparable and Comparator?
 4. Why is TimSort used in Java and Python?
 5. What is a stable sorting algorithm?
 6. What is the difference between Collections.sort() and Stream.sorted()?
 7. What is the difference between Python sort() and sorted()?
 8. How does SQL ORDER BY work?
 9. How do you sort by multiple fields?
 10. How would you sort employees by salary descending and then by name ascending?
-

20. Summary

- **Sorting** arranges employee records according to one or more fields such as salary, age, name, or department.
- The core idea is to repeatedly compare employee objects using a comparison rule until the entire collection is ordered.
- Java 1.5 uses Collections.sort() together with Comparable or Comparator.
- Java 8 Streams provide the sorted() operation with Comparator.comparing() and methods such as reversed() and thenComparing() for expressive multi-level sorting.
- SQL uses the ORDER BY clause to sort query results and supports sorting by multiple columns with independent ascending or descending directions.
- Python provides list.sort() for in-place sorting and sorted() for creating a new sorted sequence, both using the stable **TimSort** algorithm.
- Understanding sorting is fundamental because it underpins many interview questions, including **top-N employees**, **highest salary**, **second-highest salary**, **ranking**, **binary search**, and **grouped reporting**.

Finding the First Employee Using Java 1.5, Java 8 Stream API, SQL, and Python

Finding the **first employee** is one of the most common interview questions involving collections, streams, SQL queries, and Python. Although it looks simple, interviewers often ask follow-up questions about ordering, edge cases, and internal working.

This problem helps evaluate your understanding of:

- Collections
 - Lists
 - Streams
 - SQL ORDER BY
 - SQL LIMIT
 - Python Lists
 - Time Complexity
 - Space Complexity
-

1. Problem Definition

Suppose we have the following employee list.

ID	Name	Age	Department	Salary
101	John	25	IT	50000
102	David	35	HR	60000
103	Alice	32	IT	70000
104	Robert	45	Finance	90000
105	James	28	HR	65000

Find the **first employee**.

Expected Output

ID	Name	Age	Department	Salary
101	John	25	IT	50000

2. What is the First Employee?

Definition

The **first employee** is the employee located at the **first position** in an ordered collection.

For a Java List, Python list, or a SQL query with an explicit ordering, the first employee is the element that appears first according to that order.

Important: A "first employee" only has meaning if the data has a defined order.

- Lists preserve order.
 - HashSet and HashMap do **not** guarantee iteration order.
 - SQL tables are **unordered** by definition unless an ORDER BY clause is used.
-

3. Real-World Examples

Finding the first employee is useful for:

- First employee in a department
 - First employee by joining date
 - First employee alphabetically
 - First employee with the highest salary after sorting
 - First employee in a report
-

4. Mathematical Definition

Let

Employees = $[e_1, e_2, e_3, \dots, e_n]$

The first employee is

e_1

Example

John
David
Alice
Robert
James

First Employee

John

5. Algorithm

The algorithm depends on whether the collection is already ordered.

Algorithm

Step 1

Check if the collection is empty.

Step 2

If empty

Return "No Employee Found".

Step 3

Otherwise

Return the employee at index 0.

Step 4

End.

6. Flow Diagram

Start

↓

Employee List Empty?

↓

Yes

↓

Return No Employee

↓

No

↓

Return Employee[0]

↓

End

7. Dry Run

Input

John

David

Alice

Robert

James

First Position

Index

0

Output

John

8. Time Complexity

Accessing the first element of an ArrayList or Python list:

$O(1)$

For a LinkedList, `getFirst()` is also **$O(1)$** .

9. Space Complexity

$O(1)$

No additional storage is required.

10. Java 1.5 Solution

Employee Class

```
public class Employee {

    private int id;
    private String name;
    private int age;
    private String department;
    private double salary;

    public Employee(int id,
                    String name,
                    int age,
                    String department,
                    double salary){

        this.id=id;
        this.name=name;
        this.age=age;
        this.department=department;
        this.salary=salary;
    }

    public String getName(){
        return name;
    }

    @Override
    public String toString(){

        return id + " " +
            name + " " +
            age + " " +
            department + " " +
            salary;
    }
}
```

Method 1 (Recommended)

```
import java.util.*;

public class FirstEmployee {

    public static void main(String[] args){

        List<Employee> employees =
            new ArrayList<Employee>();
    }
}
```

```
employees.add(new Employee(101,"John",25,"IT",50000));
employees.add(new Employee(102,"David",35,"HR",60000));
employees.add(new Employee(103,"Alice",32,"IT",70000));

if(!employees.isEmpty()){

    Employee first = employees.get(0);

    System.out.println(first);

}

}
```

Output

101 John 25 IT 50000

Method 2 (LinkedList)

```
LinkedList<Employee> employees =
    new LinkedList<Employee>();
```

```
Employee first = employees.getFirst();
```

getFirst() throws NoSuchElementException if the list is empty. Use isEmpty() first or use peekFirst() if appropriate.

11. Internal Working

ArrayList

↓

Index

0

↓

John

ArrayList stores elements in an indexed array.

12. Java 8 Stream API

Java 8 introduced the `findFirst()` terminal operation.

Method 1 (Recommended)

```
Optional<Employee> employee =  
employees.stream()  
.findFirst();  
employee.ifPresent(System.out::println);
```

Output

```
101 John 25 IT 50000
```

Method 2

Get the employee directly.

```
Employee employee =  
employees.stream()  
.findFirst()  
.orElse(null);
```

Method 3

Throw an exception if no employee exists.

```
Employee employee =  
employees.stream()  
.findFirst()  
.orElseThrow(  
    () -> new RuntimeException(  
        "Employee Not Found"));
```

13. Stream Pipeline

Employee List

↓

Stream

↓

findFirst()

↓

Optional<Employee>

↓

Employee

14. Internal Working

Conceptually,

Employee first = null;

```
for(Employee employee : employees){
```

```
    first = employee;
```

```
    break;
```

```
}
```

findFirst() stops processing after the first element.

15. SQL Solution

Method 1 (Recommended)

Assume employees should be ordered by employee ID.

```
SELECT *  
FROM Employee  
ORDER BY id  
LIMIT 1;
```

This works in MySQL, PostgreSQL, and SQLite.

SQL Server

```
SELECT TOP 1 *  
FROM Employee  
ORDER BY id;
```

Oracle Database (12c and later)

```
SELECT *  
FROM Employee  
ORDER BY id  
FETCH FIRST 1 ROW ONLY;
```

Why ORDER BY Matters

Incorrect

```
SELECT *  
FROM Employee  
LIMIT 1;
```

Without ORDER BY, SQL does **not** guarantee which row will be returned because relational tables are unordered.

16. Python Solution

Method 1 (Recommended)

```
employees = [  
    ("John",25),  
    ("David",35),  
    ("Alice",32)  
]
```

if employees:

```
    print(employees[0])
```

Output

```
('John',25)
```

Method 2

```
first = employees[0] if employees else None
```

```
print(first)
```

Method 3

Using next()

```
employees = [  
    ("John",25),  
    ("David",35),  
    ("Alice",32)  
]
```

```
first = next(iter(employees), None)
```

```
print(first)
```

next(iter(...), None) safely returns the first element or None if the iterable is empty.

17. Algorithm Comparison

Method	Time Complexity	Space Complexity
Java ArrayList.get(0)	O(1)	O(1)
Java Stream findFirst()	O(1)*	O(1)
SQL ORDER BY ... LIMIT 1	Depends on indexing and execution plan	Database-managed
Python List Index	O(1)	O(1)
Python next(iter())	O(1)	O(1)

*For ordered sequential streams, findFirst() returns after the first element. If earlier stream operations (such as filter()) are present, it processes elements until the first matching element is found.

18. findFirst() vs findAny()

Feature	findFirst()	findAny()
Ordered Streams	Returns the first element	May return any element

Parallel Streams	Preserves encounter order	May return any matching element for better performance
Deterministic	Yes (for ordered streams)	Not necessarily

Example

```
employees.stream()
    .findFirst();
```

Always returns the first employee in the stream's encounter order.

19. Edge Cases

Empty Employee List

Java

```
employees.stream()
    .findFirst()
    .orElse(null);
```

Output

null

Python

```
employees = []

first = employees[0] if employees else None

print(first)
```

Output

None

SQL

If the query returns no rows:

Empty Result Set

Single Employee

Input

Name

John

Output

John

Unordered Collections

Example

```
HashSet<Employee> employees =  
    new HashSet<Employee>();
```

Calling

```
employees.stream().findFirst();
```

returns the **first element in the set's iteration order**, which is **not guaranteed** to be the same across runs or JVM implementations.

If a specific employee is required, convert to a list or sort the stream first.

20. Common Interview Questions

1. What is the difference between `get(0)` and `findFirst()`?
 2. Why does `findFirst()` return an `Optional`?
 3. What is the difference between `findFirst()` and `findAny()`?
 4. Why is `ORDER BY` required in SQL before `LIMIT 1`?
 5. What happens if the employee list is empty?
 6. Can `findFirst()` short-circuit?
 7. How would you find the first employee older than 30?
 8. How would you find the first employee after sorting by salary?
 9. How would you find the first employee in each department?
 10. Why should `HashSet` not be used when a predictable first element is required?
-

21. Summary

- **Finding the first employee** means retrieving the employee at the beginning of an ordered collection or ordered query result.
- The operation is typically **O(1)** for indexed collections such as ArrayList and Python lists.
- Java 1.5 retrieves the first employee using `get(0)` after checking that the list is not empty.
- Java 8 provides `findFirst()`, which returns an `Optional<Employee>` and safely handles empty streams.
- SQL requires an explicit `ORDER BY` before `LIMIT 1` (or `TOP 1 / FETCH FIRST 1 ROW ONLY`) to ensure deterministic results.
- Python retrieves the first element using index 0 or `next(iter(...), None)` for a safe, iterator-based approach.
- Understanding this operation provides the foundation for related interview questions such as **first employee older than 30**, **first employee after sorting**, **first employee in each department**, and **top-ranked employee queries**.

Finding the Last Employee Using Java 1.5, Java 8 Stream API, SQL, and Python

Finding the **last employee** is a common interview question that tests your understanding of collections, iteration, streams, SQL ordering, and list operations. Unlike finding the first element, finding the last element may require traversing the collection, depending on the data structure.

This problem evaluates your understanding of:

- Collections
- Lists
- Streams
- SQL `ORDER BY`
- Python Lists
- Time Complexity
- Space Complexity

1. Problem Definition

Suppose we have the following employee list.

ID	Name	Age	Department	Salary
101	John	25	IT	50000
102	David	35	HR	60000
103	Alice	32	IT	70000
104	Robert	45	Finance	90000
105	James	28	HR	65000

Find the **last employee**.

Expected Output

ID	Name	Age	Department	Salary
105	James	28	HR	65000

2. What is the Last Employee?

Definition

The **last employee** is the employee located at the **last position** in an ordered collection.

For example:

Employees

Index

0 John
1 David
2 Alice
3 Robert
4 James

The last employee is:

James

Important: Just like the "first employee," the "last employee" is meaningful only when the data has a defined order.

- List preserves insertion order.
 - LinkedHashSet preserves insertion order.
 - HashSet and HashMap do **not** guarantee iteration order.
 - SQL tables are unordered unless ORDER BY is specified.
-

3. Real-World Applications

Finding the last employee is useful for:

- Last employee who joined (ordered by joining date)
- Last employee alphabetically
- Employee with the highest ID
- Last record in a report
- Last transaction processed

4. Mathematical Definition

Let

Employees = $[e_1, e_2, e_3, \dots, e_n]$

The last employee is

e_n

5. Algorithm

The algorithm depends on the data structure.

Algorithm

Step 1

Check if the collection is empty.

Step 2

If empty

Return "No Employee Found".

Step 3

Otherwise

Return the employee at index (size - 1).

Step 4

End.

6. Flow Diagram

Start

↓

Employee List Empty?

↓

Yes

↓

Return No Employee

↓

No

↓

Return Employee[size-1]

↓

End

7. Dry Run

Input

John

David

Alice

Robert

James

Size

5

Last Index

5 - 1

=

4

Employee at index 4

James

8. Time Complexity

ArrayList

`get(size-1)`

$O(1)$

LinkedList

`getLast()`

$O(1)$

`LinkedList.get(size-1)` also runs in **$O(1)$** because the implementation traverses from the tail node when accessing the last element.

General Iteration

$O(n)$

when you scan through all employees to keep the last one.

9. Space Complexity

$O(1)$

10. Java 1.5 Solution

Employee Class

```
public class Employee {  
    private int id;  
    private String name;  
    private int age;  
    private String department;  
    private double salary;  
  
    public Employee(int id,  
                    String name,  
                    int age,
```

```

        String department,
        double salary){

    this.id=id;
    this.name=name;
    this.age=age;
    this.department=department;
    this.salary=salary;
}

@Override
public String toString(){

    return id+" "+name+" "+age+" "+department+" "+salary;

}

}

```

Method 1 (ArrayList)

```

import java.util.*;

public class LastEmployee {

    public static void main(String[] args){

        List<Employee> employees =
            new ArrayList<Employee>();

        employees.add(new Employee(101,"John",25,"IT",50000));
        employees.add(new Employee(102,"David",35,"HR",60000));
        employees.add(new Employee(103,"Alice",32,"IT",70000));
        employees.add(new Employee(104,"Robert",45,"Finance",90000));
        employees.add(new Employee(105,"James",28,"HR",65000));

        if(!employees.isEmpty()){

            Employee last =
                employees.get(
                    employees.size()-1);

            System.out.println(last);

        }

    }
}

```



```
}
```

Output

```
105 James 28 HR 65000
```

Method 2 (LinkedList)

```
LinkedList<Employee> employees =  
    new LinkedList<Employee>();
```

```
Employee last = employees.getLast();
```

getLast() throws NoSuchElementException if the list is empty. Use isEmpty() first or use peekLast() if appropriate.

Method 3 (Manual Iteration)

```
Employee last = null;
```

```
for(Employee employee : employees){  
    last = employee;  
}
```

```
System.out.println(last);
```

11. Internal Working

ArrayList

↓

size = 5

↓

size-1 = 4

↓

Employee[4]

↓

James

12. Java 8 Stream API

Unlike `findFirst()`, Java 8 **does not provide** a built-in `findLast()` method.

Method 1 (Using `reduce()`)

```
Employee last =  
employees.stream()  
.reduce((first, second) -> second)  
.orElse(null);  
  
System.out.println(last);
```

Output

105 James 28 HR 65000

How `reduce()` Works

Conceptually:

```
Employee result = null;  
for(Employee employee : employees){  
    result = employee;  
}
```

The accumulator always keeps the current employee, so after the stream finishes, it contains the last one.

Method 2 (Using List Index)

If you already have a List, simply use:

```
Employee last =  
    employees.get(
```

```
employees.size()-1);
```

This is more efficient than streaming because it is **O(1)** for an ArrayList.

13. Stream Pipeline

Employee List

↓

Stream

↓

reduce()

↓

Last Employee

14. SQL Solution

Method 1 (Recommended)

If the employee with the largest ID is considered the last employee:

```
SELECT *  
FROM Employee  
ORDER BY id DESC  
LIMIT 1;
```

Output

ID	Name	Age
105	James	28

SQL Server

```
SELECT TOP 1 *  
FROM Employee  
ORDER BY id DESC;
```

Oracle (12c+)

```
SELECT *  
FROM Employee  
ORDER BY id DESC  
FETCH FIRST 1 ROW ONLY;
```

Why ORDER BY Is Required

Incorrect:

```
SELECT *  
FROM Employee  
LIMIT 1;
```

This does **not** guarantee that the returned row is the "last" employee because SQL tables have no inherent row order.

15. Python Solution

Method 1 (Recommended)

```
employees = [  
    ("John",25),  
    ("David",35),  
    ("Alice",32),  
    ("Robert",45),  
    ("James",28)  
]
```

```
if employees:  
    print(employees[-1])
```

Output

```
('James', 28)
```

Python uses **negative indexing**:

- -1 → last element
 - -2 → second last
 - -3 → third last
-

Method 2

```
last = employees[-1] if employees else None
```

```
print(last)
```

Method 3 (Manual Iteration)

```
last = None
```

```
for employee in employees:  
    last = employee
```

```
print(last)
```

16. Algorithm Comparison

Method	Time Complexity	Space Complexity
Java ArrayList.get(size-1)	O(1)	O(1)
Java LinkedList.getLast()	O(1)	O(1)
Java Stream reduce()	O(n)	O(1)
SQL ORDER BY ... DESC LIMIT 1	Depends on indexing and execution plan	Database-managed
Python employees[-1]	O(1)	O(1)
Manual Iteration	O(n)	O(1)

17. findFirst() vs findLast()

Feature	findFirst()	Last Employee
Java Stream Support	Built-in (findFirst())	No built-in method
Stops Early	Yes	No (reduce() must consume the stream)
Time Complexity	O(1) for a simple ordered stream	O(n) with reduce()
Best for List	get(0)	get(size-1)

18. Edge Cases

Empty List

Java

```
Employee last =  
    employees.stream()  
        .reduce((a, b) -> b)
```

```
.orElse(null);
```

Returns:

null

Python

```
employees = []
```

```
last = employees[-1] if employees else None
```

Returns:

None

SQL

If no rows match:

Empty Result Set

One Employee

Input

Name

John

Output

John

Unordered Collections

Example

```
HashSet<Employee> employees =  
    new HashSet<Employee>();
```

A stream over a HashSet has **no guaranteed encounter order**, so the "last" element is not deterministic.

If a predictable last employee is required, use:

- List
 - LinkedHashSet
 - A sorted stream
 - A SQL query with ORDER BY
-

19. Common Interview Questions

1. Why doesn't Java Stream provide `findLast()`?
 2. Why does `reduce()` work for finding the last element?
 3. What is the difference between `ArrayList.get(size-1)` and `reduce()`?
 4. Why is ORDER BY required in SQL?
 5. Why is Python negative indexing useful?
 6. How would you find the last employee older than 30?
 7. How would you find the last employee after sorting by salary?
 8. How would you find the last employee in each department?
 9. Why shouldn't HashSet be used when the last element matters?
 10. Which solution is the most efficient for a List?
-

20. Summary

- **Finding the last employee** means retrieving the employee at the end of an ordered collection or ordered query result.
- For Java `ArrayList`, `get(size() - 1)` is the most efficient solution, running in **$O(1)$** time.
- For Java `LinkedList`, `getLast()` provides direct access to the tail node in **$O(1)$** time.
- Java 8 Streams do not include a `findLast()` method, so `reduce((first, second) -> second)` is commonly used, although it requires traversing the entire stream and runs in **$O(n)$** time.
- SQL requires an explicit ORDER BY (typically descending by a key such as `id` or `joining_date`) before limiting the result to one row to ensure deterministic behavior.
- Python provides a concise and efficient solution using negative indexing (`employees[-1]`), which runs in **$O(1)$** time for lists.
- This concept is closely related to other interview questions such as **last employee older than 30**, **last employee by salary**, **last employee in each department**, and **finding the last matching element after filtering or sorting**.

Checking Whether Any Employee Exists Using Java 1.5, Java 8 Stream API, SQL, and Python

Checking whether **any employee exists** is one of the most common interview questions. It appears simple, but interviewers often use it to assess your understanding of:

- Searching
 - Existence Checking
 - Boolean Logic
 - Short-Circuit Evaluation
 - Java Collections
 - Java 8 Stream API
 - SQL EXISTS
 - Python any()
 - Time Complexity
-

1. Problem Definition

Suppose we have the following employee list.

ID	Name	Age	Department	Salary
101	John	25	IT	50000
102	David	35	HR	60000
103	Alice	32	IT	70000
104	Robert	45	Finance	90000
105	James	28	HR	65000

Question

Does any employee exist whose age is greater than 30?

Expected Output

true

because David, Alice, and Robert satisfy the condition.

2. What Does "Any Employee Exists" Mean?

Definition

An **existence check** determines whether **at least one** employee satisfies a given condition.

Unlike counting or filtering, we are **not interested in all matching employees**.

We only need to know:

- Yes (true)
 - No (false)
-

3. Real-World Examples

Existence checks are common in:

- Login systems
- Employee verification
- Payroll validation
- Banking
- Hospital systems
- Inventory management

Examples

- Is there any employee in the HR department?
 - Is there any employee earning more than ₹1,00,000?
 - Is there any employee older than 60?
 - Is there any inactive employee?
-

4. Mathematical Definition

Let

Employees = E

Predicate

P(Employee)

We want to determine

$\exists$ employee $\in$ E

such that

P(employee) = true

The symbol

$\exists$

means

"There exists at least one."

5. Algorithm

The algorithm is based on **short-circuit searching**.

As soon as one matching employee is found, the search stops.

Algorithm

Step 1

Read employee.

Step 2

Check condition.

Step 3

Condition true?

Yes

Return true.

No

Continue.

Step 4

Repeat.

Step 5

No match?

Return false.

6. Flow Diagram

Start

↓

Read Employee

↓

Condition True?

↓

Yes

↓

Return TRUE

↓

No

↓

Next Employee

↓

End of List?

↓

Yes

↓

Return FALSE

7. Dry Run

Input

John

Age = 25

$25 > 30$

No

Next

David

Age = 35

35 > 30

Yes

Immediately

Return TRUE

The algorithm stops.

It never checks

Alice

Robert

James

This is called **short-circuit evaluation**.

8. Time Complexity

Best Case

First employee matches.

$O(1)$

Worst Case

No employee matches.

$O(n)$

9. Space Complexity

$O(1)$

10. Java 1.5 Solution

Employee Class

```
public class Employee {  
  
    private int id;  
    private String name;  
    private int age;  
    private String department;  
    private double salary;  
  
    public Employee(int id,  
                    String name,  
                    int age,  
                    String department,  
                    double salary){  
  
        this.id=id;  
        this.name=name;  
        this.age=age;  
        this.department=department;  
        this.salary=salary;  
    }  
  
    public int getAge(){  
  
        return age;  
    }  
}
```

Java 1.5 Code

```
boolean exists = false;  
  
for(Employee emp : employees){  
  
    if(emp.getAge() > 30){  
  
        exists = true;  
  
        break;  
    }  
}  
  
System.out.println(exists);
```

Output

true

11. Internal Working

John

↓

25 > 30

No

↓

David

↓

35 > 30

Yes

↓

TRUE

↓

STOP

The break statement prevents unnecessary comparisons.

12. Java 8 Stream API

Java 8 introduced the **anyMatch()** terminal operation.

Method 1 (Recommended)

boolean exists =

employees.stream()

.anyMatch(emp ->
 emp.getAge() > 30);

```
System.out.println(exists);
```

Output

```
true
```

Method 2

Check for IT employees.

```
boolean exists =
```

```
employees.stream()
```

```
.anyMatch(emp ->
```

```
"IT".equals(  
emp.getDepartment()));
```

Method 3

Check salary greater than ₹80,000.

```
boolean exists =
```

```
employees.stream()
```

```
.anyMatch(emp ->
```

```
emp.getSalary() > 80000);
```

13. Stream Pipeline

Employee List

↓

Stream

↓

anyMatch()

↓

Predicate

↓

TRUE/FALSE

14. Internal Working

Conceptually,

```
boolean result = false;
```

```
for(Employee emp : employees){
```

```
    if(emp.getAge() > 30){
```

```
        result = true;
```

```
        break;
```

```
    }
```

```
}
```

anyMatch() performs the same logic internally and **short-circuits** as soon as a match is found.

15. SQL Solution

Method 1 (Using EXISTS) — Recommended

```
SELECT EXISTS (  
    SELECT 1  
    FROM Employee  
    WHERE age > 30  
) AS employee_exists;
```

Typical Output

employee_exists
1

Many databases represent TRUE as 1 and FALSE as 0.

Method 2 (Using COUNT)

```
SELECT COUNT(*)  
FROM Employee  
WHERE age > 30;
```

If the count is greater than zero:

Employee Exists

Otherwise:

No Employee

EXISTS is generally preferred when you only need to know whether at least one row matches, because the database can stop searching as soon as it finds the first qualifying row.

16. Python Solution

Method 1 (Recommended)

Using any()

```
employees = [  
    ("John",25),  
    ("David",35),  
    ("Alice",32),  
    ("Robert",45),  
    ("James",28)  
]  
  
exists = any(  
    age > 30  
    for _, age in employees  
)  
  
print(exists)
```

Output

True

Method 2

Using a loop

```

exists = False

for _, age in employees:

    if age > 30:

        exists = True

        break

print(exists)

```

17. Algorithm Comparison

Method	Best Case	Worst Case	Space
Java Loop	O(1)	O(n)	O(1)
Java anyMatch()	O(1)	O(n)	O(1)
SQL EXISTS	Depends on indexes and execution plan	Database-managed	
Python any()	O(1)	O(n)	O(1)

18. anyMatch() vs filter()

Feature	anyMatch()	filter()
Returns	boolean	Stream
Stops Early	Yes	No (unless followed by a short-circuiting terminal operation such as findFirst())
Purpose	Existence checking	Select matching elements

Example

```

employees.stream()

.anyMatch(emp ->
    emp.getAge() > 30);

```

returns

true

Whereas

```

employees.stream()

```

```

.filter(emp ->

```

```
emp.getAge() > 30)

.collect(Collectors.toList());

returns

All matching employees
```

19. anyMatch() vs findFirst()

Feature	anyMatch()	findFirst()
Return Type	boolean	Optional<Employee>
Returns Employee	No	Yes
Returns Existence	Yes	Indirectly (check isPresent())
Stops Early	Yes	Yes

20. Edge Cases

Empty Employee List

Java

```
employees.stream()
    .anyMatch(emp -> emp.getAge() > 30);
```

Output

false

Python

```
any([])
```

Output

False

SQL

```
SELECT EXISTS (
    SELECT 1
    FROM Employee
```

```
WHERE age > 30  
);
```

Output

0

No Matching Employee

Employees

Name Age

John 25

James 28

Output

false

First Employee Matches

Name Age

David 35

John 25

The algorithm immediately returns:

true

without checking the remaining employees.

21. Common Interview Questions

1. What is short-circuit evaluation?
2. Why is `anyMatch()` more efficient than filtering all employees?
3. What is the difference between `anyMatch()` and `allMatch()`?
4. What is the difference between `anyMatch()` and `findFirst()`?

5. Why is SQL EXISTS usually preferred over COUNT(*) for existence checks?
6. What happens if the employee list is empty?
7. How would you check if any employee belongs to the IT department?
8. How would you check if any employee has a salary greater than ₹1,00,000?
9. How would you check if any employee has a null department?
10. How would you check if any employee satisfies multiple conditions?

22. Related Stream Operations

Operation	Purpose	Return Type
anyMatch()	At least one element matches	boolean
allMatch()	Every element matches	boolean
noneMatch()	No element matches	boolean
findFirst()	Returns the first matching element (after any preceding filters)	Optional<T>
findAny()	Returns any matching element	Optional<T>

23. Summary

- **Checking whether any employee exists** is an **existence-checking** problem rather than a filtering or counting problem.
- The algorithm examines employees one by one and **stops immediately** when it finds the first employee satisfying the condition. This short-circuit behavior gives a **best-case time complexity of O(1)** and a **worst-case time complexity of O(n)**.
- Java 1.5 implements this using a loop, a boolean flag, and a break statement.
- Java 8 Stream API provides the concise anyMatch() terminal operation, which performs the same short-circuit evaluation internally.
- SQL uses the EXISTS operator for efficient existence checks, while COUNT(*) can also be used when the actual number of matching rows is required.
- Python provides the built-in any() function, which evaluates lazily and also stops as soon as a matching element is found.
- This pattern forms the basis for many interview questions, such as **checking if any employee belongs to a department, checking if any salary exceeds a threshold, validating business rules, and detecting the existence of matching records.**

Checking Whether All Employees Match a Condition Using Java 1.5, Java 8 Stream API, SQL, and Python

Checking whether **all employees satisfy a given condition** is one of the most frequently asked Java 8 Stream API interview questions. It demonstrates your understanding of:

- Universal Quantification
 - Predicate Evaluation
 - Short-Circuit Evaluation
 - Java Collections
 - Java 8 Stream API
 - SQL Logic
 - Python all()
 - Time Complexity
 - Space Complexity
-

1. Problem Definition

Suppose we have the following employee data.

ID	Name	Age	Department	Salary
101	John	25	IT	50000
102	David	35	HR	60000
103	Alice	32	IT	70000
104	Robert	45	Finance	90000
105	James	28	HR	65000

Question

Do all employees have a salary greater than ₹40,000?

Expected Output

true

because every employee's salary is greater than ₹40,000.

2. What Does "All Employees Match" Mean?

Definition

An **all-match** operation checks whether **every employee** satisfies a specified condition.

Unlike `anyMatch()`, where only one matching employee is sufficient, `allMatch()` requires **every employee** to satisfy the predicate.

Example

Employees' Ages

25
35
32
45
28

Condition

Age > 20

Result

true

Because every employee is older than 20.

3. Real-World Applications

Checking whether all employees satisfy a condition is common in:

- HR Systems
- Payroll Validation
- Employee Eligibility
- Data Validation
- Compliance Audits
- Banking
- Insurance

Examples

- Are all employees active?
 - Are all salaries positive?
 - Have all employees completed training?
 - Are all employees above the legal working age?
 - Have all employees submitted documents?
-

4. Mathematical Definition

Let

Employees = E

Predicate

$P(\text{Employee})$

We want to determine

$\forall \text{ employee} \in E$

$P(\text{employee}) = \text{true}$

The symbol

$\forall$

means

"For every employee."

5. Algorithm

The algorithm is also based on **short-circuit evaluation**.

The search stops immediately when one employee fails the condition.

Algorithm

Step 1

Read employee.

Step 2

Check condition.

Step 3

Condition false?

Yes

Return FALSE.

No

Continue.

Step 4

Repeat.

Step 5

All employees checked?

Return TRUE.

6. Flow Diagram

Start



Read Employee



Condition True?



No



Return FALSE



Yes



Next Employee



More Employees?



Yes

Repeat

↓

No

↓

Return TRUE

7. Dry Run

Input

John

Salary = 50000
 $50000 > 40000$

Yes

Next

David

Salary = 60000
 $60000 > 40000$

Yes

Next

Alice

Salary = 70000

Yes

All employees satisfy the condition.

Output

true

Another Example

Condition

Age > 30

First employee

John

Age = 25

25 > 30

False

Immediately

Return FALSE

The algorithm stops without checking the remaining employees.

8. Time Complexity

Best Case

First employee fails.

$O(1)$

Worst Case

All employees satisfy the condition.

$O(n)$

9. Space Complexity

$O(1)$

10. Java 1.5 Solution

Employee Class

```
public class Employee {  
    private int id;  
    private String name;  
    private int age;
```

```
private String department;
private double salary;

public Employee(int id,
                String name,
                int age,
                String department,
                double salary){

    this.id = id;
    this.name = name;
    this.age = age;
    this.department = department;
    this.salary = salary;
}

public double getSalary(){
    return salary;
}

public int getAge(){
    return age;
}
}
```

Example 1

Check whether all employees earn more than ₹40,000.

```
boolean allMatch = true;

for(Employee emp : employees){

    if(emp.getSalary() <= 40000){

        allMatch = false;

        break;

    }

}

System.out.println(allMatch);
```

Output

true

Example 2

Check whether all employees are older than 30.

```
boolean allOlderThan30 = true;
for(Employee emp : employees){
    if(emp.getAge() <= 30){
        allOlderThan30 = false;
        break;
    }
}
System.out.println(allOlderThan30);
```

Output

false

11. Internal Working

Example

John

50000

↓

Condition

True

↓

David

60000

↓

True

↓

Alice

70000

↓

True

↓

Robert

90000

↓

True

↓

James

65000

↓

True

↓

TRUE

12. Java 8 Stream API

Java 8 introduced the **allMatch()** terminal operation.

Example 1

boolean result =

employees.stream()

```
.allMatch(emp ->  
    emp.getSalary() > 40000);
```

```
System.out.println(result);
```

Output

true

Example 2

boolean result =

```
employees.stream()
```

```
.allMatch(emp ->  
    emp.getAge() > 30);
```

Output

false

Example 3

Check whether all employees belong to IT.

boolean result =

```
employees.stream()
```

```
.allMatch(emp ->  
    "IT".equals(emp.getDepartment()));
```

Output

false

13. Stream Pipeline

Employee List

↓

Stream

↓

allMatch()

↓

Predicate

↓

TRUE/FALSE

14. Internal Working

Conceptually,

```
boolean result = true;
```

```
for(Employee emp : employees){
```

```
    if(!(emp.getSalary() > 40000)){
```

```
        result = false;
```

```
        break;
```

```
    }
```

```
}
```

The Stream API performs exactly the same logic internally.

15. SQL Solution

SQL does not provide an ALLMATCH() function, but the same logic can be expressed using NOT EXISTS.

Method 1 (Recommended)

Check whether all employees earn more than ₹40,000.

```
SELECT NOT EXISTS (  
    SELECT 1
```



```
FROM Employee
WHERE salary <= 40000
) AS all_match;
```

Output

all_match

1

The logic is:

- If **no employee violates** the condition, then **all employees satisfy** it.
-

Method 2

Using COUNT.

```
SELECT
CASE
WHEN COUNT(*) =
(
    SELECT COUNT(*)
    FROM Employee
)
THEN 'TRUE'
ELSE 'FALSE'
END
FROM Employee
WHERE salary > 40000;
```

This compares:

- Total employees
- Employees satisfying the condition

If both counts are equal, then all employees satisfy the condition.

16. Python Solution

Method 1 (Recommended)

Using all().

```

employees = [
    ("John",50000),
    ("David",60000),
    ("Alice",70000),
    ("Robert",90000),
    ("James",65000)
]

result = all(
    salary > 40000
    for _, salary in employees
)

print(result)

```

Output

True

Method 2

Using a loop.

```

result = True

for _, salary in employees:

    if salary <= 40000:

        result = False

        break

print(result)

```

17. Algorithm Comparison

Method	Best Case	Worst Case	Space
Java Loop	O(1)	O(n)	O(1)
Java allMatch()	O(1)	O(n)	O(1)
SQL NOT EXISTS	Depends on indexes and execution plan	Database-managed	
Python all()	O(1)	O(n)	O(1)

18. allMatch() vs anyMatch() vs noneMatch()

Operation	Meaning	Returns
allMatch()	Every employee satisfies the condition	boolean
anyMatch()	At least one employee satisfies the condition	boolean
noneMatch()	No employee satisfies the condition	boolean

Example

Employees' Ages

25
35
45

Condition

Age > 30

Results

Method	Result	Reason
allMatch()	false	John is 25
anyMatch()	true	David and Robert match
noneMatch()	false	Some employees match

19. Empty Collection Behavior

This is an important interview topic.

Suppose the employee list is empty.

```
List<Employee> employees = new ArrayList<>();
```

Java

```
employees.stream()  
    .allMatch(emp -> emp.getSalary() > 40000);
```

Returns

true

Python

```
all([])
```

Returns

True

Why?

This follows the mathematical concept of **vacuous truth**:

If there are no employees, then there is **no employee that violates the condition**, so the statement "all employees satisfy the condition" is considered true.

SQL

```
SELECT NOT EXISTS (  
    SELECT 1  
    FROM Employee  
    WHERE salary <= 40000  
);
```

Also evaluates to **TRUE** when the table is empty.

20. Common Interview Questions

1. What is `allMatch()`?
2. How does `allMatch()` work internally?
3. Why does `allMatch()` stop early?
4. What is short-circuit evaluation?
5. Why does `allMatch()` return true for an empty stream?
6. What is the difference between `allMatch()` and `filter()`?
7. What is the difference between `allMatch()` and `noneMatch()`?
8. Why is `NOT EXISTS` preferred over counting rows in SQL?
9. How would you check whether all employees belong to the same department?
10. How would you check whether all employees have positive salaries?

21. Summary

- **Checking whether all employees match a condition** is a **universal validation** problem.
- The algorithm examines employees one by one and **stops immediately** when it finds the first employee that **does not satisfy** the condition.
- This short-circuit behavior gives a **best-case time complexity of $O(1)$** and a **worst-case time complexity of $O(n)$** .
- Java 1.5 implements the logic using a loop, a boolean flag, and a break statement.
- Java 8 Stream API provides the `allMatch()` terminal operation, which performs the same logic internally using short-circuit evaluation.

- SQL typically expresses the same logic using NOT EXISTS, which checks that **no rows violate** the required condition.
- Python provides the built-in all() function, which lazily evaluates the iterable and stops at the first failing element.
- This pattern is commonly used for validating business rules such as **all employees are active, all salaries are positive, all employees belong to the same department, or all records satisfy required constraints.**

Finding the Employee with the Highest Salary Using Java 1.5, Java 8 Stream API, SQL, and Python

Finding the **employee with the highest salary** is one of the most frequently asked interview questions for Java, SQL, and Python developers.

Unlike finding only the **highest salary value**, this problem requires returning the **entire employee object (or row)** associated with that highest salary.

This problem evaluates your understanding of:

- Searching Algorithms
- Maximum Value Selection
- Java Collections
- Java 8 Stream API
- SQL ORDER BY, MAX(), Window Functions
- Python
- Time Complexity
- Space Complexity

1. Problem Definition

Suppose we have the following employee data.

ID	Name	Department	Salary
101	John	IT	50000
102	David	HR	60000
103	Alice	IT	70000
104	Robert	Finance	90000
105	James	HR	65000

Find the **employee who has the highest salary**.

Expected Output

ID	Name	Department	Salary
104	Robert	Finance	90000

2. Difference Between Two Similar Questions

Many interview candidates confuse these two questions.

Question 1

Find the highest salary.

Output

90000

Only the salary value is returned.

Question 2

Find the employee having the highest salary.

Output

Name Salary

Robert 90000

The **entire employee record** is returned.

3. Definition

The employee with the highest salary is:

The employee whose salary is greater than or equal to every other employee's salary.

Mathematically,

Given

Employees = E

Find employee

e

such that

Salary(e)

=

MAX(All Salaries)

4. Real-World Applications

This problem is used in:

- Payroll Systems
 - HR Analytics
 - Employee Ranking
 - Executive Reports
 - Compensation Analysis
 - Business Intelligence
-

5. Basic Algorithm

Instead of storing only the highest salary, store the **employee object** representing the current maximum.

Algorithm

Step 1

Read first employee.

Store it as highestEmployee.

Step 2

Read next employee.

Compare salary.

Current Salary >

highestEmployee Salary ?

Yes

Replace highestEmployee.

No

Continue.

Step 3

Repeat.

Step 4

Return highestEmployee.

6. Flow Diagram

Start

↓

Read First Employee

↓

highestEmployee

↓

Read Next Employee

↓

Salary Greater?

↓

Yes

↓

Update highestEmployee

↓

No

↓

Continue

↓

End

↓

Print Employee

7. Dry Run

Input

Name	Salary
John	50000
David	60000
Alice	70000
Robert	90000
James	65000

Initially

highestEmployee

John

Compare

David

$60000 > 50000$

Yes

Update

highestEmployee

David

Compare

Alice

$70000 > 60000$

Yes

Update

highestEmployee

Alice

Compare

Robert

$90000 > 70000$

Yes

Update

highestEmployee

Robert

Compare

James

$65000 > 90000$

No

Final

Robert

8. Time Complexity

$O(n)$

Each employee is visited exactly once.

9. Space Complexity

O(1)

Only one employee reference is maintained.

10. Java 1.5 Solution

Employee Class

```
public class Employee {  
  
    private int id;  
    private String name;  
    private String department;  
    private double salary;  
  
    public Employee(int id,  
                    String name,  
                    String department,  
                    double salary){  
  
        this.id=id;  
        this.name=name;  
        this.department=department;  
        this.salary=salary;  
    }  
  
    public double getSalary(){  
        return salary;  
    }  
  
    @Override  
    public String toString(){  
  
        return id + " " +  
            name + " " +  
            department + " " +  
            salary;  
    }  
}
```

Java 1.5 Code

```
Employee highestEmployee = null;
```

```
for(Employee employee : employees){
```

```
if(highestEmployee == null ||
    employee.getSalary() >
    highestEmployee.getSalary()){
    highestEmployee = employee;
}
}

System.out.println(highestEmployee);
```

Output

104 Robert Finance 90000

11. Internal Working

John

↓

Highest

↓

David

↓

Higher

↓

Alice

↓

Higher

↓

Robert

↓

Higher

↓

James

↓

Lower

↓

Robert

12. Java 8 Stream API

Java 8 introduced `Stream.max()`.

Method 1 (Recommended)

```
Optional<Employee> employee =  
employees.stream()  
    .max(  
        Comparator.comparing(  
            Employee::getSalary  
        )  
    );  
employee.ifPresent(System.out::println);
```

Output

104 Robert Finance 90000

Internal Working

`Stream.max()` compares employee objects using the provided comparator.

Conceptually:

```
Employee highest = null;  
for(Employee employee : employees){  
    if(highest == null ||  
        employee.getSalary() >
```

```
highest.getSalary()){  
    highest = employee;  
}  
}
```

Method 2

Return the employee directly.

Employee highest =

employees.stream()

```
.max(  
    Comparator.comparing(  
        Employee::getSalary  
    )  
)
```

```
.orElse(null);
```

Method 3

Using reduce()

Employee highest =

employees.stream()

```
.reduce((e1, e2) ->
```

```
e1.getSalary() > e2.getSalary()
```

```
? e1
```

```
: e2)
```

```
.orElse(null);
```

13. Stream Pipeline

Employee List

↓

Stream

↓

Comparator

↓

max()

↓

Highest Employee

14. SQL Solution

Method 1 (Recommended)

Sort by salary descending.

```
SELECT *  
FROM Employee  
ORDER BY salary DESC  
LIMIT 1;
```

Output

Name	Salary
------	--------

Robert	90000
--------	-------

SQL Server

```
SELECT TOP 1 *  
FROM Employee  
ORDER BY salary DESC;
```

Oracle (12c+)

```
SELECT *  
FROM Employee  
ORDER BY salary DESC  
FETCH FIRST 1 ROW ONLY;
```

Method 2

Using MAX().

```
SELECT *  
FROM Employee  
WHERE salary =  
(  
SELECT MAX(salary)  
FROM Employee  
);
```

Why This Method Is Often Better

If two employees have the same highest salary,

Robert

90000

Maria

90000

this query returns **both employees**, whereas ORDER BY ... LIMIT 1 returns only one row.

Method 3

Using DENSE_RANK()

```
SELECT id,  
       name,  
       department,  
       salary  
FROM (  
  SELECT id,  
         name,  
         department,  
         salary,  
         DENSE_RANK() OVER (  
           ORDER BY salary DESC  
         ) AS salary_rank  
  FROM Employee  
) e  
WHERE salary_rank = 1;
```


This also returns **all employees** tied for the highest salary.

15. Python Solution

Method 1 (Recommended)

Using max()

```
employees = [  
    ("John","IT",50000),  
    ("David","HR",60000),  
    ("Alice","IT",70000),  
    ("Robert","Finance",90000),  
    ("James","HR",65000)  
]  
  
highest = max(  
    employees,  
    key=lambda employee: employee[2]  
)  
  
print(highest)
```

Output

```
('Robert', 'Finance', 90000)
```

Method 2

Using a loop

```
highest = None  
  
for employee in employees:  
    if (highest is None or  
        employee[2] > highest[2]):  
        highest = employee  
  
print(highest)
```

16. Algorithm Comparison

Method	Time Complexity	Space Complexity
Java Loop	$O(n)$	$O(1)$
Java Stream max()	$O(n)$	$O(1)$
Java Stream reduce()	$O(n)$	$O(1)$
SQL ORDER BY ... LIMIT 1	Depends on indexing and execution plan	Database-managed
SQL MAX() + Subquery	Depends on indexing and execution plan	Database-managed
Python max()	$O(n)$	$O(1)$

17. max() vs sorted()

Feature	max()	sorted()
Purpose	Find largest element	Sort all elements
Time Complexity	$O(n)$	$O(n \log n)$
Memory	$O(1)$	Usually higher
Best Choice	Find one maximum	Full ordering required

If you only need the highest-paid employee, use max() rather than sorting the entire collection.

18. Edge Cases

Empty Employee List

Java

```
Employee highest =
    employees.stream()
        .max(
            Comparator.comparing(
                Employee::getSalary
            )
        )
        .orElse(null);
```

Result

null

Python

```
highest = max(
    employees,
    key=lambda employee: employee[2],
    default=None
```

)

Result

None

One Employee

Name Salary

John 50000

Result

John

Multiple Employees with the Same Highest Salary

Name Salary

Robert 90000

Maria 90000

- Java `max()` returns one of the maximum elements according to the encounter order.
 - Python `max()` returns the first maximum encountered.
 - SQL `MAX()` with a subquery or `DENSE_RANK()` returns **all** employees tied for the maximum salary.
-

19. Common Interview Questions

1. What is the difference between finding the highest salary and finding the highest-paid employee?
2. Why is `max()` more efficient than sorting?
3. What is the difference between `Stream.max()` and `reduce()`?
4. Why does `Stream.max()` return an `Optional`?
5. How would you return all employees having the highest salary?
6. What happens if the employee list is empty?
7. How would you find the highest-paid employee in each department?
8. How would you find the second-highest-paid employee?
9. Why is `ORDER BY ... LIMIT 1` different from `MAX()` in SQL?

10. How would you compare employees using multiple fields?

20. Summary

- Finding the **employee with the highest salary** requires returning the **entire employee record**, not just the maximum salary value.
- The optimal algorithm maintains a reference to the current highest-paid employee while scanning the collection once, resulting in **$O(n)$** time and **$O(1)$** extra space.
- Java 1.5 uses a simple loop with a running maximum employee reference.
- Java 8 Stream API provides `Stream.max()` with `Comparator.comparing()` as the most expressive solution, while `reduce()` offers an alternative implementation.
- SQL can solve the problem using `ORDER BY salary DESC LIMIT 1`, but `MAX()` combined with a subquery or `DENSE_RANK()` is preferable when all employees tied for the highest salary should be returned.
- Python provides the built-in `max()` function with the `key` parameter, making the solution concise and efficient.
- This pattern is the foundation for many advanced interview questions, including **highest-paid employee by department**, **top N highest-paid employees**, **second-highest salary**, **salary rankings**, and **employee performance analytics**.

Partitioning Employees Using Java 1.5, Java 8 Stream API, SQL, and Python

Partitioning employees is one of the most common Java 8 Stream API interview questions. It tests your understanding of:

- Partitioning
- Boolean Classification
- Predicate
- HashMap
- Java Collections
- Java 8 Stream API
- SQL Conditional Aggregation
- Python Dictionaries
- Time Complexity
- Space Complexity

Unlike **grouping**, which can create **many groups**, **partitioning** always creates **exactly two groups** based on a boolean condition.

1. What is Partitioning?

Definition

Partitioning is the process of dividing data into **exactly two groups** based on whether each element satisfies a given condition (predicate).

The two groups are:

- **True** → Elements satisfying the condition.
- **False** → Elements not satisfying the condition.

Mathematically:

Partition = {
True -> Elements satisfying predicate,
False -> Elements not satisfying predicate
}

2. Problem Statement

Suppose we have the following employees.

ID	Name	Age	Department	Salary
101	John	25	IT	50000
102	David	35	HR	60000
103	Alice	32	IT	70000
104	Robert	45	Finance	90000
105	James	28	HR	65000
106	Peter	24	IT	45000

Partition employees based on:

Age > 30

Expected Output

True (Age > 30)

Name	Age
David	35
Alice	32
Robert	45

False (Age ≤ 30)

Name	Age
John	25
James	28
Peter	24

3. Partitioning vs Grouping

Feature	Partitioning	Grouping
Number of Groups	Exactly 2	One or more
Key Type	boolean	Any object
Java Collector	<code>partitioningBy()</code>	<code>groupingBy()</code>
Example	Age > 30	Department

Example

Grouping

IT

HR

Finance

Partitioning

TRUE

FALSE

4. Real-World Applications

Partitioning is used in:

- HR Systems
- Employee Eligibility
- Payroll
- Compliance Checks
- Banking
- Insurance
- Healthcare

Examples

- Employees older than 30
- Employees earning more than ₹50,000
- Active vs Inactive employees
- Permanent vs Contract employees
- Married vs Unmarried employees

5. Algorithm

Maintain two collections:

- One for employees satisfying the condition.
- One for employees not satisfying the condition.

Algorithm

Step 1

Create

True List

False List

Step 2

Read Employee

Step 3

Condition True?

Yes

Add to True List

No

Add to False List

Step 4

Repeat

Step 5

Return Both Lists

6. Flow Diagram

Start

↓

Read Employee

↓

Condition?

↓

TRUE

↓

True List

↓

FALSE

↓

False List

↓

Repeat

↓

End

7. Dry Run

Input

John 25

David 35

Alice 32

Robert 45

James 28

Peter 24

Condition

Age > 30

Processing

John

False
David

True
Alice

True
Robert

True
James

False
Peter

False

Result

True

David

Alice

Robert
False

John

James

Peter

8. Time Complexity

$O(n)$

Each employee is processed exactly once.

9. Space Complexity

$O(n)$

Because every employee is placed into one of the two output groups.

10. Java 1.5 Solution

Employee Class

```
public class Employee {  
  
    private int id;  
    private String name;  
    private int age;  
    private String department;  
    private double salary;  
  
    public Employee(int id,  
                    String name,  
                    int age,  
                    String department,  
                    double salary){  
  
        this.id=id;  
        this.name=name;  
        this.age=age;  
        this.department=department;  
        this.salary=salary;  
    }  
  
    public int getAge(){  
        return age;  
    }  
  
    @Override  
    public String toString(){  
  
        return name + " " + age;  
    }  
}
```

Java 1.5 Code

```
List<Employee> olderEmployees =  
    new ArrayList<Employee>();  
  
List<Employee> youngerEmployees =  
    new ArrayList<Employee>();  
  
for(Employee employee : employees){  
    if(employee.getAge() > 30){  
        olderEmployees.add(employee);  
    }else{  
        youngerEmployees.add(employee);  
    }  
}  
  
System.out.println("Older");  
  
System.out.println(olderEmployees);  
  
System.out.println("Younger");  
  
System.out.println(youngerEmployees);
```

Output

Older

David

Alice

Robert

Younger

John

James

Peter

11. Internal Working

Employee

↓

Age > 30 ?

↓

TRUE

↓

Older List

↓

FALSE

↓

Younger List

12. Java 8 Stream API

Java 8 introduced **Collectors.partitioningBy()**.

Method 1 (Recommended)

```
Map<Boolean, List<Employee>> result =
```

```
employees.stream()
```

```
.collect(
```

```
Collectors.partitioningBy(
```

```
employee -> employee.getAge() > 30
```

```
)
```

```
);
```

Output

true

David

Alice

Robert

false

John

James

Peter

Internal Structure

The returned map is conceptually:

```
{  
  true  = [David, Alice, Robert],  
  
  false = [John, James, Peter]  
}
```

Method 2

Partition based on salary.

```
Map<Boolean, List<Employee>> result =  
employees.stream()  
.collect(  
  Collectors.partitioningBy(  
    employee -> employee.getSalary() > 60000  
  )  
);
```

Method 3

Count employees in each partition.

```
Map<Boolean, Long> result =  
employees.stream()  
.collect(  
Collectors.partitioningBy(  
employee -> employee.getAge() > 30,  
Collectors.counting()  
)  
);
```

Output

true = 3

false = 3

13. Stream Pipeline

Employee List

↓

Stream

↓

Predicate

↓

partitioningBy()

↓

Map<Boolean,List<Employee>>

14. SQL Solution

SQL does not have a built-in PARTITIONING BY equivalent for boolean partitioning like Java Streams, but the same result can be achieved using conditional logic.

Method 1

Classify each employee.

```
SELECT
  id,
  name,
  age,
  CASE
    WHEN age > 30 THEN 'TRUE'
    ELSE 'FALSE'
  END AS age_partition
FROM Employee;
```

Output

Name	Partition
John	FALSE
David	TRUE
Alice	TRUE
Robert	TRUE
James	FALSE
Peter	FALSE

Method 2

Count each partition.

```
SELECT
CASE
  WHEN age > 30
  THEN 'Older'
  ELSE 'Younger'
END AS category,
COUNT(*)
FROM Employee
GROUP BY category;
```

Output

Category Count

Older 3

Younger 3

Method 3

Return only older employees.

```
SELECT *  
FROM Employee  
WHERE age > 30;
```

Return younger employees.

```
SELECT *  
FROM Employee  
WHERE age <= 30;
```

15. Python Solution

Method 1

Using dictionaries.

```
employees = [  
    ("John",25),  
    ("David",35),  
    ("Alice",32),  
    ("Robert",45),  
    ("James",28),  
    ("Peter",24)  
]
```

```
partition = {  
    True: [],  
    False: []  
}
```

for employee in employees:

```
    partition[  
        employee[1] > 30  
    ].append(employee)
```



```
print(partition)
```

Output

```
{
True:
[
('David',35),
('Alice',32),
('Robert',45)
],
False:
[
('John',25),
('James',28),
('Peter',24)
]
}
```

Method 2

Using list comprehensions.

```
older = [
    employee
    for employee in employees
    if employee[1] > 30
]

younger = [
    employee
    for employee in employees
    if employee[1] <= 30
]
```

16. Algorithm Comparison

Method	Time Complexity	Space Complexity
Java Loop	$O(n)$	$O(n)$
Java partitioningBy()	$O(n)$	$O(n)$
SQL CASE + GROUP BY	Depends on execution plan	Database-managed
Python Dictionary	$O(n)$	$O(n)$

17. partitioningBy() vs groupingBy()

Feature	partitioningBy()	groupingBy()
Number of Keys	2 (true/false)	Any number
Key Type	Boolean	Any object
Use Case	Boolean condition	Categories such as department, city, designation
Return Type	Map<Boolean, List<T>>	Map<K, List<T>>

Example

```
Collectors.partitioningBy(  
    employee -> employee.getAge() > 30  
);
```

versus

```
Collectors.groupingBy(  
    Employee::getDepartment  
);
```

18. Edge Cases

Empty Employee List

Output

```
{  
  true=[],  
  false=[]  
}
```

All Employees Match

Input

Name Age

David 35

Robert 45

Output

True

David

Robert

False

(empty)

No Employees Match

Input

Name Age

John 25

James 28

Output

True

(empty)

False

John

James

Age Exactly 30

Condition

Age > 30

Employee

Name Age

Alice 30

Output

False

If the requirement changes to:

Age >= 30

then Alice belongs to the true partition.

19. Common Interview Questions

1. What is partitioning?
2. What is the difference between partitioning and grouping?
3. Why does `partitioningBy()` always produce two keys?
4. How does `partitioningBy()` work internally?
5. Can `partitioningBy()` be combined with downstream collectors?
6. How do you count employees in each partition?
7. How do you partition employees by salary?
8. What happens when no employees satisfy the condition?
9. Why is the time complexity $O(n)$?
10. When should you choose `partitioningBy()` instead of `groupingBy()`?

20. Internal Working of `Collectors.partitioningBy()`

Conceptually, `partitioningBy()` performs the following:

```
Map<Boolean, List<Employee>> map = new HashMap<>();
```

```
map.put(true, new ArrayList<>());
```

```
map.put(false, new ArrayList<>());
```

```
for (Employee employee : employees) {
```

```
    boolean key = employee.getAge() > 30;
```

```
    map.get(key).add(employee);
```

```
}
```

The Stream API hides this implementation and produces the same result in a declarative style.

21. Summary

- **Partitioning** divides employees into **exactly two groups** based on a boolean condition.
- It differs from **grouping**, which can produce any number of groups based on key values.
- The algorithm processes each employee exactly once, evaluates the predicate, and places the employee into either the true or false partition, resulting in **$O(n)$** time complexity.
- Java 1.5 implements partitioning using two separate lists and a loop.
- Java 8 Stream API simplifies the implementation with `Collectors.partitioningBy()`, which returns a `Map<Boolean, List<Employee>>` and supports downstream collectors such as `counting()`, `mapping()`, and `summarizingDouble()`.
- SQL achieves equivalent behavior using CASE expressions, WHERE clauses, or conditional aggregation with GROUP BY.
- Python can implement partitioning using dictionaries keyed by boolean values or by creating two filtered lists with comprehensions.
- Partitioning is widely used in interview problems such as **active vs inactive employees**, **salary above or below a threshold**, **age-based eligibility**, **permanent vs contract employees**, and **employees satisfying a business rule versus those who do not**.

Grouping Employees by Department Using Java 1.5, Java 8 Stream API, SQL, and Python

Grouping employees by department is one of the **most frequently asked interview questions** in Java, SQL, and Python interviews. It is a fundamental data-processing operation used in reporting, analytics, and business applications.

This question evaluates your understanding of:

- Grouping
- HashMap
- Collections
- Java 1.5
- Java 8 Stream API
- SQL GROUP BY
- Python Dictionaries
- Time Complexity
- Space Complexity

1. What is Grouping?

Definition

Grouping is the process of organizing data into multiple groups based on a common key or attribute.

In this problem, the **department** is the grouping key.

Each employee belongs to exactly one department, and employees with the same department are placed into the same group.

2. Problem Statement

Suppose we have the following employees.

ID	Name	Department	Age	Salary
101	John	IT	25	50000
102	David	HR	35	60000
103	Alice	IT	32	70000
104	Robert	Finance	45	90000
105	James	HR	28	65000
106	Peter	IT	24	55000

Group employees by **department**.

Expected Output

IT

Name	Salary
John	50000
Alice	70000
Peter	55000

HR

Name	Salary
David	60000
James	65000

Finance

Name	Salary
Robert	90000

3. Real-World Applications

Grouping is widely used in:

- HR Management Systems
- Payroll Reports
- Banking Applications
- Business Intelligence

- Sales Reports
- Inventory Systems
- Hospital Management
- Data Analytics

Examples:

- Group employees by department.
- Group employees by city.
- Group employees by designation.
- Group employees by gender.
- Group employees by project.

4. Mathematical Definition

Let

Employees = E

Grouping function

Department(Employee)

Result

{
IT -> Employees,

HR -> Employees,

Finance -> Employees
}

5. Grouping vs Partitioning

Feature	Grouping	Partitioning
Number of Groups	One or more	Exactly two
Key Type	Any type	Boolean
Java Collector	groupingBy()	partitioningBy()
Example	Department	Age > 30

Grouping

IT

HR

Finance

Partitioning

True

False

6. Algorithm

Maintain a map where:

- **Key** = Department
 - **Value** = List of employees
-

Algorithm

Step 1

Create Map

Department -> Employee List

Step 2

Read Employee

Step 3

Department Exists?

Yes

Add employee to existing list.

No

Create new list.

Add employee.

Step 4

Repeat

Step 5

Return Map.

7. Flow Diagram

Start

↓

Read Employee

↓

Get Department

↓

Department Exists?

↓

Yes

↓

Add Employee

↓

No

↓

Create New List

↓

Repeat

↓

End

8. Dry Run

Input

John IT

David HR

Alice IT

Robert Finance

James HR

Peter IT

Processing

John

IT

Create List

Add John

David

HR

Create List

Add David

Alice

IT

Add Alice

Robert

Finance

Create List

James

HR

Add James

Peter

IT

Add Peter

Final

IT

John

Alice

Peter
HR

David

James
Finance

Robert

9. Time Complexity

$O(n)$

Each employee is processed exactly once.

10. Space Complexity

$O(n)$

Every employee appears in one of the grouped lists.

11. Java 1.5 Solution

Employee Class

```
public class Employee {  
    private int id;
```

```

private String name;
private String department;
private int age;
private double salary;

public Employee(int id,
                String name,
                String department,
                int age,
                double salary){

    this.id=id;
    this.name=name;
    this.department=department;
    this.age=age;
    this.salary=salary;
}

public String getDepartment(){
    return department;
}

@Override
public String toString(){
    return name + " " + salary;
}
}

```

Java 1.5 Code

```

Map<String, List<Employee>> result =
    new HashMap<String, List<Employee>>();

for(Employee employee : employees){

    String department =
        employee.getDepartment();

    List<Employee> list =
        result.get(department);

    if(list == null){

        list = new ArrayList<Employee>();

        result.put(department, list);

    }
}

```

```
list.add(employee);  
}  
System.out.println(result);
```

Output

```
{  
IT=[John, Alice, Peter],  
HR=[David, James],  
Finance=[Robert]  
}
```

12. Internal Working

Employee

↓

Department

↓

HashMap

↓

Department Exists?

↓

Yes

↓

Add Employee

↓

No

↓

Create List

↓

Add Employee

13. Java 8 Stream API

Java 8 introduced **Collectors.groupingBy()**.

Method 1 (Recommended)

```
Map<String, List<Employee>> result =  
employees.stream()  
.collect(  
Collectors.groupingBy(  
Employee::getDepartment  
)  
);
```

Output

```
{  
IT=[John, Alice, Peter],  
HR=[David, James],  
Finance=[Robert]  
}
```

Method 2

Count employees department-wise.

```
Map<String, Long> result =  
employees.stream()  
.collect(  
Collectors.groupingBy(  

```

```
Employee::getDepartment,  
Collectors.counting()  
)  
);
```

Output

IT = 3

HR = 2

Finance = 1

Method 3

Average salary department-wise.

```
Map<String, Double> result =  
employees.stream()  
.collect(  
Collectors.groupingBy(  
Employee::getDepartment,  
Collectors.averagingDouble(  
Employee::getSalary)  
)  
);
```

Method 4

Highest salary department-wise.

```
Map<String, Optional<Employee>> result =  
employees.stream()  
.collect(  

```

```
Collectors.groupingBy(  
    Employee::getDepartment,  
    Collectors.maxBy(  
        Comparator.comparing(  
            Employee::getSalary)))  
));
```

14. Stream Pipeline

Employee List

↓

Stream

↓

groupingBy()

↓

Department

↓

Map<Department,List<Employee>>

15. Internal Working of groupingBy()

Conceptually, Collectors.groupingBy() performs logic similar to:

```
Map<String, List<Employee>> map =  
    new HashMap<>();  
  
for(Employee employee : employees){  
    String department =  
        employee.getDepartment();  
  
    map.computeIfAbsent(  
        department,
```



```
        key -> new ArrayList<>())
    .add(employee);
}
```

In Java 1.5, `computeIfAbsent()` is unavailable, so the explicit null check shown earlier is used.

16. SQL Solution

Method 1

Count employees department-wise.

```
SELECT department,
       COUNT(*) AS employee_count
FROM Employee
GROUP BY department;
```

Output

Department	Employee Count
IT	3
HR	2
Finance	1

Method 2

Average salary department-wise.

```
SELECT department,
       AVG(salary) AS average_salary
FROM Employee
GROUP BY department;
```

Method 3

Maximum salary department-wise.

```
SELECT department,
       MAX(salary) AS highest_salary
FROM Employee
GROUP BY department;
```

Method 4

List employees ordered by department.

```
SELECT *  
FROM Employee  
ORDER BY department,  
        name;
```

Note: SQL GROUP BY returns aggregated rows. If your goal is to display employees grouped together, ORDER BY department is commonly used. To return nested employee lists per department, database-specific JSON/XML aggregation functions are typically required.

17. Python Solution

Method 1 (Recommended)

Using a dictionary.

```
employees = [  
    ("John", "IT"),  
    ("David", "HR"),  
    ("Alice", "IT"),  
    ("Robert", "Finance"),  
    ("James", "HR"),  
    ("Peter", "IT")  
]  
  
groups = {}  
  
for employee in employees:  
    department = employee[1]  
  
    groups.setdefault(  
        department,  
        []  
    ).append(employee)  
  
print(groups)
```

Output

```
{  
'IT':  
[  
    ('John', 'IT'),  
    ('Alice', 'IT'),  
    ('Peter', 'IT')  
]
```

```

],

'HR':
[
('David','HR'),
('James','HR')
],

'Finance':
[
('Robert','Finance')
]
}

```

Method 2

Using defaultdict.

```

from collections import defaultdict

groups = defaultdict(list)

for employee in employees:

    groups[employee[1]].append(employee)

print(dict(groups))

```

18. Algorithm Comparison

Method	Time Complexity	Space Complexity
Java HashMap	O(n)	O(n)
Java groupingBy()	O(n)	O(n)
SQL GROUP BY	Depends on execution plan (hash or sort aggregation)	Database-managed
Python Dictionary	O(n)	O(n)

19. groupingBy() vs partitioningBy()

Feature	groupingBy()	partitioningBy()
Number of Groups	Multiple	Exactly 2
Key Type	Any object	Boolean
Return Type	Map<K, List<T>>	Map<Boolean, List<T>>

Example	Department	Salary > 50000
----------------	------------	----------------

20. Edge Cases

Empty Employee List

Output

{

Single Department

Input

Name Department

John IT

Alice IT

Output

{
IT

John

Alice
}

Every Employee in a Different Department

Input

Name	Department
John	IT
David	HR
Robert	Finance

Output

{
IT=[John],

```
HR=[David],  
Finance=[Robert]  
}
```

Null Department

Java example:

```
String department =  
    employee.getDepartment();  
  
if (department == null) {  
    department = "Unknown";  
}
```

Then group using the normalized value.

21. Common Interview Questions

1. What is grouping?
 2. What is the difference between grouping and partitioning?
 3. How does `Collectors.groupingBy()` work internally?
 4. Why does `groupingBy()` return a `Map`?
 5. How do you count employees in each department?
 6. How do you calculate the average salary for each department?
 7. How do you find the highest-paid employee in each department?
 8. Can `groupingBy()` be combined with downstream collectors?
 9. Why is the algorithm $O(n)$?
 10. How would you group employees by multiple fields, such as department and designation?
-

22. Summary

- **Grouping** organizes employees into multiple categories based on a common key, such as department.
- The most efficient algorithm uses a map where the **key** is the department and the **value** is the list of employees belonging to that department, processing each employee exactly once ($O(n)$ time).
- Java 1.5 implements grouping using a `HashMap` and manual list creation.
- Java 8 Stream API simplifies the implementation with `Collectors.groupingBy()` and supports powerful downstream collectors such as `counting()`, `mapping()`, `averagingDouble()`, `summarizingDouble()`, and `maxBy()`.

- SQL uses GROUP BY together with aggregate functions like COUNT(), SUM(), AVG(), MIN(), and MAX() to summarize data by department. To display grouped employee rows rather than aggregates, ORDER BY or database-specific aggregation functions are used.
- Python commonly uses dictionaries or collections.defaultdict to build grouped collections efficiently.
- Grouping is a core concept for interview questions involving **department-wise counts, salary analysis, employee statistics, nested grouping, multi-level grouping, and business reporting.**

Finding the Frequency of Characters Using Java 1.5, Java 8 Stream API, SQL, and Python

Finding the **frequency of characters** is one of the most common programming interview questions asked in companies like Oracle, Infosys, TCS, Accenture, IBM, Capgemini, Wipro, Cognizant, Amazon, Microsoft, Google, and many others.

This problem tests your understanding of:

- Strings
- Arrays
- HashMap
- Collections
- Java 1.5
- Java 8 Stream API
- SQL
- Python Dictionaries
- Time Complexity
- Space Complexity

1. Problem Definition

Given a string

Programming

Find the frequency of every character.

Expected Output

Character	Frequency
P	1
r	2
o	1

g	2
a	1
m	2
i	1
n	1

2. What is Character Frequency?

Definition

Character frequency means:

The number of times each character appears in a string.

Example

Input

APPLE

Output

Character	Count
A	1
P	2
L	1
E	1

3. Real-World Applications

Character frequency is widely used in:

- Text Analytics
 - Search Engines
 - Data Compression (Huffman Coding)
 - Spell Checkers
 - Cryptography
 - Password Analysis
 - NLP (Natural Language Processing)
 - AI Chatbots
-

4. Mathematical Definition

Given

S = "Programming"

Frequency Function

Frequency(c)

=

Number of occurrences of character c

Example

Frequency(g)

=

2

5. Logic

The idea is simple.

Maintain a map.

Character

↓

Count

Whenever a character appears

Increase its count by one.

6. Algorithm

Step 1

Create HashMap<Character,Integer>

Step 2

Read one character.

Step 3

Character Exists?

Yes

Increase Count

No

Insert Count = 1

Step 4

Repeat

Step 5

Print Map

7. Flow Diagram

Start

↓

Read Character

↓

Character Exists?

↓

Yes

↓

Count++

↓

No

↓

Insert Count = 1

↓

Repeat

↓

Print Frequency

↓

End

8. Dry Run

Input

JAVA

Initially

{}

Read

J
{J=1}

Read

A
{J=1,A=1}

Read

V
{J=1,A=1,V=1}

Read

A
{J=1,A=2,V=1}

Output

Character	Count
J	1
A	2
V	1

9. Time Complexity

$O(n)$

Every character is processed once.

10. Space Complexity

$O(k)$

where

- **n** = length of string
 - **k** = number of distinct characters
-

11. Java 1.5 Solution

```
import java.util.*;

public class CharacterFrequency {

    public static void main(String[] args) {

        String str = "Programming";

        Map<Character,Integer> map =
            new HashMap<Character,Integer>();

        for(int i=0;i<str.length();i++){

            char ch = str.charAt(i);

            if(map.containsKey(ch)){

                map.put(ch,map.get(ch)+1);

            }else{

                map.put(ch,1);

            }

        }

    }

}
```

```
    }  
    System.out.println(map);  
}  
}
```

Output

```
{P=1, r=2, o=1, g=2, a=1, m=2, i=1, n=1}
```

12. Internal Working

Input

JAVA

Processing

J

↓

```
{J=1}
```

A

↓

```
{J=1,A=1}
```

V

↓

```
{J=1,A=1,V=1}
```

A

↓

```
{J=1,A=2,V=1}
```

13. Java 8 Stream API

Java 8 provides a much cleaner solution.

Method 1 (Recommended)

```
import java.util.*;
import java.util.function.Function;
import java.util.stream.Collectors;

String str = "Programming";

Map<Character, Long> frequency =

str.chars()

.mapToObj(c -> (char)c)

.collect(

Collectors.groupingBy(

Function.identity(),

Collectors.counting()

)

);

System.out.println(frequency);

Output

{P=1, r=2, o=1, g=2, a=1, m=2, i=1, n=1}
```

How chars() Works

Programming

↓

80

114

111

103

...

ASCII/Unicode values are converted back into Character objects.

Pipeline

String

↓

chars()

↓

Character Stream

↓

groupingBy()

↓

counting()

↓

Map<Character,Long>

14. Alternative Java 8 Solution

Using toMap().

Map<Character,Integer> map =

str.chars()

.mapToObj(c -> (char)c)

.collect(

Collectors.toMap(

c -> c,

c -> 1,

Integer::sum

)

);

15. SQL Solution

Unlike Java or Python, SQL does not iterate through characters directly. A common approach is to first split the string into individual characters using a recursive query or numbers table, then group them.

Example (MySQL 8+ using a recursive common table expression):

```
WITH RECURSIVE numbers(n) AS (  
    SELECT 1  
    UNION ALL  
    SELECT n + 1  
    FROM numbers  
    WHERE n < LENGTH('Programming')  
)  
SELECT  
    SUBSTRING('Programming', n, 1) AS character,  
    COUNT(*) AS frequency  
FROM numbers  
GROUP BY SUBSTRING('Programming', n, 1)  
ORDER BY character;
```

Output

Character	Frequency
P	1
a	1
g	2
i	1
m	2
n	1
o	1
r	2

Note: The exact SQL syntax differs across databases (Oracle, SQL Server, PostgreSQL, MySQL), but the overall approach is the same:

1. Split the string into characters.
2. Group by each character.
3. Count occurrences.

16. Python Solution

Method 1 (Recommended)

Using a dictionary.

```
text = "Programming"

frequency = {}

for ch in text:

    frequency[ch] = frequency.get(ch,0)+1

print(frequency)
```

Output

```
{
'P':1,
'r':2,
'o':1,
'g':2,
'a':1,
'm':2,
'i':1,
'n':1
}
```

Method 2

Using collections.Counter.

```
from collections import Counter

text = "Programming"

frequency = Counter(text)

print(frequency)
```

Output

```
Counter({
'r':2,
'g':2,
'm':2,
'P':1,
'o':1,
'a':1,
'i':1,
'n':1
})
```


Counter is specifically designed for counting hashable objects and is the most concise Python solution.

17. Algorithm Comparison

Method	Time Complexity	Space Complexity
Java 1.5 HashMap	$O(n)$	$O(k)$
Java 8 groupingBy()	$O(n)$	$O(k)$
Java 8 toMap()	$O(n)$	$O(k)$
SQL GROUP BY	Depends on query plan	Database-managed
Python Dictionary	$O(n)$	$O(k)$
Python Counter	$O(n)$	$O(k)$

18. HashMap Internal Working

Input

BANANA

Processing

B

↓

{B=1}

A

↓

{B=1,A=1}

N

↓

{B=1,A=1,N=1}

A

↓

{B=1,A=2,N=1}

N

↓

{B=1,A=2,N=2}

A

↓

{B=1,A=3,N=2}

19. Interview Variations

Once you know character frequency, interviewers often ask:

1. First Non-Repeated Character

Input

Programming

Output

P

2. First Repeated Character

Input

Programming

Output

r

3. Most Frequent Character

Input

Programming

Output

r

g

m

Frequency = 2

4. Least Frequent Character

Output

P

o

a

i

n

5. Count Only Alphabets

Ignore

- Numbers
 - Spaces
 - Symbols
-

6. Ignore Case

Input

JavaJAVA

Convert to lowercase first.

javajava

Output

j=2

a=4

v=2

20. Edge Cases

Empty String

Input

Output

{}

Single Character

Input

A

Output

A=1

All Characters Same

Input

AAAAAA

Output

A=6

Spaces Included

Input

Java Programming

You can either:

- Count spaces as characters.
- Ignore spaces by checking:

Java

```
if(ch != ' ')
```

Python

```
if ch != ' ':
```

21. Common Interview Questions

1. What is character frequency?
 2. Why is a HashMap suitable for this problem?
 3. How does `Collectors.groupingBy()` work internally?
 4. What is the difference between `groupingBy()` and `toMap()`?
 5. Why does `Collectors.counting()` return Long instead of Integer?
 6. Why is the algorithm $O(n)$?
 7. How would you ignore spaces or punctuation?
 8. How would you make the counting case-insensitive?
 9. How would you find the first non-repeated character using the frequency map?
 10. How would you count the frequency of words instead of characters?
-

22. Summary

- **Character frequency** is the process of counting how many times each character appears in a string.
- The most efficient algorithm uses a **HashMap** (or dictionary) where the **key** is the character and the **value** is its occurrence count.
- Each character is processed exactly once, giving a **time complexity of $O(n)$** and **space complexity of $O(k)$** , where k is the number of distinct characters.
- Java 1.5 implements this using a HashMap and explicit iteration.
- Java 8 Stream API provides elegant solutions using `Collectors.groupingBy()` with `Collectors.counting()` or `Collectors.toMap()`.
- SQL requires splitting the string into individual characters before applying GROUP BY and COUNT().
- Python offers both a manual dictionary approach and the specialized `collections.Counter` class.
- Understanding this problem provides the foundation for solving many related interview questions, such as **duplicate character detection**, **first non-repeated character**, **most frequent character**, **anagram checking**, **character statistics**, and **text analysis**.

Reversing a String Using Java 1.5, Java 8 Stream API, SQL, and Python

Reversing a string is one of the most frequently asked interview questions for Java, SQL, and Python developers. It is a fundamental problem that helps interviewers assess your understanding of:

- Strings
- Arrays

- Character Manipulation
 - Loops
 - Recursion
 - Java Collections
 - Java 8 Stream API
 - SQL Recursive Queries
 - Python String Operations
 - Time Complexity
 - Space Complexity
-

1. Problem Definition

Given a string:

Programming

Reverse the string.

Expected Output

gnimmargorP

2. What is String Reversal?

Definition

String reversal is the process of rearranging the characters of a string in the opposite order.

If the original string is:

ABCDE

The reversed string is:

EDCBA

The first character becomes the last, the second becomes the second last, and so on.

3. Real-World Applications

String reversal is used in:

- Palindrome checking

- Cryptography
 - Data encoding
 - Text processing
 - Compiler design
 - Interview coding exercises
 - Algorithm learning
-

4. Mathematical Definition

Let

$$S = S_0 S_1 S_2 \dots S_{n-1}$$

The reversed string is

Reverse(S)

=

$$S_{n-1} \dots S_2 S_1 S_0$$

Example

JAVA

becomes

AVAJ

5. Logic

The simplest idea is:

- Start from the last character.
 - Move towards the first character.
 - Append each character to a new string.
-

6. Algorithm

Step 1

Read String

Step 2

Create Empty Result

Step 3

Start from last index

Step 4

Append character

Step 5

Move left

Step 6

Repeat

Step 7

Return Result

7. Flow Diagram

Start



Read String



Last Character



Append



Move Left



Repeat

↓

End

↓

Return Reversed String

8. Dry Run

Input

JAVA

Characters

J

A

V

A

Processing

A

↓

AV

↓

AVA

↓

AVAJ

Output

AVAJ

9. Time Complexity

$O(n)$

Every character is processed exactly once.

10. Space Complexity

Using a new string or StringBuilder:

$O(n)$

because a reversed copy is created.

11. Java 1.5 Solutions

Method 1 (Recommended)

Using StringBuilder.

```
public class ReverseString {  
    public static void main(String[] args) {  
        String str = "Programming";  
        StringBuilder reverse =  
            new StringBuilder();  
        for(int i = str.length()-1;  
            i >= 0;  
            i--){  
            reverse.append(  
                str.charAt(i));  
        }  
        System.out.println(reverse);  
    }  
}
```

Output

gnimmargorP

Method 2

Using Character Array

```
String str = "Programming";  
  
char[] chars = str.toCharArray();  
  
for(int i = chars.length-1; i>=0; i--){  
    System.out.print(chars[i]);  
}
```

Method 3

Using StringBuilder.reverse()

```
String str = "Programming";  
  
String reversed =  
    new StringBuilder(str)  
        .reverse()  
        .toString();  
  
System.out.println(reversed);
```

How StringBuilder.reverse() Works Internally

Internally it swaps characters from both ends toward the center.

Example

JAVA

Swap

J ↔ A

Result

AAVJ

Swap

A ↔ V

Final

AVAJ

The algorithm performs approximately **$n/2$ swaps**, resulting in **$O(n)$** time complexity.

12. Java 8 Stream API

Unlike collections, Java Streams do not provide a built-in reverse operation because streams are designed for forward traversal.

Method 1

Using IntStream.

```
import java.util.stream.IntStream;
```

```
String str = "Programming";
```

```
String reversed =
```

```
IntStream.range(0, str.length())
```

```
.mapToObj(i ->  
    String.valueOf(  
        str.charAt(  
            str.length()-1-i)))
```

```
.reduce("", String::concat);
```

```
System.out.println(reversed);
```

Output

```
gnimmargorP
```

Method 2

Using StringBuilder inside Stream

```
String reversed =
```

```
str.chars()
```

```
.collect(  

```

```
StringBuilder::new,  
(sb,c) -> sb.insert(0,(char)c),  
StringBuilder::append)  
.toString();  
System.out.println(reversed);
```

Note: This approach is educational but less efficient because insert(0, ...) shifts existing characters on every insertion, leading to **$O(n^2)$** time complexity.

13. Stream Pipeline

```
String  
↓  
chars()  
↓  
Reverse Order  
↓  
Collect  
↓  
Reversed String
```

14. SQL Solution

SQL is not intended for character manipulation, but string reversal can still be implemented.

Method 1 (Recursive CTE)

Example (MySQL 8+, PostgreSQL, SQL Server with minor syntax differences):

```
WITH RECURSIVE reverse_cte AS (  
  SELECT  
    'Programming' AS original,  
    LENGTH('Programming') AS pos,  
    '' AS reversed
```

```
UNION ALL
SELECT
    original,
    pos - 1,
    CONCAT(reversed, SUBSTRING(original, pos, 1))
FROM reverse_cte
WHERE pos > 0
)
SELECT reversed
FROM reverse_cte
WHERE pos = 0;
```

Output

gnimmargorP

Method 2

Some database systems provide built-in functions.

Example (SQL Server)

```
SELECT REVERSE('Programming');
```

Output

gnimmargorP

Note: Built-in reverse functions are **database-specific**. Oracle, MySQL, PostgreSQL, and SQL Server differ in the functions they provide.

15. Python Solutions

Method 1 (Recommended)

Using slicing.

```
text = "Programming"
```

```
print(text[::-1])
```

Output

gnimmargorP

How It Works

[start : end : step]

Using

[::-1]

means:

- Start from the end.
- Move backward one character at a time.

Method 2

Using reversed().

```
text = "Programming"
```

```
print("".join(reversed(text)))
```

Method 3

Using a loop.

```
text = "Programming"
```

```
result = ""
```

```
for ch in text:
```

```
    result = ch + result
```

```
print(result)
```

This loop prepends each character to the result. Because Python strings are immutable, repeated concatenation makes this approach **$O(n^2)$** .

16. Algorithm Comparison

Method	Time Complexity	Space Complexity
Java Loop + StringBuilder	$O(n)$	$O(n)$
Java	$O(n)$	$O(n)$

StringBuilder.reverse()		
Java Stream (IntStream)	$O(n^2)^*$	$O(n)$
SQL Recursive CTE	Depends on database implementation	Database-managed
SQL Server REVERSE()	$O(n)$ (implementation-dependent)	Database-managed
Python Slicing	$O(n)$	$O(n)$
Python reversed()	$O(n)$	$O(n)$

*The example using `reduce("", String::concat)` repeatedly creates new String objects, making it $O(n^2)$. A stream solution that collects into a StringBuilder while traversing indices from end to start can achieve $O(n)$.

17. Different Algorithms

Algorithm 1

Reverse using a loop.

Programming

↓

gnimmargorP

Algorithm 2

Two-Pointer Technique

Characters

J

A

V

A

Swap

$J \leftrightarrow A$

Then

$A \leftrightarrow V$

Result

AVAJ

This algorithm is commonly used when reversing a mutable character array.

Algorithm 3

Recursion

JAVA

↓

Reverse(AVA)

↓

Reverse(VA)

↓

Reverse(A)

↓

A

↓

AV

↓

AVA

↓

AVAJ

18. Edge Cases

Empty String

Input

Output

Single Character

Input

A

Output

A

Palindrome

Input

MADAM

Output

MADAM

The reversed string is identical.

Spaces

Input

Java Programming

Output

gnimmargorP avaJ

Spaces are treated like ordinary characters.

Unicode Characters

Modern Java (`StringBuilder.reverse()`), Python, and most SQL databases support Unicode strings. However, characters represented as surrogate pairs or combining sequences may require additional care in some languages and libraries.

19. Common Interview Questions

1. What is string reversal?
 2. Why is `StringBuilder` preferred over string concatenation in Java?
 3. How does `StringBuilder.reverse()` work internally?
 4. Why doesn't Java Stream have a built-in `reverse()` operation?
 5. What is the difference between reversing a string and sorting a string?
 6. How would you reverse a string without using built-in methods?
 7. How would you reverse words instead of characters?
 8. How would you reverse only alphabetic characters?
 9. How would you reverse a string recursively?
 10. How would you check whether a string is a palindrome using string reversal?
-

20. Summary

- **String reversal** rearranges the characters of a string in the opposite order.
- The most efficient approach iterates from the last character to the first (or swaps characters in a mutable array), resulting in **$O(n)$** time complexity.
- Java 1.5 commonly uses a loop with `StringBuilder`, while `StringBuilder.reverse()` provides a built-in optimized implementation.
- Java 8 Streams do not include a native reverse operation, so reversing typically involves `IntStream` over indices or collecting characters into a `StringBuilder`.
- SQL implementations vary: SQL Server provides a built-in `REVERSE()` function, while other databases often require recursive queries or custom logic.
- Python offers concise solutions using slicing (`[::-1]`) and the `reversed()` function.
- Understanding string reversal is fundamental because it forms the basis for many interview problems, including **palindrome checking, word reversal, string rotation, anagram validation, character manipulation, and text-processing algorithms**.

Finding a Palindrome Using Java 1.5, Java 8 Stream API, SQL, and Python

Finding whether a string is a **palindrome** is one of the most frequently asked interview questions in Java, Python, SQL, C++, and many other programming languages.

This problem tests your understanding of:

- Strings
- Character Comparison

- Two-Pointer Algorithm
 - String Reversal
 - Java Collections
 - Java 8 Stream API
 - SQL String Functions
 - Python
 - Time Complexity
 - Space Complexity
-

1. What is a Palindrome?

Definition

A **palindrome** is a word, string, number, or sequence that reads the same **forward and backward**.

If reversing the string produces the same string, then it is a palindrome.

Examples

Palindrome Strings

Input	Reverse	Palindrome?
MADAM	MADAM	✓Yes
LEVEL	LEVEL	✓Yes
RADAR	RADAR	✓Yes
ROTATOR	ROTATOR	✓Yes
NOON	NOON	✓Yes

Non-Palindrome Strings

Input	Reverse	Palindrome?
JAVA	AVAJ	✗No
PROGRAM	MARGORP	✗No
HELLO	OLLEH	✗No

2. Problem Statement

Given a string

MADAM

Determine whether it is a palindrome.

Expected Output

Palindrome

3. Real-World Applications

Palindrome checking is used in:

- Compiler Design
 - DNA Sequence Analysis
 - Natural Language Processing
 - Cryptography
 - Data Validation
 - Competitive Programming
 - Interview Questions
-

4. Mathematical Definition

Let

$$S = s_0 s_1 s_2 \dots s_{n-1}$$

A string is a palindrome if

$$S = \text{Reverse}(S)$$

or equivalently

$$s[i] = s[n - 1 - i]$$

for every valid index i .

5. Logic

There are several ways to determine whether a string is a palindrome.

Method 1

Reverse the string.

Compare

Original == Reversed

If equal

Palindrome

Otherwise

Not Palindrome

Method 2 (Most Efficient)

Compare characters from both ends.

M A D A M
↑ ↑

Compare

M == M

Move pointers

A D A
↑ ↑

Compare

A == A

Move pointers

D
↑

Done

Palindrome.

6. Two-Pointer Algorithm

Step 1

left = 0

right = length-1

Step 2

Compare characters.

Equal?

Yes

Move both pointers.

left++

right--

No

Return False

Step 3

Repeat

Step 4

Pointers crossed?

Return True

7. Flow Diagram

Start

↓

Left & Right

↓

Characters Equal?

↓

No

↓

Not Palindrome

↓

Yes

↓

Move Pointers

↓

Repeat

↓

Palindrome

8. Dry Run

Input

LEVEL

Iteration 1

L == L

Iteration 2

E == E

Iteration 3

V

Pointers meet.

Output

Palindrome

9. Time Complexity

For the two-pointer algorithm:

$O(n)$

Only half the string is examined.

10. Space Complexity

Two-pointer approach

$O(1)$

No extra string is created.

If using string reversal, extra space is:

$O(n)$

11. Java 1.5 Solutions

Method 1 (Recommended: Two Pointers)

```
public class Palindrome {  
  
    public static void main(String[] args) {  
  
        String str = "MADAM";  
  
        int left = 0;  
        int right = str.length() - 1;  
  
        boolean palindrome = true;  
  
        while (left < right) {  
  
            if (str.charAt(left) != str.charAt(right)) {  
  
                palindrome = false;  
                break;  
            }  
  
            left++;  
            right--;  
  
        }  
  
        System.out.println(palindrome);  
    }  
}
```

```
}  
}
```

Output

true

Method 2

Using `StringBuilder.reverse()`.

```
String str = "MADAM";
```

```
String reverse =  
    new StringBuilder(str)  
        .reverse()  
        .toString();
```

```
System.out.println(  
    str.equals(reverse));
```

Output

true

12. Internal Working (Two Pointers)

Input

```
M A D A M  
↑   ↑
```

Compare

M == M

Move

```
A D A  
↑   ↑
```

Compare

A == A

Move

D

Finished.

13. Java 8 Stream API

Java Streams do not provide a dedicated palindrome operation, but it can be implemented using IntStream.

Method 1 (Recommended)

```
import java.util.stream.IntStream;
```

```
String str = "MADAM";
```

```
boolean palindrome =
```

```
IntStream.range(  
    0,  
    str.length() / 2)
```

```
.allMatch(i ->
```

```
str.charAt(i) ==
```

```
str.charAt(  
    str.length() - 1 - i));
```

```
System.out.println(palindrome);
```

Output

true

Internal Working

For "MADAM":

i = 0

M == M

i = 1

A == A

All comparisons succeed.

Method 2

Using reversal.

```
boolean palindrome =  
    str.equals(  
        new StringBuilder(str)  
            .reverse()  
            .toString()  
    );
```

14. Stream Pipeline

String

↓

IntStream

↓

Compare Characters

↓

allMatch()

↓

Boolean

15. SQL Solution

Some databases (such as SQL Server) provide a built-in REVERSE() function.

SQL Server

```
SELECT  
CASE  
  
WHEN 'MADAM' = REVERSE('MADAM')  
  
THEN 'Palindrome'  
  
ELSE 'Not Palindrome'  
  
END AS Result;
```

Output

Result
Palindrome

Generic SQL (Recursive Example)

If a built-in reverse function is unavailable, first reverse the string using a recursive query (or a user-defined function), then compare the original string with the reversed string.

Conceptually:

```
IF original = reversed  
THEN Palindrome  
ELSE Not Palindrome
```

The exact implementation depends on the database (Oracle, PostgreSQL, MySQL, etc.).

16. Python Solutions

Method 1 (Recommended)

Using slicing.

```
text = "MADAM"  
  
print(text == text[::-1])
```

Output

True

Method 2

Using two pointers.

```
text = "MADAM"
```

```
left = 0
```

```
right = len(text) - 1
```

```
palindrome = True
```

```
while left < right:
```

```
    if text[left] != text[right]:
```

```
        palindrome = False
        break
```

```
    left += 1
```

```
    right -= 1
```

```
print(palindrome)
```

17. Algorithm Comparison

Method	Time Complexity	Space Complexity
Java Two Pointers	$O(n)$	$O(1)$
Java Reverse String	$O(n)$	$O(n)$
Java Stream IntStream	$O(n)$	$O(1)$
SQL REVERSE()	$O(n)$ (implementation-dependent)	Database-managed
Python Two Pointers	$O(n)$	$O(1)$
Python Slicing	$O(n)$	$O(n)$

18. Two-Pointer vs Reverse String

Feature	Two Pointers	Reverse String
Extra Memory	$O(1)$	$O(n)$
Speed	Faster	Slightly slower
Interview Preference	✔Recommended	Acceptable
Creates New String	No	Yes

The two-pointer algorithm is generally preferred in interviews because it is both time-efficient and memory-efficient.

19. Edge Cases

Empty String

Input

Output

Palindrome

An empty string is considered a palindrome because it reads the same forwards and backwards.

Single Character

Input

A

Output

Palindrome

Even-Length Palindrome

Input

NOON

Output

Palindrome

Odd-Length Palindrome

Input

RADAR

Output

Palindrome

Case Sensitivity

Input

Madam

Simple comparison

Madam

!=

madaM

Result

Not Palindrome

To perform a **case-insensitive** check:

Java

String normalized = str.toLowerCase();

Python

text = text.lower()

Ignoring Spaces and Punctuation

Input

A man, a plan, a canal: Panama

Normalize the string first by:

- Converting to one case.
- Removing spaces.
- Removing punctuation.

Normalized value

amanaplanacanalpanama

Now apply the palindrome algorithm.

20. Common Interview Questions

1. What is a palindrome?
2. Why is the two-pointer algorithm preferred?
3. What is the time complexity of palindrome checking?
4. What is the space complexity of the two-pointer solution?
5. Why doesn't Java Stream provide a palindrome method?
6. How would you ignore case differences?
7. How would you ignore spaces and punctuation?
8. How would you check whether a number is a palindrome?
9. How would you find the longest palindromic substring?
10. What is the difference between checking a palindrome using reversal and using two pointers?

21. Related Interview Questions

Once you understand palindrome checking, interviewers often ask:

1. Reverse a string.
2. Reverse words in a sentence.
3. Check whether a number is a palindrome.
4. Find the longest palindromic substring.
5. Count all palindromic substrings.
6. Find the longest palindrome that can be formed from given characters.
7. Check whether two strings are mirror images.
8. Ignore spaces and punctuation while checking a palindrome.
9. Ignore case while checking a palindrome.
10. Find the first non-palindromic substring.

22. Summary

- A **palindrome** is a string that reads the same from left to right and from right to left.
- The **two-pointer algorithm** is the most efficient solution because it compares characters from both ends toward the center, requiring **$O(n)$** time and **$O(1)$** extra space.
- Java 1.5 commonly implements palindrome checking using either the two-pointer approach or by reversing the string with `StringBuilder`.
- Java 8 Stream API can implement the same logic elegantly using `IntStream.range()` together with `allMatch()`.
- SQL solutions typically compare the original string with its reversed form using a built-in `REVERSE()` function (where available) or equivalent custom logic.
- Python provides concise solutions using slicing (`text[::-1]`) as well as the memory-efficient two-pointer technique.
- Mastering palindrome checking provides the foundation for solving advanced interview problems such as **palindromic numbers**, **longest palindromic substring**, **counting**

palindromic substrings, case-insensitive palindrome validation, and palindrome checking after removing spaces and punctuation.

Finding Even Numbers Using Java 1.5, Java 8 Stream API, SQL, and Python

Finding **even numbers** is one of the most fundamental programming and interview questions. It is commonly asked because it tests your understanding of:

- Arithmetic Operators
- Modulus (%) Operator
- Conditional Statements
- Loops
- Collections
- Java 1.5
- Java 8 Stream API
- SQL Filtering
- Python
- Time Complexity
- Space Complexity

Although simple, the logic of checking for even numbers is used in many real-world algorithms such as array processing, mathematical computations, data filtering, and optimization.

1. What is an Even Number?

Definition

An **even number** is an integer that is **exactly divisible by 2**, leaving a remainder of **0**.

Mathematically,

Even Number

=

`Number % 2 == 0`

Examples:

Number	Even?
2	✓
4	✓
8	✓
10	✓

100	✓
0	✓
-8	✓
7	✗
13	✗

2. Problem Statement

Given a list of numbers:

5 8 10 15 18 21 24 30

Find all even numbers.

Expected Output

8

10

18

24

30

3. Mathematical Definition

For any integer **n**

n is Even

if

$n \bmod 2 = 0$

where

mod

=

Remainder

Example

$18 \% 2$

=

0

Therefore

18

is Even

4. Understanding the Modulus Operator

The modulus operator % returns the **remainder** after division.

Examples

$10 \% 2$

=

0

$11 \% 2$

=

1

$18 \% 2$

=

0

$25 \% 2$

=

1

Decision Rule

If

$\text{Number \% } 2 == 0$

↓

Even

Else

Odd

5. Algorithm

Step 1

Read Number

Step 2

Calculate

$\text{Number} \% 2$

Step 3

Result == 0 ?

Yes

Print Number

No

Ignore

Step 4

Repeat

Step 5

End

6. Flow Diagram

Start

↓

Read Number

↓

Number % 2 == 0 ?

↓

Yes

↓

Print Number

↓

No

↓

Next Number

↓

Repeat

↓

End

7. Dry Run

Input

5

8

10

15

Processing

5 % 2

=

1

Odd

8 % 2

=

0

Even
 $10 \% 2$

=

0

Even
 $15 \% 2$

=

1

Odd

Output

8

10

8. Time Complexity

For **n** numbers

$O(n)$

Each number is checked exactly once.

9. Space Complexity

Printing directly:

$O(1)$

If storing the result in another list:

$O(k)$

where **k** is the number of even numbers.

10. Java 1.5 Solution

```
import java.util.*;

public class EvenNumbers {

    public static void main(String[] args) {

        List<Integer> numbers =
            Arrays.asList(5,8,10,15,18,21,24,30);

        for(Integer number : numbers){

            if(number % 2 == 0){

                System.out.println(number);

            }

        }

    }

}
```

Output

```
8
10
18
24
30
```

Internal Working

Input

```
5
8
10
15
```


Processing

5

↓

Odd

↓

Ignore

8

↓

Even

↓

Print

11. Java 8 Stream API

Java 8 introduced **filter()**, which is the preferred approach.

Method 1 (Recommended)

```
import java.util.Arrays;

Arrays.asList(5,8,10,15,18,21,24,30)

.stream()

.filter(number -> number % 2 == 0)

.forEach(System.out::println);
```

Output

8
10
18
24
30

Method 2

Collect into a list.

```
List<Integer> evenNumbers =  
numbers.stream()  
.filter(number -> number % 2 == 0)  
.collect(Collectors.toList());
```

12. Stream Pipeline

Numbers

↓

Stream

↓

filter()

↓

Even Numbers

↓

Print

13. Internal Working of filter()

Conceptually,

```
for(Integer number : numbers){  
    if(number % 2 == 0){  
        output.add(number);  
    }  
}
```

The Stream API performs the same logic internally but in a declarative style.

14. SQL Solution

Suppose we have the following table:

Number
5
8
10
15
18
21
24
30

Retrieve all even numbers.

```
SELECT number
FROM Numbers
WHERE MOD(number, 2) = 0;
```

Some databases also allow:

```
SELECT number
FROM Numbers
WHERE number % 2 = 0;
```

The modulus syntax varies slightly by database:

- Oracle/MySQL: MOD(number, 2)
- PostgreSQL/SQL Server: % is commonly supported.

Output

Number
8
10
18
24
30

15. Python Solution

Method 1 (Recommended)

```
numbers = [5,8,10,15,18,21,24,30]
```

```
for number in numbers:
```

```
if number % 2 == 0:
```

```
    print(number)
```

Output

```
8
10
18
24
30
```

Method 2

Using list comprehension.

```
numbers = [5,8,10,15,18,21,24,30]
```

```
even_numbers = [
    number
    for number in numbers
    if number % 2 == 0
]
```

```
print(even_numbers)
```

Output

```
[8, 10, 18, 24, 30]
```

Method 3

Using filter().

```
numbers = [5,8,10,15,18,21,24,30]
```

```
even_numbers = list(
    filter(
        lambda number: number % 2 == 0,
        numbers
    )
)
```

```
print(even_numbers)
```

16. Algorithm Comparison

Method	Time Complexity	Space Complexity
Java Loop	$O(n)$	$O(1)$ (printing) / $O(k)$ (collecting)
Java Stream filter()	$O(n)$	$O(k)$ when collecting
SQL WHERE MOD(...)	Depends on execution plan	Database-managed
Python Loop	$O(n)$	$O(1)$ (printing) / $O(k)$ (collecting)
Python List Comprehension	$O(n)$	$O(k)$

17. Modulus vs Bitwise Operator

Another common interview topic is using the **bitwise AND (&)** operator.

Modulus

`number % 2 == 0`

Easy to read and recommended.

Bitwise

`(number & 1) == 0`

Example

10

Binary

1010

1010

&

0001

=

0000

Result

Even

For an odd number:

11

Binary

1011

1011

&

0001

=

0001

Result

Odd

Which Should You Use?

- `% 2 == 0` is clearer and preferred in most business applications.
- `(number & 1) == 0` is sometimes used in low-level programming or performance-sensitive code, though modern compilers and JVMs optimize both well.

18. Edge Cases

Empty List

Input

[]

Output

[]

All Even Numbers

Input

2

4

6

8

Output

2

4

6

8

All Odd Numbers

Input

1

3

5

7

Output

No Even Numbers

(or an empty result, depending on the implementation)

Negative Numbers

Input

-6

-3

-2

5

Output

-6

-2

The same $\% 2 == 0$ rule works for negative even integers.

Zero

Input

0

Output

Even

Because

$0 \% 2$

=

0

19. Common Interview Questions

1. What is an even number?
 2. How does the modulus operator work?
 3. Why does $\% 2 == 0$ identify even numbers?
 4. What is the difference between $\%$ and $\&$ for checking parity?
 5. How does `filter()` work internally in Java Streams?
 6. What is the time complexity of finding even numbers?
 7. How would you count even numbers instead of printing them?
 8. How would you sum all even numbers?
 9. How would you partition numbers into even and odd groups?
 10. How would you find only the first even number?
-

20. Related Interview Questions

After solving this problem, interviewers often ask:

1. Find odd numbers.
 2. Count even numbers.
 3. Sum even numbers.
 4. Find the first even number.
 5. Find the last even number.
 6. Partition numbers into even and odd.
 7. Separate positive and negative numbers.
 8. Find prime numbers.
 9. Find multiples of a given number.
 10. Find numbers divisible by both 2 and 3.
-

21. Summary

- An **even number** is any integer divisible by **2** with a remainder of **0**.
- The core algorithm processes each number once and checks the condition `number % 2 == 0`, resulting in **$O(n)$** time complexity.
- Java 1.5 uses a loop and an if statement to filter even numbers.
- Java 8 Stream API provides the `filter()` intermediate operation for concise and expressive filtering.
- SQL retrieves even numbers using `WHERE MOD(number, 2) = 0` (or `%` where supported).
- Python offers multiple approaches, including loops, list comprehensions, and the built-in `filter()` function.
- Understanding this problem provides the foundation for many related interview questions, including **odd numbers**, **counting even numbers**, **summing even numbers**, **partitioning data**, **finding the first or last matching element**, and **predicate-based filtering**.

Finding Odd Numbers Using Java 1.5, Java 8 Stream API, SQL, and Python

Finding **odd numbers** is one of the most fundamental programming and interview questions. It is commonly asked because it evaluates your understanding of:

- Arithmetic Operators
- Modulus (%) Operator
- Conditional Statements
- Loops
- Collections
- Java 1.5
- Java 8 Stream API
- SQL Filtering
- Python
- Time Complexity
- Space Complexity

Although simple, the logic for identifying odd numbers is widely used in algorithms, mathematics, data filtering, and real-world applications.

1. What is an Odd Number?

Definition

An **odd number** is an integer that is **not exactly divisible by 2**. When divided by 2, it leaves a remainder of **1** (or **-1** for some negative numbers, depending on the language).

Mathematically,

Odd Number

=

$\text{Number \% 2} \neq 0$

Examples

Number	Odd?
1	✓
3	✓
5	✓
7	✓
11	✓
-5	✓
2	✗
8	✗
10	✗
0	✗

2. Problem Statement

Given a list of numbers

5 8 10 15 18 21 24 30

Find all odd numbers.

Expected Output

5

15

21

3. Mathematical Definition

For any integer **n**

n is Odd

if

$n \bmod 2 \neq 0$

Example

$21 \bmod 2$

=

1

Therefore

21

is Odd

4. Understanding the Modulus Operator

The modulus (%) operator returns the remainder after division.

Examples

$8 \bmod 2$

=

0

Even

$15 \bmod 2$

=

1

Odd

$18 \bmod 2$

=

0

Even

21 % 2

=

1

Odd

Decision Rule

If

Number % 2 != 0

↓

Odd

Else

Even

Note: In Java and Python, negative odd numbers produce a non-zero remainder (-5 % 2 == -1 in Java, -5 % 2 == 1 in Python), so the condition number % 2 != 0 works correctly in both languages.

5. Algorithm

Step 1

Read Number

Step 2

Calculate

Number % 2

Step 3

Result != 0 ?

Yes

Print Number

No

Ignore

Step 4

Repeat

Step 5

End

6. Flow Diagram

Start

↓

Read Number

↓

Number % 2 != 0 ?

↓

Yes

↓

Print Number

↓

No

↓

Next Number

↓

Repeat

↓

End

7. Dry Run

Input

5

8

10

15

Processing

$5 \% 2$

=

1

Odd

$8 \% 2$

=

0

Even

$10 \% 2$

=

0

Even

$15 \% 2$

=

1

Odd

Output

5

15

8. Time Complexity

For **n** numbers

$O(n)$

Each number is examined exactly once.

9. Space Complexity

Printing directly

$O(1)$

If storing the odd numbers in another collection

$O(k)$

where **k** is the number of odd numbers.

10. Java 1.5 Solution

```
import java.util.*;
```

```
public class OddNumbers {
```

```
    public static void main(String[] args) {
```

```
        List<Integer> numbers =  
            Arrays.asList(5,8,10,15,18,21,24,30);
```

```
        for(Integer number : numbers){
```

```
            if(number % 2 != 0){
```

```
                System.out.println(number);
```

```
    }  
  }  
}  
}
```

Output

5
15
21

Internal Working

Input

5
8
10
15

Processing

5
↓

Odd

↓

Print
8

↓

Even

↓

Ignore

11. Java 8 Stream API

Java 8 introduced **filter()**, making the solution concise.

Method 1 (Recommended)

```
import java.util.Arrays;

Arrays.asList(5,8,10,15,18,21,24,30)

.stream()

.filter(number -> number % 2 != 0)

.forEach(System.out::println);
```

Output

```
5
15
21
```

Method 2

Collect odd numbers into a list.

```
List<Integer> oddNumbers =

numbers.stream()

.filter(number -> number % 2 != 0)

.collect(Collectors.toList());
```

12. Stream Pipeline

Numbers

↓

Stream

↓

filter()

↓

Odd Numbers

↓

Print / Collect

13. Internal Working of filter()

Conceptually,

```
for(Integer number : numbers){  
    if(number % 2 != 0){  
        output.add(number);  
    }  
}
```

The Stream API internally performs the same filtering logic.

14. SQL Solution

Suppose the table contains:

Number
5
8
10
15
18
21
24
30

Retrieve odd numbers.

Using MOD()

```
SELECT number  
FROM Numbers  
WHERE MOD(number, 2) <> 0;
```

Using % (where supported)

```
SELECT number
FROM Numbers
WHERE number % 2 <> 0;
```

The exact syntax depends on the database:

- Oracle/MySQL: MOD(number, 2)
- PostgreSQL/SQL Server: % is commonly supported.

Output

Number
5
15
21

15. Python Solution

Method 1 (Recommended)

```
numbers = [5,8,10,15,18,21,24,30]
```

```
for number in numbers:
```

```
    if number % 2 != 0:
```

```
        print(number)
```

Output

```
5
15
21
```

Method 2

Using list comprehension.

```
numbers = [5,8,10,15,18,21,24,30]
```

```
odd_numbers = [
    number
    for number in numbers
    if number % 2 != 0
```

```
]
print(odd_numbers)
```

Output

```
[5, 15, 21]
```

Method 3

Using filter().

```
numbers = [5,8,10,15,18,21,24,30]

odd_numbers = list(
    filter(
        lambda number: number % 2 != 0,
        numbers
    )
)

print(odd_numbers)
```

16. Algorithm Comparison

Method	Time Complexity	Space Complexity
Java Loop	O(n)	O(1) (printing) / O(k) (collecting)
Java Stream filter()	O(n)	O(k) when collecting
SQL WHERE MOD(...)	Depends on execution plan	Database-managed
Python Loop	O(n)	O(1) (printing) / O(k) (collecting)
Python List Comprehension	O(n)	O(k)

17. Modulus vs Bitwise Operator

A common interview variation is checking odd numbers using the bitwise AND operator.

Modulus

```
number % 2 != 0
```

Simple and readable.

Bitwise

$(\text{number} \& 1) == 1$

Example

15

Binary

1111

1111

&

0001

=

0001

Result

Odd

For an even number:

18

Binary

10010

10010

&

00001

=

00000

Result

Even

Which Should You Use?

- $\text{number} \% 2 \neq 0$ is clearer and preferred for business applications.

- $(\text{number} \& 1) == 1$ (or more generally $(\text{number} \& 1) != 0$) is common in low-level programming and bit manipulation.

18. Edge Cases

Empty List

Input

[]

Output

[]

All Odd Numbers

Input

1

3

5

7

Output

1

3

5

7

All Even Numbers

Input

2

4

6

8

Output

No Odd Numbers

(or an empty result)

Negative Numbers

Input

-5

-4

-3

2

Output

-5

-3

The condition `number % 2 != 0` correctly identifies negative odd integers.

Zero

Input

0

Output

Not Odd

Because

`0 % 2`

=

0

19. Common Interview Questions

1. What is an odd number?
 2. How does the modulus operator work?
 3. Why does `% 2 != 0` identify odd numbers?
 4. What is the difference between `%` and `&` for checking odd numbers?
 5. How does `filter()` work internally in Java Streams?
 6. What is the time complexity of finding odd numbers?
 7. How would you count odd numbers instead of printing them?
 8. How would you sum all odd numbers?
 9. How would you partition numbers into odd and even groups?
 10. How would you find only the first odd number?
-

20. Related Interview Questions

After solving this problem, interviewers often ask:

1. Find even numbers.
 2. Count odd numbers.
 3. Sum odd numbers.
 4. Find the first odd number.
 5. Find the last odd number.
 6. Partition numbers into odd and even groups.
 7. Find prime numbers.
 8. Find numbers divisible by 3 or 5.
 9. Find multiples of a given number.
 10. Filter numbers based on custom predicates.
-

21. Summary

- An **odd number** is an integer that is **not divisible by 2** and therefore has a non-zero remainder when divided by 2.
- The standard algorithm checks each number using the condition `number % 2 != 0`, resulting in **O(n)** time complexity.
- Java 1.5 uses a loop with an if statement to identify odd numbers.
- Java 8 Stream API provides the `filter()` operation for expressive filtering based on the odd-number predicate.
- SQL retrieves odd numbers using `WHERE MOD(number, 2) <> 0` (or `% where supported`).

- Python supports multiple approaches, including loops, list comprehensions, and the built-in `filter()` function.
- Understanding odd-number filtering forms the basis for many interview problems involving **predicate-based filtering, counting, aggregation, partitioning, bit manipulation, and number classification.**

Finding Unique Elements Using Java 1.5, Java 8 Stream API, SQL, and Python

Finding **unique elements** is one of the most frequently asked programming interview questions. It is commonly used to test your understanding of:

- Arrays
- Collections
- HashMap
- HashSet
- Java 1.5
- Java 8 Stream API
- SQL Aggregation
- Python Collections
- Time Complexity
- Space Complexity

This question is often confused with **removing duplicates**, but they are different problems.

1. What are Unique Elements?

Definition

A **unique element** is an element that appears **exactly one time** in a collection.

Mathematically,

$$\text{Frequency}(\text{element}) = 1$$

Only those elements whose frequency is **1** are considered unique.

2. Difference Between Unique Elements and Distinct Elements

This is one of the most common interview questions.

Suppose the array is

[1,2,2,3,4,4,5]

Distinct Elements

Distinct means removing duplicate occurrences while keeping one copy of every value.

Output

[1,2,3,4,5]

Notice that **2** and **4** still appear because only duplicate copies are removed.

Unique Elements

Unique means elements that occur **only once**.

Output

[1,3,5]

Since:

- 2 occurs twice ✗
- 4 occurs twice ✗

they are **not unique**.

3. Problem Statement

Given the array

10 20 30 20 40 50 10 60

Find all unique elements.

Expected Output

30

40

50

60

4. Real-World Applications

Finding unique elements is used in:

- Log Analysis
- Fraud Detection
- Inventory Management
- Banking Transactions
- Customer Analytics
- Data Cleaning
- Search Engines
- Social Media Analytics

Examples:

- Customers who made only one purchase.
- Products sold only once.
- Employees assigned to only one project.
- IP addresses appearing only once in server logs.

5. Logic

The basic idea is:

1. Count the frequency of every element.
2. Print only those elements whose frequency is exactly one.

6. Algorithm

Step 1

Create HashMap<Element, Count>

Step 2

Read each element

Increase its count

Step 3

Traverse the map

Step 4

If count == 1

Print the element

Step 5

End

7. Flow Diagram

Start

↓

Read Element

↓

Update Frequency

↓

More Elements?

↓

Yes

Repeat

↓

Traverse Map

↓

Count == 1 ?

↓

Yes

Print

↓

End

8. Dry Run

Input

10

20

30

20

40

50

10

60

Frequency Map

Element	Count
10	2
20	2
30	1
40	1
50	1
60	1

Output

30

40

50

60

9. Time Complexity

Using a HashMap

$O(n)$

Each element is processed once to build the frequency map, and then the map is traversed once.

10. Space Complexity

$O(k)$

where:

- **n** = total number of elements
 - **k** = number of distinct elements
-

11. Java 1.5 Solution

```
import java.util.*;

public class UniqueElements {

    public static void main(String[] args) {

        List<Integer> numbers = Arrays.asList(
            10,20,30,20,40,50,10,60);

        Map<Integer,Integer> frequency =
            new HashMap<Integer,Integer>();

        for(Integer number : numbers){

            if(frequency.containsKey(number)){

                frequency.put(number,
                    frequency.get(number)+1);

            }else{

                frequency.put(number,1);

            }

        }

        for(Map.Entry<Integer,Integer> entry :
            frequency.entrySet()){
```

```
        if(entry.getValue()==1){  
            System.out.println(  
                entry.getKey());  
        }  
    }  
}  
}
```

Output

```
30  
40  
50  
60
```

12. Internal Working

Input

```
10  
20  
30  
20  
40  
50  
10  
60
```

HashMap

```
10 → 2  
20 → 2  
30 → 1
```

40 → 1

50 → 1

60 → 1

Print

30

40

50

60

13. Java 8 Stream API

The recommended approach is to first group elements by value and count them, then filter those with a count of 1.

```
import java.util.*;
import java.util.function.Function;
import java.util.stream.Collectors;

List<Integer> numbers =
    Arrays.asList(10,20,30,20,40,50,10,60);

List<Integer> unique =

numbers.stream()

.collect(

Collectors.groupingBy(

Function.identity(),

Collectors.counting()

))

.entrySet()

.stream()

.filter(entry -> entry.getValue() == 1)
```



```
.map(Map.Entry::getKey)
.collect(Collectors.toList());
System.out.println(unique);
```

Output

```
[30, 40, 50, 60]
```

Stream Pipeline

List

↓

Stream

↓

groupingBy()

↓

counting()

↓

Filter Count == 1

↓

Unique Elements

14. Internal Working of groupingBy()

Conceptually, the Stream API performs logic similar to:

```
Map<Integer, Long> map = new HashMap<>();
for (Integer number : numbers) {
    map.put(number,
        map.getOrDefault(number, 0L) + 1);
}
```

Then it filters entries where the count equals 1.

15. SQL Solution

Suppose the table is:

Number
10
20
30
20
40
50
10
60

Retrieve elements that appear exactly once.

```
SELECT number
FROM Numbers
GROUP BY number
HAVING COUNT(*) = 1;
```

Output

Number
30
40
50
60

The logic is:

1. Group identical values.
2. Count occurrences.
3. Keep only groups with a count of 1.

16. Python Solution

Method 1 (Dictionary)

```
numbers = [10,20,30,20,40,50,10,60]
```

```
frequency = {}
```

```
for number in numbers:
```

```
frequency[number] = frequency.get(number,0) + 1

unique = [
    number
    for number, count in frequency.items()
    if count == 1
]

print(unique)
```

Output

[30, 40, 50, 60]

Method 2 (Counter)

```
from collections import Counter

numbers = [10,20,30,20,40,50,10,60]

frequency = Counter(numbers)

unique = [
    number
    for number, count in frequency.items()
    if count == 1
]

print(unique)
```

Output

[30, 40, 50, 60]

Counter is the most concise and commonly recommended Python solution.

17. Algorithm Comparison

Method	Time Complexity	Space Complexity
Java HashMap	$O(n)$	$O(k)$
Java Stream groupingBy()	$O(n)$	$O(k)$
SQL GROUP BY ... HAVING	Depends on execution plan (hash or sort aggregation)	Database-managed

Python Dictionary	$O(n)$	$O(k)$
Python Counter	$O(n)$	$O(k)$

18. Brute Force Algorithm

A beginner approach compares every element with every other element.

For each element

↓

Compare with every element

↓

Count occurrences

↓

Count == 1 ?

↓

Print

Complexity

Time Space

$O(n^2)$ $O(1)$

This approach is generally not recommended for interviews when a hash-based solution is expected.

19. HashMap vs HashSet

Feature	HashMap	HashSet
Stores Count	✓ Yes	✗ No
Suitable for Frequency	✓ Yes	✗ No
Suitable for Unique Elements	✓ Yes	✗ Not by itself
Time Complexity	$O(n)$	$O(n)$

A HashSet removes duplicate values but **does not track how many times an element appears**, so it cannot identify elements that occur exactly once without additional data structures.

20. Edge Cases

Empty List

Input

[]

Output

[]

Single Element

Input

[10]

Output

[10]

All Elements Unique

Input

1

2

3

4

Output

1

2

3

4

No Unique Elements

Input

1

1

2

2

3

3

Output

[]

Negative Numbers

Input

-2

-1

-2

5

6

Output

-1

5

6

The same frequency-counting algorithm works regardless of whether the values are positive, negative, or zero.

21. Common Interview Questions

1. What is a unique element?
2. What is the difference between unique and distinct elements?
3. Why is a HashMap used instead of a HashSet?
4. How does `Collectors.groupingBy()` work internally?
5. Why is the time complexity $O(n)$?
6. How would you preserve the original insertion order of unique elements?
7. How would you find duplicate elements instead?
8. How would you count the frequency of each element?
9. How would you find the first unique element?
10. How would you find the unique characters in a string?

22. Related Interview Questions

After solving this problem, interviewers often ask:

1. Remove duplicate elements.
2. Find duplicate elements.
3. Count duplicate elements.
4. Count the frequency of elements.
5. Find the first unique element.
6. Find the second unique element.
7. Find unique characters in a string.
8. Find duplicate characters in a string.
9. Group elements by frequency.
10. Sort elements by frequency.

23. Summary

- A **unique element** is an element whose **frequency is exactly one**.
- Unique elements are different from **distinct elements**; distinct values include one copy of every value, whereas unique values exclude anything that appears more than once.
- The most efficient algorithm uses a **HashMap** (or dictionary) to count frequencies and then selects elements with a count of **1**, achieving **$O(n)$** time complexity.
- Java 1.5 implements this with a HashMap and two passes over the data.
- Java 8 Stream API combines `Collectors.groupingBy()`, `Collectors.counting()`, `filter()`, and `map()` to produce a concise solution.
- SQL solves the problem using GROUP BY together with `HAVING COUNT(*) = 1`.
- Python provides straightforward solutions using either a dictionary or `collections.Counter`.
- Mastering this problem prepares you for many related interview questions involving **frequency counting**, **duplicate detection**, **first unique element**, **character frequency**, **data deduplication**, and **grouping by occurrence count**.

Joining Two Tables Using Java 1.5, Java 8 Stream API, SQL, and Python

Joining two tables is one of the **most important interview topics** in SQL, Java, and Python. It is asked in almost every database and backend interview because it tests your understanding of:

- Relational Databases
 - Primary Key
 - Foreign Key
 - HashMap
 - Java Collections
 - Java 8 Stream API
 - SQL JOINS
 - Python Dictionaries
 - Time Complexity
 - Space Complexity
-

1. What is a Join?

Definition

A **join** is the process of combining data from **two or more tables (or collections)** based on a common column (called the **join key**).

The purpose of a join is to retrieve related information that is stored separately.

Example

Employee Table

Employee ID	Name	Department ID
101	John	1
102	David	2
103	Alice	1
104	Robert	3

Department Table

Department ID	Department Name
1	IT

2	HR
3	Finance

Joined Result

Employee ID	Name	Department
101	John	IT
102	David	HR
103	Alice	IT
104	Robert	Finance

2. Why Do We Need Joins?

In a relational database, data is normalized to reduce duplication.

Instead of storing

John
IT

David
HR

Alice
IT

the database stores

Employee

John

Department ID = 1

Department

1

IT

The join combines both tables when required.

3. Types of SQL Joins

Join Type	Description
INNER JOIN	Matching rows only
LEFT JOIN	All rows from left table + matching rows
RIGHT JOIN	All rows from right table + matching rows
FULL OUTER JOIN	All rows from both tables
CROSS JOIN	Cartesian Product
SELF JOIN	Join a table with itself

4. Problem Statement

Employee

ID	Name	Dept ID
101	John	1
102	David	2
103	Alice	1
104	Robert	3

Department

Dept ID	Name
1	IT
2	HR
3	Finance

Expected Result

Employee	Department
John	IT
David	HR
Alice	IT
Robert	Finance

5. Join Logic

The basic idea is:

For every employee,

- Find the department whose department ID matches the employee's department ID.
 - Combine both records.
-

6. Brute Force Algorithm

Step 1

Read Employee

Step 2

Read every Department

Step 3

Department IDs Equal?

Yes

Print Employee + Department

No

Continue

Step 4

Repeat

Flow Diagram

Employee

↓

Department Loop

↓

IDs Equal?

↓

Yes

↓

Combine

↓

Next Employee

Time Complexity

$O(n \times m)$

where:

- n = employees
- m = departments

This becomes slow for large datasets.

7. Optimized Algorithm (HashMap)

The optimized solution is the one most interviewers expect.

Step 1

Store Departments in HashMap

Dept ID

↓

Department

Step 2

Read Employee

Step 3

HashMap Lookup

Step 4

Print Result

Time Complexity

Building HashMap

$O(m)$

Lookup

$O(n)$

Overall

$O(n + m)$

which is much better than $O(n \times m)$.

8. Java 1.5 Solution

Employee Class

```
public class Employee {  
  
    private int id;  
    private String name;  
    private int departmentId;  
  
    public Employee(int id, String name, int departmentId) {  
        this.id = id;  
        this.name = name;  
        this.departmentId = departmentId;  
    }  
  
    public int getDepartmentId() {  
        return departmentId;  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```

Department Class

```
public class Department {  
  
    private int id;  
    private String name;  
  
    public Department(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
}
```

```
public int getId() {  
    return id;  
}  
  
public String getName() {  
    return name;  
}  
}
```

Method 1 (Brute Force)

```
for(Employee employee : employees){  
  
    for(Department department : departments){  
  
        if(employee.getDepartmentId()  
            == department.getId()){  
  
            System.out.println(  
                employee.getName()  
                + " -> "  
                + department.getName());  
  
        }  
  
    }  
  
}
```

Complexity

$O(n \times m)$

Method 2 (Recommended)

Using HashMap

```
Map<Integer, Department> departmentMap =  
    new HashMap<Integer, Department>();  
  
for(Department department : departments){  
  
    departmentMap.put(  
        department.getId(),  
        department);  
  
}
```

```
}  
  
for(Employee employee : employees){  
  
    Department department =  
        departmentMap.get(  
            employee.getDepartmentId());  
  
    if(department != null){  
  
        System.out.println(  
            employee.getName()  
            + " -> "  
            + department.getName());  
  
    }  
}
```

Output

John -> IT

David -> HR

Alice -> IT

Robert -> Finance

9. Internal Working

Department Table

↓

HashMap

1 -> IT

2 -> HR

3 -> Finance

↓

Employee

↓

Department Lookup

↓

Output

10. Java 8 Stream API

Java Streams do **not** have a built-in `join()` operation for combining two collections like SQL.

The common approach is to build a lookup map and then stream over the employees.

```
Map<Integer, Department> departmentMap =
```

```
departments.stream()
```

```
.collect(
```

```
Collectors.toMap(
```

```
Department::getId,
```

```
Function.identity()
```

```
)
```

```
);
```

```
employees.stream()
```

```
.map(employee ->
```

```
employee.getName()
```

```
+ " -> "
```

```
+ departmentMap.get(  
    employee.getDepartmentId()  
    .getName())
```

```
.forEach(System.out::println);
```

Output

John -> IT

David -> HR

Alice -> IT

Robert -> Finance

11. Stream Pipeline

Department List

↓

toMap()

↓

HashMap

↓

Employee Stream

↓

Lookup

↓

Joined Result

12. SQL Solution

Suppose

Employee

ID	Name	Department_ID
101	John	1
102	David	2
103	Alice	1
104	Robert	3

Department

ID	Name
1	IT
2	HR

INNER JOIN

```
SELECT
  e.id,
  e.name,
  d.name AS department
FROM Employee e
INNER JOIN Department d
ON e.department_id = d.id;
```

Output

Employee	Department
John	IT
David	HR
Alice	IT
Robert	Finance

LEFT JOIN

```
SELECT
  e.id,
  e.name,
  d.name
FROM Employee e
LEFT JOIN Department d
ON e.department_id = d.id;
```

Returns every employee, even if a matching department does not exist.

RIGHT JOIN

```
SELECT
  e.id,
  e.name,
  d.name
FROM Employee e
RIGHT JOIN Department d
ON e.department_id = d.id;
```

Returns every department, even if no employee belongs to it.

FULL OUTER JOIN

```
SELECT
  e.id,
  e.name,
  d.name
FROM Employee e
FULL OUTER JOIN Department d
ON e.department_id = d.id;
```

Returns every row from both tables, matching where possible.

Note: Some databases (such as MySQL) do not support FULL OUTER JOIN directly.

CROSS JOIN

```
SELECT
  e.name,
  d.name
FROM Employee e
CROSS JOIN Department d;
```

Produces the Cartesian product.

If there are:

- 4 employees
- 3 departments

Result

4×3

=

12 rows

13. Python Solution

Method 1 (Recommended)

Using a dictionary.

```

employees = [
    (101,"John",1),
    (102,"David",2),
    (103,"Alice",1),
    (104,"Robert",3)
]

departments = {
    1:"IT",
    2:"HR",
    3:"Finance"
}

for employee_id, name, department_id in employees:

    print(
        name,
        "->",
        departments.get(department_id)
    )

```

Output

John -> IT

David -> HR

Alice -> IT

Robert -> Finance

Method 2

Brute Force

```

departments = [
    (1,"IT"),
    (2,"HR"),
    (3,"Finance")
]

for employee in employees:

    for department in departments:

        if employee[2] == department[0]:

            print(

```

```

    employee[1],
    "->",
    department[1]
)

```

14. Algorithm Comparison

Method	Time Complexity	Space Complexity
Java Nested Loop	$O(n \times m)$	$O(1)$
Java HashMap	$O(n + m)$	$O(m)$
Java Stream + Map	$O(n + m)$	$O(m)$
SQL INNER JOIN	Depends on indexes and execution plan	Database-managed
Python Nested Loop	$O(n \times m)$	$O(1)$
Python Dictionary	$O(n + m)$	$O(m)$

15. SQL Join vs Java Join

Feature	SQL	Java/Python
Built-in Join	Yes	No
Uses Indexes	Yes	No (unless implemented)
Lookup Structure	Database engine	HashMap / Dictionary
Best Complexity	Depends on optimizer and indexes	$O(n + m)$ with a map

16. Edge Cases

Employee Without Department

Employee

Name	Dept ID
John	5

Department

ID	Name
1	IT

Result

- **INNER JOIN:** No row returned.
- **LEFT JOIN:** John appears with a NULL department.
- **Java/Python:** `HashMap.get()` / `dict.get()` returns null / None, so check before accessing the department name.

Empty Employee List

Output

No Employees

Empty Department Table

- **INNER JOIN:** Empty result.
 - **LEFT JOIN:** Employees appear with NULL departments.
 - **Java/Python:** Lookups fail unless handled safely.
-

Duplicate Department IDs

A properly normalized database should have unique department IDs. If duplicates exist:

- SQL joins may return multiple matching rows.
 - HashMap and Python dictionaries overwrite earlier entries with later ones.
-

17. Common Interview Questions

1. What is a join?
 2. What is the difference between INNER JOIN and LEFT JOIN?
 3. What is a foreign key?
 4. Why is a HashMap used in Java for joins?
 5. Why is the HashMap solution faster than nested loops?
 6. How does SQL execute joins internally?
 7. What is the difference between a hash join and a nested loop join?
 8. How would you join three tables?
 9. How would you perform a left join in Java?
 10. How would you handle missing departments during a join?
-

18. SQL Join Algorithms (Internal Database Concepts)

Modern relational databases choose the join algorithm automatically based on table sizes, indexes, and statistics.

Algorithm	Best Used When	Typical Complexity
Nested Loop	Small tables or indexed lookups	$O(n \times m)$ (or better with indexes)

Join		
Hash Join	Large tables, equality joins	Approximately $O(n + m)$
Merge Join	Both inputs sorted on join key	Approximately $O(n + m)$

The database optimizer selects the most efficient algorithm.

19. Summary

- A **join** combines related records from two or more tables or collections using a common key.
- The straightforward nested-loop algorithm compares every employee with every department, resulting in **$O(n \times m)$** time complexity.
- The optimized approach builds a **HashMap** (Java) or **dictionary** (Python) for department lookups, reducing the complexity to **$O(n + m)$** .
- Java 1.5 typically uses nested loops or a HashMap, while Java 8 Streams commonly create a lookup map with `Collectors.toMap()` before streaming over employees.
- SQL provides built-in join operations such as **INNER JOIN**, **LEFT JOIN**, **RIGHT JOIN**, **FULL OUTER JOIN**, **CROSS JOIN**, and **SELF JOIN**, with the database optimizer choosing the most appropriate execution algorithm.
- Python commonly uses dictionaries for efficient joins or nested loops for simpler demonstrations.
- Understanding joins is essential because they underpin many interview questions involving **employee-department relationships, orders and customers, students and courses, multi-table reporting, data integration, and relational database design**.

Ranking Employees Using Java 1.5, Java 8 Stream API, SQL, and Python

Ranking employees is one of the **most common interview questions** in Java, SQL, and Python. It evaluates your understanding of:

- Sorting Algorithms
- Ranking Algorithms
- Collections
- Comparators
- Java 1.5
- Java 8 Stream API
- SQL Window Functions
- Python Sorting
- Time Complexity
- Space Complexity

Ranking is widely used in:

- Employee Performance Systems

- Payroll
 - Leaderboards
 - Sales Dashboards
 - Student Results
 - Banking
 - Business Analytics
-

1. What is Ranking?

Definition

Ranking is the process of assigning a position (rank) to each record after sorting it according to one or more criteria.

For employees, the ranking criterion is often:

- Salary
- Performance Score
- Experience
- Sales
- Age

Example:

Employee	Salary	Rank
Robert	90000	1
Alice	80000	2
David	70000	3
John	60000	4

The highest salary receives **Rank 1**.

2. Problem Statement

Suppose we have the following employees.

ID	Name	Department	Salary
101	John	IT	60000
102	David	HR	70000
103	Alice	IT	80000
104	Robert	Finance	90000
105	James	HR	70000

Assign ranks based on **salary in descending order**.

Expected Output (Using RANK())

Employee	Salary	Rank
Robert	90000	1
Alice	80000	2
David	70000	3
James	70000	3
John	60000	5

Notice that David and James have the same salary, so they receive the same rank, and the next rank skips from 3 to 5.

3. Types of Ranking

1. ROW_NUMBER

Every row gets a unique number.

Employee	Salary	Row Number
Robert	90000	1
Alice	80000	2
David	70000	3
James	70000	4
John	60000	5

2. RANK

Equal values receive the same rank, and the next rank is skipped.

Employee	Salary	Rank
Robert	90000	1
Alice	80000	2
David	70000	3
James	70000	3
John	60000	5

3. DENSE_RANK

Equal values receive the same rank, but there are **no gaps**.

Employee	Salary	Dense Rank
----------	--------	------------

Robert	90000	1
Alice	80000	2
David	70000	3
James	70000	3
John	60000	4

4. Real-World Applications

Ranking is used in:

- Employee appraisal
 - Student ranking
 - Sales ranking
 - Product popularity
 - Sports leaderboards
 - Banking reports
 - Customer segmentation
-

5. Algorithm

The basic algorithm consists of two phases:

1. Sort employees.
 2. Assign ranks.
-

Algorithm (ROW_NUMBER)

Step 1

Sort employees by salary descending.

Step 2

rank = 1

Step 3

For each employee

Assign current rank.

rank++

Step 4

End

Algorithm (RANK)

Step 1

Sort by salary descending.

Step 2

If current salary equals previous salary

Use previous rank.

Else

Current rank = current position.

Step 3

Repeat.

6. Flow Diagram

Employees

↓

Sort by Salary

↓

Highest Salary

↓

Assign Rank

↓

Next Employee

↓

Repeat

7. Dry Run

Input

Name	Salary
John	60000
David	70000
Alice	80000
Robert	90000
James	70000

Sorted

Name	Salary
Robert	90000
Alice	80000
David	70000
James	70000
John	60000

Ranks (RANK())

Robert → 1

Alice → 2

David → 3

James → 3

John → 5

8. Time Complexity

Sorting dominates the algorithm.

$O(n \log n)$

9. Space Complexity

If sorting creates another collection:

$O(n)$

If sorting is performed in place (where applicable):

$O(1)$ to $O(\log n)$

depending on the sorting implementation.

10. Java 1.5 Solution

Step 1 – Sort Employees

```
Collections.sort(employees,
    new Comparator<Employee>() {

        public int compare(Employee e1,
            Employee e2){

            return Double.compare(
                e2.getSalary(),
                e1.getSalary());

        }

    });
```

Step 2 – Assign ROW_NUMBER

```
int rank = 1;

for(Employee employee : employees){

    System.out.println(

        rank++

        + " "

        + employee.getName()

        + " "

        + employee.getSalary());

}
```

Output

1 Robert 90000

2 Alice 80000

3 David 70000

4 James 70000

5 John 60000

Step 3 - Assign RANK()

```
int rank = 1;
int position = 1;
double previousSalary = -1;

for(Employee employee : employees){

    if(employee.getSalary() != previousSalary){

        rank = position;
    }

    System.out.println(
        employee.getName()
        + " Rank: "
        + rank);

    previousSalary =
        employee.getSalary();

    position++;
}
```

Output

Robert Rank: 1

Alice Rank: 2

David Rank: 3

James Rank: 3

John Rank: 5

11. Java 8 Stream API

Streams can easily sort employees.

```
employees.stream()  
.sorted(  
    Comparator.comparing(  
        Employee::getSalary  
    )  
    .reversed()  
)  
.forEach(System.out::println);
```

However, **Streams do not provide a built-in ranking operation.**

To assign ranks, first collect the sorted list.

```
List<Employee> sorted =  
employees.stream()  
.sorted(  
    Comparator.comparing(  
        Employee::getSalary  
    )  
    .reversed()  
)  
.collect(Collectors.toList());
```

Then iterate through the sorted list to assign ROW_NUMBER, RANK, or DENSE_RANK using logic similar to the Java 1.5 example.

12. Stream Pipeline

Employee List

↓

Stream

↓

sorted()

↓

Collect

↓

Assign Rank

13. SQL Solution

SQL provides built-in window functions specifically designed for ranking.

ROW_NUMBER

```
SELECT
  id,
  name,
  salary,
  ROW_NUMBER() OVER (
    ORDER BY salary DESC
  ) AS row_number
FROM Employee;
```

RANK

```
SELECT
  id,
  name,
  salary,
  RANK() OVER (
    ORDER BY salary DESC
  ) AS rank
FROM Employee;
```

Output

Name	Salary	Rank
Robert	90000	1
Alice	80000	2
David	70000	3
James	70000	3
John	60000	5

DENSE_RANK

```
SELECT
    id,
    name,
    salary,
    DENSE_RANK() OVER (
        ORDER BY salary DESC
    ) AS dense_rank
FROM Employee;
```

Output

Name	Salary	Dense Rank
Robert	90000	1
Alice	80000	2
David	70000	3
James	70000	3
John	60000	4

Department-wise Ranking

```
SELECT
    name,
    department,
    salary,
    RANK() OVER (
        PARTITION BY department
        ORDER BY salary DESC
    ) AS department_rank
FROM Employee;
```

Each department has its own independent ranking.

14. Python Solution

ROW_NUMBER

```
employees = [
    ("John", 60000),
    ("David", 70000),
    ("Alice", 80000),
    ("Robert", 90000),
```

```
    ("James",70000)
]

sorted_employees = sorted(
    employees,
    key=lambda employee: employee[1],
    reverse=True
)

for rank, employee in enumerate(
    sorted_employees,
    start=1):

    print(rank, employee)
```

RANK

```
employees = sorted(
    employees,
    key=lambda employee: employee[1],
    reverse=True
)

rank = 1
previous_salary = None

for position, employee in enumerate(
    employees,
    start=1):

    if employee[1] != previous_salary:
        rank = position

    print(employee[0], rank)

    previous_salary = employee[1]
```

Output

Robert 1

Alice 2

David 3

James 3

John 5

15. Algorithm Comparison

Method	Time Complexity	Space Complexity
Java Collections Sort	$O(n \log n)$	$O(n)$ or implementation-dependent
Java Stream sorted()	$O(n \log n)$	$O(n)$
SQL Window Functions	Depends on execution plan (sorting required)	Database-managed
Python sorted()	$O(n \log n)$	$O(n)$

16. Edge Cases

Empty Employee List

Output

No Employees

Single Employee

Name	Salary	Rank
John	50000	1

All Salaries Equal

Input

Name	Salary
John	50000
David	50000
Alice	50000

ROW_NUMBER

1

2

3

RANK

1

1

1

DENSE_RANK

1

1

1

Negative Salaries

Although uncommon in payroll systems, the ranking algorithm still works because it compares numeric values.

17. Common Interview Questions

1. What is ranking?
 2. What is the difference between ROW_NUMBER(), RANK(), and DENSE_RANK()?
 3. Why is sorting required before ranking?
 4. Why is the time complexity $O(n \log n)$?
 5. How would you rank employees department-wise?
 6. How would you find the top 3 employees?
 7. How would you find employees with the second-highest salary using ranking?
 8. How would you break ties using another field such as employee ID?
 9. Why doesn't Java Stream provide a built-in ranking operation?
 10. How would you rank employees in ascending salary order?
-

18. Internal Working of SQL Ranking Functions

Assume the sorted salaries are:

90000

80000

70000

70000

60000

ROW_NUMBER

1

2

3

4

5

RANK

1

2

3

3

5

DENSE_RANK

1

2

3

3

4

The difference lies only in how ties are handled.

19. Summary

- **Ranking** assigns an ordered position to each employee after sorting based on one or more criteria.
- The algorithm consists of two main steps: **sort the employees** and **assign ranks**.
- Java 1.5 uses `Collections.sort()` together with manual ranking logic.

- Java 8 Stream API simplifies sorting with `sorted()`, but ranking still requires explicit iteration after sorting because the Stream API does not include built-in ranking operations.
- SQL provides powerful window functions—`ROW_NUMBER()`, `RANK()`, and `DENSE_RANK()`—which are the preferred approach for ranking in databases.
- Python uses the built-in `sorted()` function and manual rank assignment with `enumerate()` or custom logic.
- Ranking algorithms generally have a **time complexity of $O(n \log n)$** because sorting is the dominant operation.
- Mastering ranking prepares you for advanced interview questions such as **top N employees, department-wise ranking, highest and second-highest salary, leaderboards, percentile calculations, and window function problems.**

Running Totals (Cumulative Sum) Using Java 1.5, Java 8 Stream API, SQL, and Python

Running Totals (also called **Cumulative Sum**, **Rolling Sum**, or **Prefix Sum**) are among the most frequently asked interview questions in Java, SQL, and Python.

This problem evaluates your understanding of:

- Arrays
- Loops
- Accumulators
- Prefix Sum Algorithm
- Java Collections
- Java 8 Stream API
- SQL Window Functions
- Python
- Time Complexity
- Space Complexity

Running totals are extensively used in:

- Banking
- Payroll
- Financial Reporting
- Sales Analytics
- Inventory Management
- Data Warehousing
- Business Intelligence
- Time-Series Analysis

1. What is a Running Total?

Definition

A **Running Total** (or **Cumulative Sum**) is the sum of all previous values including the current value.

Mathematically,

For an array:

$$A = [a_1, a_2, a_3, \dots, a_n]$$

The running total at position **i** is:

$$\text{RunningTotal}(i) = a_1 + a_2 + \dots + a_i$$

2. Example

Input

Day	Sales
1	100
2	200
3	300
4	400
5	500

Running Total

Day	Sales	Running Total
1	100	100
2	200	300
3	300	600
4	400	1000
5	500	1500

3. Real-World Applications

Running totals are used in:

- Bank account balances
- Employee salary accumulation
- Monthly sales reports
- Stock market analysis
- Inventory tracking
- Financial dashboards
- Expense tracking
- Profit/Loss statements

4. Logic

Maintain one variable:

sum = 0

For every element:

sum = sum + currentValue

Print the updated sum.

5. Algorithm

Step 1

sum = 0

Step 2

Read Number

Step 3

sum = sum + number

Step 4

Print sum

Step 5

Repeat

6. Flow Diagram

Start

↓

sum = 0

↓

Read Number

↓

Add Number

↓

Print Running Total

↓

Next Number

↓

End

7. Dry Run

Input

100

200

300

400

Initially

sum = 0

Iteration 1

sum = 100

Iteration 2

100 + 200

=

300

Iteration 3

300 + 300

=

600

Iteration 4

600 + 400

=

1000

Output

100

300

600

1000

8. Time Complexity

$O(n)$

Each element is processed exactly once.

9. Space Complexity

If printing immediately:

$O(1)$

If storing running totals:

$O(n)$

10. Java 1.5 Solution

```
import java.util.Arrays;  
import java.util.List;
```

```
public class RunningTotal {  
    public static void main(String[] args) {  
        List<Integer> numbers =  
            Arrays.asList(100,200,300,400,500);  
  
        int sum = 0;  
  
        for(Integer number : numbers){  
            sum += number;  
  
            System.out.println(sum);  
        }  
    }  
}
```

Output

100

300

600

1000

1500

11. Internal Working

Input

100

200

300

Processing

sum = 0

↓

sum = 100

↓

sum = 300

↓

sum = 600

12. Java 8 Stream API

The Java Stream API **does not include a built-in running total operation** because streams are designed for stateless transformations. Running totals require maintaining state between elements.

Method 1 (Recommended)

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.stream.Collectors;
```

```
List<Integer> numbers =
    Arrays.asList(100,200,300,400,500);
```

```
AtomicInteger runningTotal =
    new AtomicInteger();
```

```
List<Integer> result =
```

```
numbers.stream()
```

```
.map(runningTotal::addAndGet)
```

```
.collect(Collectors.toList());
```

```
System.out.println(result);
```

Output

```
[100, 300, 600, 1000, 1500]
```

Why AtomicInteger?

A lambda expression can only capture local variables that are **final or effectively final**. AtomicInteger provides a mutable holder that allows the running total to be updated inside the stream pipeline.

Note: This approach is suitable for sequential streams. Do **not** use it with parallel streams because running totals depend on element order.

Method 2

Using reduce()

Although reduce() computes the **final total**, it does **not** produce intermediate running totals.

int total =

numbers.stream()

.reduce(0, Integer::sum);

Output

1500

13. Stream Pipeline

Numbers

↓

Stream

↓

Running Accumulator

↓

Running Totals

14. SQL Solution

Suppose we have:

Day	Sales
-----	-------

1	100
2	200
3	300
4	400
5	500

Modern SQL databases support running totals using window functions.

```
SELECT
    day,
    sales,
    SUM(sales) OVER (
        ORDER BY day
    ) AS running_total
FROM Sales;
```

Output

Day	Sales	Running Total
1	100	100
2	200	300
3	300	600
4	400	1000
5	500	1500

Department-wise Running Total

```
SELECT
    department,
    employee_name,
    salary,
    SUM(salary) OVER (
        PARTITION BY department
        ORDER BY employee_id
    ) AS running_total
FROM Employee;
```

Each department maintains its own independent running total.

15. Python Solution

Method 1 (Loop)

```
numbers = [100,200,300,400,500]
```

```
running_total = 0
```

for number in numbers:

```
    running_total += number
```

```
    print(running_total)
```

Output

100

300

600

1000

1500

Method 2 (Recommended)

Using itertools.accumulate.

```
from itertools import accumulate
```

```
numbers = [100,200,300,400,500]
```

```
print(list(accumulate(numbers)))
```

Output

[100, 300, 600, 1000, 1500]

accumulate() is specifically designed to generate cumulative results.

16. Algorithm Comparison

Method	Time Complexity	Space Complexity
Java Loop	$O(n)$	$O(1)$ (printing) / $O(n)$ (collecting)
Java Stream + AtomicInteger	$O(n)$	$O(n)$
Java Stream reduce() (final sum only)	$O(n)$	$O(1)$
SQL Window Function	Depends on execution plan (ordered	Database-managed

	processing required)	
Python Loop	$O(n)$	$O(1)$ (printing) / $O(n)$ (collecting)
Python accumulate()	$O(n)$	$O(n)$

17. Running Total vs Sum

Suppose:

100

200

300

Running Total

100

300

600

Sum

600

Difference

Running Total	Sum
Intermediate totals	Final total only
One value per row	One value overall
Used in reports	Used in aggregation

18. Edge Cases

Empty List

Input

[]

Output

[]

Single Value

Input

100

Output

100

Negative Numbers

Input

100

-50

200

Running Total

100

50

250

Zero Values

Input

100

0

200

Running Total

100

100

19. Common Interview Questions

1. What is a running total?
 2. What is the difference between a running total and a total sum?
 3. Why is the algorithm $O(n)$?
 4. Why doesn't Java Stream have a built-in running total operation?
 5. What is AtomicInteger used for in Stream examples?
 6. Why should stateful operations be avoided in parallel streams?
 7. How do SQL window functions compute running totals?
 8. How would you compute department-wise running totals?
 9. How would you compute a running average?
 10. How would you reset the running total when the department changes?
-

20. Prefix Sum Algorithm

Running totals are also known as the **Prefix Sum Algorithm**.

Example

Input

2

5

7

3

Prefix Sum

2

7

14

17

The prefix sum array allows many range-sum queries to be answered efficiently:

Sum(L...R)

=

Prefix[R]

-

Prefix[L-1]

This technique is widely used in competitive programming and algorithm design.

21. Related Interview Questions

Once you understand running totals, interviewers often ask:

1. Running average.
 2. Cumulative product.
 3. Moving average.
 4. Sliding window sum.
 5. Prefix sum.
 6. Range sum queries.
 7. Department-wise cumulative salary.
 8. Monthly cumulative sales.
 9. Year-to-date (YTD) totals.
 10. Rolling 7-day or 30-day totals.
-

22. Summary

- A **running total** (or **cumulative sum**) is the cumulative sum of all values from the beginning of the sequence up to the current element.
- The core algorithm maintains an accumulator variable and updates it for each element, resulting in **O(n)** time complexity.
- Java 1.5 implements running totals using a simple loop and an accumulator variable.
- Java 8 Streams do not include a native running-total operation because it is inherently stateful; a sequential stream with an `AtomicInteger` (or a custom collector) can be used when appropriate.
- SQL provides elegant support through **window functions**, using `SUM(...) OVER (ORDER BY ...)` to compute running totals directly.
- Python offers both a manual loop implementation and the highly readable `itertools.accumulate()` function.
- Running totals are a special case of the **Prefix Sum** algorithm and form the basis for many advanced interview topics such as **running averages**, **sliding windows**, **range-sum queries**, **year-to-date reporting**, and **time-series analytics**.